GEK-83820

DRIVE SYSTEM INSTRUCTIONS

PRELIMINARY

Distributed Micro Controller

Firmware Version 1

User's Manual

Note: This instruction book should be used for Firmware Version 1 only. For Firmware Version 2, use GEK-83839.

DRIVE SYSTEMS OPERATIONS
SALEM, VIRGINIA 24153

GENERAL@ ELECTRIC

PRELIMINARY

DISTRIBUTED MICRO CONTROLLER

USER'S MANUAL

FIRMWARE VERSION 1

GEK-83820

ORIGINAL ISSUE DATE: January 1984

This document contains proprietary information of General Electric Company, USA, and is furnished to the customer solely to assist that customer in the installation, testing, and maintenance of the equipment described herein. This document shall not be reproduced in whole or in part nor shall its contents be disclosed to any third party without the written approval of General Electric Company, Drive Systems Department, Salem, Virginia.

These instructions do not purport to cover all details or variations in equipment nor to provide for every possible contingency to be met during installation, operation, and maintenance. Should further information be desired or should particular problems arise which are not covered sufficiently for the purchaser's purposes, the matter should be referred to General Electric Company, USA.

DIRECTO-MATIC is a registered trademark of General Electric Company.

SILTRON is a trademark of General Electric Company.

Intel is a registered trademark of Intel Corporation.

TABLE OF CONTENTS

Page

CHAPTER 1 - INTRODUCTION
SOFTWARE OVERVIEW 1-1
Segments 1-2
Monitor. 1-2
HARDWARE OVERVIEW 1-3
Memory 1-4
Bus and I/O Structure. 1-4
INTRODUCTION TO CBL 1-5
COMMUNICATION WITH EXTERNAL CONTROLLERS AND COMPUTERS 1-7

CHAPTER 2 - DMC OPERATION
HARDWARE STRUCTURE. 2-1
MEMORY ORGANIZATION 2-3
DATA TYPES. 2-5
FLOATING POINT SUPPORT. 2-7
CONTROL PROGRAM SEGMENTS. 2-7
Periodic Segments. 2-7
Interrupt Segments 2-8
Control Program Interpreter. 2-9
Elementary Instructions. 2-9
SYSTEM I/O SUPPORT. 2-10
SIGNAL IDENTIFICATION 2-11
I/O TYPES 2-12
Module/Local I/O (HADA). 2-12
HADA Configuration Information. 2-13
F-bus Mapping into L-bus. 2-14
F-bus to DMC Address Mapping. 2-15
DS3800HMHA (DS3815PCLA) With DS3800 HSHA Slave 2-15
HSHA Configuration Requirements 2-16
HSHA F-bus Mapping. 2-17
HMHA Dual-ported RAM Strappings 2-17
HMHA With Master DS3800HIOB Slave. 2-18
Configuration Requirements. 2-18
HMHA Master With HMHA Slave (DS3815PRBA) 2-18
Configuration Requirements. 2-18
C-bus Virtual I/O Using DS3800HXMA Dual-Ported Memory Board. . . 2-19
Configuration Requirements. 2-19
Control Signal Highway (CSH) I/O Using DS3800HRMB. 2-19
Configuration Requirements. 2-19
Global I/O 2-20
Configuration Requirements. 2-20

TABLE OF CONTENTS (Continued)

Page

I/O TYPES (Continued)
HMHA Slave With HMHA Master. 2-20
Configuration Requirements. 2-20
HMHA Dual-ported RAM Strappings 2-21
C-bus Master Using DS3800HXCA. 2-21
Configuration Requirements. 2-22
INTERRUPT CONFIGURATIONS. 2-23
SYSTEM CONFIGURATIONS 2-25
Minimal DMC System 2-25
Standard Memory Configuration. 2-26
Expanded Memory Configuration. 2-27
Interrupt Expansion. 2-28
Local I/O Configuration. 2-29
Master DMC To CPL I/O Slave. 2-30
CPL Master DMC To I/O Slave DMC. 2-32
CSH Slave Station Interface. 2-33
C-Bus Virtual I/O To Another Controller. 2-34
C-BusMaster 2-35

CHAPTER 3 - DMC MONITOR
INTRODUCTION 3-l
GENERAL TERMINAL CHARACTERISTICS. 3-l
System Set-Up. 3-2
SYSTEM SELF-TESTS 3-3
Failure Diagnosing Without A Local Terminal. 3-6
Bus Time-Outs. 3-6
ACTIVATING THE MONITOR. 3-7
COMMAND STRUCTURE 3-7
Memory Access Levels 3-8
SIGNAL REFERENCES 3-9
Signal Address 3-9
Signal Name 3-9
MONITOR COMMANDS AND SUBCOMMANDS. 3-10
A - ASSIGN SYMBOLS Command 3-10
H - HELP INFORMATION. 3-10
A - ASSIGN SYMBOL 3-11
D - DISPLAY SYMBOL OR ADDRESS 3-11
L - LIST ALL SYMBOL ASSIGNEMTS. 3-11
N - DISPLAY NEXT SYMBOL 3-12
P - DISPLAY PREVIOUS SYMBOL 3-12
R - RELEASE SYMBOL. 3-12
Q -QUIT 3-12

TABLE OF CONTENTS (Continued)

Page

MONITOR COMMANDS AND SUBCOMMANDS (Continued)

B - BREAKPOINT Command 3-13
C - CBL BLOCK LIBRARY Command. 3-14
H - HELP INFORMATION. 3-14
D - DISPLAY CBL BLOCKS. 3-14
Q -QUIT 3-14
D - DISPLAY AND MODIFY VARIABLE Command. 3-15
H - HELP INFORMATION. 3-15
A - ADJUST VALUE FOR REAL 3-16
D - DISPLAY VARIABLE. 3-16
E - ENTER MULTIPLE VARIABLES. 3-17
F - FAST DISPLAY. 3-17
M - MODIFY VARIABLE 3-18
R - RUN MULTIPLE VARIABLE DISPLAY 3-18
S - STATION I/O DISPLAY 3-19
+ - ADD ADJUSTMENT VALUE. 3-19
- - SUBTRACT ADJUSTMENT VALUE 3-20
Q -QUIT 3-20
E - TARGET EDITOR Command. 3-21
H - HELP INFORMATION. 3-22
B - BLOCK DISPLAY 3-23
D - DELETE INSTRUCTION. 3-23
I - INSERT INSTRUCTION. 3-24
L - DISPLAY LINE. 3-25
M - MODIFY LINE 3-25
N - DISPLAY NEXT BLOCK. 3-26
P - DISPLAY PREVIOUS BLOCK. 3-26
S - DISPLAY THE ENTIRRE SEGMENT 3-27
+ - DISPLAY NEXT LINE 3-27
- - DISPLAY PREVIOUS LINE 3-28
Q -QUIT 3-28
F - FORCE LOGICAL SIGNALS Command. 3-29
H - HELP INFORMATION. . . . . .
C - CLEAR ALL FORCED POINTS . . . . . .
D - DISPLAY LOGICAL 3-30
F - FORCE LOGICAL 3-30
N - NEXT FORCED STATE 3-30
P - PREVIOUS FORCED STATE 3-31
R - RELEASE FORCED STATE. 3-31
Q -QUIT 3-31
H - HELP Command 3-32
Q -QUIT 3-32

TABLE OF CONTENTS (Continued)

	Page
MONITOR COMMANDS AND SUBCOMMANDS (Continued)	
I - I/O Command.	.3-33
H - HELP INFORMATION.	.3-33
C - DISPLAY CHANNEL INFORMATION	.3-33
M - MODIFY RACK INFORMATION	.3-34
S - STATION DISPLAY	.3-34
Q -QUIT	.3-34
L - LOOK Command	.3-35
M - MODIFY VARIABLE Command.	.3-37
0 - ONLINE STATUS Command.	.3-38
H - HELP INFORMATION.	.3-38
C - CONTROLLER STATUS	.3-38
D - DISPLAY CURRENT ONLINE STATUS	.3-39
I - I/O CHANNEL STATUS.	.3-40
0 - OUTPUT STATUS	.3-40
S - SEGMENT STATUS CONTROL.	.3-41
Q - QUIT	.3-41
P - PASSWORD Command	.3-42
Q - QUIT MONITOR command	.3-43
S - SEGMENT CONTROL Command.	.3-44
H - HELP INFORMATION.	.3-44
D - DISPLAY SEGMENTS.	.3-44
M - MODIFY RUN SEGMENT CYCLE.	.3-45
Q -QUIT	.3-45
T - TERMINAL SUPPORT Command	.3-46
H - HELP INFORMATION.	.3-46
B - SET TERMINAL BAUD RATE.	.3-46
L - LINE MODE (HANDHELD).	.3-47
S - SCROLL MODE (CRT)	.3-47
Q - QUIT	.3-47
U - USER MONITOR command	.3-48
W - WHERE USED Command	.3-49
H - HELP INFORMATION.	.3-49
F -FIND	.3-50
N - NEXT WHERE USED LOCATION.	.3-50
P - PREVIOUS WHERE USED LOCATION.	.3-51
Q - QUIT	.3-51

TABLE OF CONTENTS (Continued)

Page

MONITOR COMMANDS AND SUBCOMMANDS (Continued)
X - TRANSFER SYSTEM Command. 3-52
H - HELP INFORMATION. 3-52
C - SAVE .C TABLE 3-53
D - VAX DOWNLOAD. 3-53
R - RESTORE MEMORY FROM EEPROM. 3-55
S - SAVE MEMORY ON EEPROM 3-57
T - STR LNK II OR III TAPE BACKUP 3-57
U - VAX UPLOAD. 3-60
Q - QUIT. 3-60
MONITOR ERROR MESSAGES. 3-61

APPENDIX A - PRINTED WIRING BOARD AND MODULE DESCRIPTIONS . . . . . A-l
CONTROLLER BOARDS A-2
I,'OBOARDS . . . . . A- 6
MODULES A-8

APPENDIX B - GLOSSARY OF TERMS. . . , ... B-l

APPENDIX C - DMC MEMORY AND I/O LAYOUT. . . . . C-l

APPENDIX D - BREAKPOINT COMMAND . . . . . D-l
H - HELP INFORMATION Subcommand . . . . . D-l
B - BASE SEGMENT ASSIGNMENT Subcommand. D-2
C - CHECKSUM ERROE COUNT Subcommand D-2
D - DISPLAY ABSOLUTE MEMORY Subcommand. D-2
E - EXECUTE BREAKPOINT Subcommand D-4
M - MODIFY ABSOLUTE MEMORY Subcommand D-5
M - SINGLE STEP subcommand. D-5
T - TEST MEMORY Subcommand. D-6
Q - QUIT Subcommand D-7

APPENDIX E - LOCAL TERMINAL CHARACTERISTICS AND CONNECTIONS . . . . . E-l
LOCAL TERMINAL CHARACTERISTICS. . . . . E-l
TYPICAL LOCAL TERMINAL CONNECTIONS. . . . . E-l

LIST OF ILLUSTRATIONS AND TABLES

Figure	Title	Page
l-l	DMC Functional Hardware Elements.	1-3
l-2	Summer and Integrator	1-5
1-3	ACIN, Symbolic AND Statement.	1-6
l-4	Logic Operation	1-7
2-l	Simplified DMC Memory Map	2-2
2-2	Data Types.	2-6
2-3	Control Program Interpreter	2-9
2-4	I/O Reference Form.	2-11
2-5	Example of CPL Application Program View of I/O.	2-13
2-6	F-bus Mapping Into the L-bus Address.	2-14
2-7	F-bus Relationship to Rack/Byte For DMC	2-16
2-8	Segment 8 to DMC Address Space.	2-20
2-9	F-bus Relationship to Rack/Byte For DMC	2-22
2-10	HXCA To L-bus Mapping	2-22
2-11	System With 4 Interrupts.	2-23
2-12	System With 11 Interrupts	2-23
2-13	System With 18 Interrupts	2-24
2-14	Minimal Memory Configuration.	2-25
2-15	Standard Memory Configuration	2-26
2-16	Expanded Memory Configuration	2-27
2-17	Interrupt Expansion	2-28
2-18	Local I/O Configuration	2-29
2-19	CPL Master To I/O Slave	2-30
2-20	CPL Master To Slave	2-32
2-21	CSH Slave Station Interface	2-33
2-22	C-Bus Virtual I/O	2-34
2-23	Example DMC To C-bus I/O.	2-35
C-l	DMC Memory Map.	C-2
c-2	Processor I/O	C-3
E-l	DMC Connection to a "DTE" Type Device	E-2
E-2	DMC Connection to a "DCE" Type Device	E-3
E-3	DMC Connection to a TERMIFLEX HT/11	E-3

Table	Title	Page
2-l	Example: F-bus to DMC Address Mapping	2-15
2-2	Example: HSHA F-bus Mapping	2-17
2-3	HMHA Address Strapping When Using HSHA Slave.	2-17
2-4	HMHA Address Strapping When Using HMHA Slave.	2-21
3-l	Tape Storage Transfer Times	3-58
E-l	Typical Local Terminal Connections.	E-2

NOTES

NOTES

-- CHAPTER 1 --
INTRODUCTION

General Electric Company's Distributed Micro Controller (DMC) is an 8086 microprocessor-based controller that performs decision making and arithmetic functions under software control. The DMC is part of the DIRECTO-MATIC* Plus family of integrated industrial control equipment, and is designed specifically for analog type regulating applications. The user can easily modify programs at the DMC through an RS-232 interface. Application software is Written in General Electric's Control Block Language (CBL), which simplies programming for analog functions.

The DMC is a standard DIRECTO-MATIC Plus electronics module containing printed wiring boards configured according to the application. The DMC incorporates a special control-oriented, ROM-based operating system to control operation and to support field program modifications. The design includes extensive board-level check and diagnostic routines. The DMC communicates with the controlled process and drives via a standard DIRECTO-MATIC Plus I/O system.

SOFTWARE OVERVIEW

The typical programmable controller operates by reading the input status of external signals (such as limit switches), executing the programmed logic, and then sending the resulting states to output boards, which perform the appropriate function. This cycle of operation is called a 'sweep'. The DMC operates similarly, except that it can concurrently execute more than one sweep, or program. The typical program segments operate at fixed sweep rates of 20 to 40 times per second, which includes I/O transfers. During the sweep time, the DMC honors interrupts to *service* high response control requirements.

The DMC software consists of the following major elements:

- CBL application program - A CBL program that defines the control functions for the specific application. The program is organized into segments, a group of CBL blocks that perform a specific function. The program provides a series of elementary instructions and calls to the control block library (defined below). Segments can be edited by the the DMC's editor.

  - The CBL Block is a reference within the segment to a specific control block library routine. The CBL block consists of the library reference and a list of parameters for the routine.

  - An Elementary Instruction is a logical operation (such as LOAD, NOT, AND, OR, and STORE). These operations are used to evaluate a segment's relay network portions. The elementary instructions are supported on all systems and do not require any support from the control block library.

* DIRECTO-MATIC is a registered trademark of General Electric Company.

- Control Block Library - A collection of routines that define the functional operation of each control block (the way in which each block simulates its hardware equivalent). They can be executed multiple times during a segment. The contents of the CBL library are set at system configuration and cannot be modified on the DMC.

- Monitor - A program that enables a user to inspect and modify the DMC's signals and CBL program via a local terminal.

- Product Executive - The program that manages the DMC's overall processor resources, input/output, communications, etc.

Segments

The DMC's executive routine (product executive) manages the execution of all segments of the application program. It executes the segments at particular sweep rates, and executes CBL blocks within the segments in block number order. Segments can also be enabled and disabled as a function of the control process.

For the DMC, there are 3 basic types of segments:

- Periodic (or priority) - Segments whose execution is based on time. A periodic segment runs at an assigned sweep rate. The segments can be set to start at cyclic intervals of from 1 millisecond to 100 days, although 20 to 100 milliseconds is typical. Additionally, a priority is set for periodic segments so that if 2 segments are scheduled to run at the same time, the higher priority segment executes first.

- Interrupt - Segments whose execution is based on an external event that creates a hardware interrupt to the DMC processor. The DMC software can be configured to handle up to 16 external interrupts.

- Initialize - Segments that execute upon power-up and restart to initialize the DMC.

Monitor

The DMC provides a monitor interface with over 100 diagnostic and maintenance commands. The monitor enables the user to display and modify external I/O variables, display and modify CBL application programs, and upload/download DMC systems. The monitor is accessed via a user terminal connected to the DMC. (Chapter 3 describes the monitor commands and their use.)

HARDWARE OVERVIEW

The DMC hardware is a combination of standard DIRECTO-MATIC Plus boards configured into a module as required by the application. (Appendix A lists and defines all the printed wiring boards used in the DMC.) The module can be 1 of 3 sizes: the 6-board electronics module (TEM), the 12-board electonics module (SEM), and the 24-board electonics module (MEM). The selected module size depends on the memory and I/O requirements for the system. The module consists of 3 major functional hardware elements, shown in Figure l-l and defined as follows:

- Microprocessor board - The DS3800HMPH, which contains an 8086 microprocessor, an optional 8087 co-processor daughter board for handling floating point operations, on-board PROM with product software for the DMC operating system, RAM, and an RS-232 port for connecting a user terminal.

- Memory board(s) - All DMC configurations include at least one DS3800HUMA memory board, and may include other types of memory boards. These boards store user changeable software such as the CBL application program, internal status and variables (register table), the CBL control block library, etc.

- I/O and Communication boards - The board type depends on the function for which the DMC was designed. These boards include I/O boards within the module, drive boards for externally located I/O boards, boards for interfacing other modules, and special purpose boards.

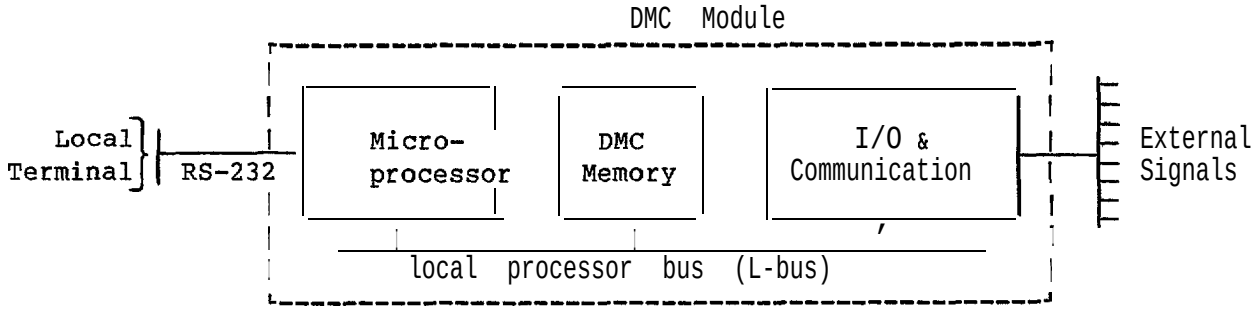


Figure l-l
DMC FUNCTIONAL HARDWARE ELEMENTS

Memory

For memory, the standard DMC uses the HUMA, a combination memory board that can include RAM and EEPROM.

Upon application of power, the DMC automatically initializes RAM from the permanent program storage in EEPROM, and then executes its program from RAM. This technique provides non-volatile storage for the program. Since the CBL program resides in RAM during operation, the user can easily modify it through the built-in monitor (see Chapter 3). After any changes, the DMC copies the modified program from RAM to EEPROM.

Memory segments can also be copied onto spare EEPROM boards, allowing quick replacement of malfunctioning memory boards. Additionally, the DMC can contain other types of memory boards, such as a battery-backed RAM board for saving dynamic information during power interruption.

The DMC can load programs to and from a magnetic tape cassette via the HMPH's RS-232 port connected to a portable tape unit. For systems which include VAX* control computers, the DMC can load the program to and from the computer if the appropriate VAX software is available.

Bus and I/O Structure

The HMPH communicates with the DMC's internal boards via the backplane's Local bus (L-bus). The DMC can optionally include an Extended bus (E-bus) to a dual ported memory for communication with the Control Signal Highway (CSH). I/O boards within the DMC use an F-bus and an R-bus (used most often). For external I/O, the DMC can use the Control Party Line (CPL) and the c-bus for the Series Six interface. (For definition of the bus structures, CSH, CPL, and Series Six, refer to Appendix B.)

The DMC I/O and communication boards can provide the following options through external signal connections:

- Control communication with a process computer via a 'virtual memory' port to the CSH. This function provides in-sweep communication of control signals treated as I/O in both the computer and in the DMC.
- Simple communication module that operates to 9600 baud with an RS-232C interface. Some systems use this for communication with a display terminal or an entry function panel.

* VAX is a trademark of Digital Equipment Corporation.

INTRODUCTION TO CBL

The DMC application program is written and modified using a simple, block-oriented language called Control Block Language, or CBL. CBL allows control functions to be represented in functional analog hardware blocks. Thus, a user familiar with control can readily modify control functions as required, without the need for programming in the 8086 microprocessor language. The user communicates with the DMC through either a simple hand-held terminal, a conventional teleprinter, or a CRT. The system stores the CBL program in either BRAM (battery-backed random access memory) or EEPROM (electrically erasable read-only memory). Program copies can be maintained on tape cartridge or via phone line to a remote computer.

CBL programs intentionally follow the pattern of the analog hardware world. For example, a 'system elementary* uses a symbolic block to represent an analog regulator. In a like manner, CBL system diagrams describe the operating characteristics of a software operation. For example, the circuit represented in Figure l-2 shows an analog 'summer' and an 'integrator' interconnected much as they would be in a typical hardware elementary.

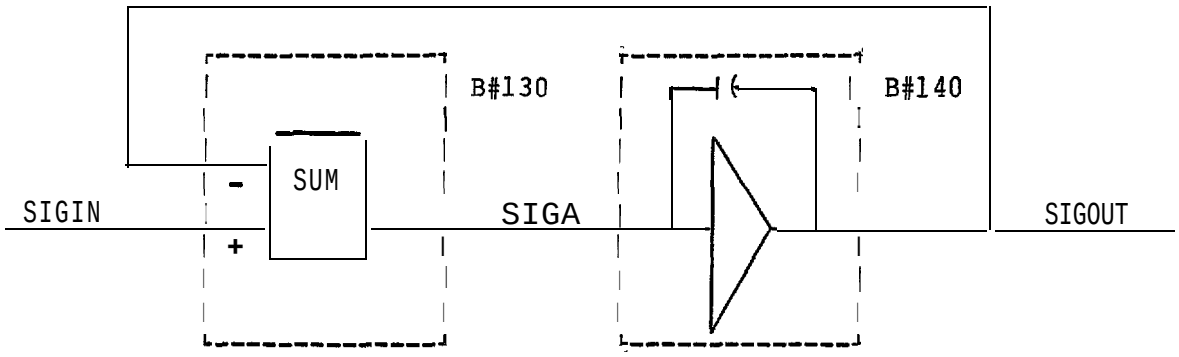


Figure 1-2
SUMMER AND INTEGRATOR

The diagram indicates that input signal SIGIN is fed to the summer, then the result (SIGA) is fed into the integrator. The integrator's output (SIGOUT) is used for control purposes, but also loops back to a subtracting input on the summer.

Figure l-2 shows 2 DMC control blocks represented as symbols and described as a 'signal flow diagram'. The lines connecting the blocks represent the signal flow, and are equivalent to the wires interconnecting functional hardware elements. Control functions performed by the DMC can be documented by using such a signal flow diagram, which is the functional equivalent of a typical hardware system elementary.

Also, note that a block number (B#130, B#140) represents each block in Figure l-2. This number is the reference number that identifies the block in the system. It also controls the execution order of the blocks, which are executed in numeric order by block number.

The DMC is designed to allow use of simple terminals for examining and editing CBL programs in a field or control room environment. Since many such terminals cannot display control blocks in 'symbolic form', the DMC accepts a corresponding 'CBL statement form' of the control block, The user can easily relate the symbolic control block shown on a signal flow diagram with the statements required by the field terminals.

For example, Figure l-3 shows a simplified Clamped Integrator in both the symbolic form and the corresponding CBL statement form. In the symbolic form, Block Number 123 (B#123) represents a simplified Analog Clamped Integrator (ACIN) Block. In the CBL statement form, a statement represents each part of the block. The symbolic form shows the integrator input signal (SIGA) connected to pin 0; the statement form indicates this connection as "P-0 SIGA". In the same ways, both forms represent the output signal (SIGOUT), the low clamp value (LOCLP), and the high clamp value (HICLP).

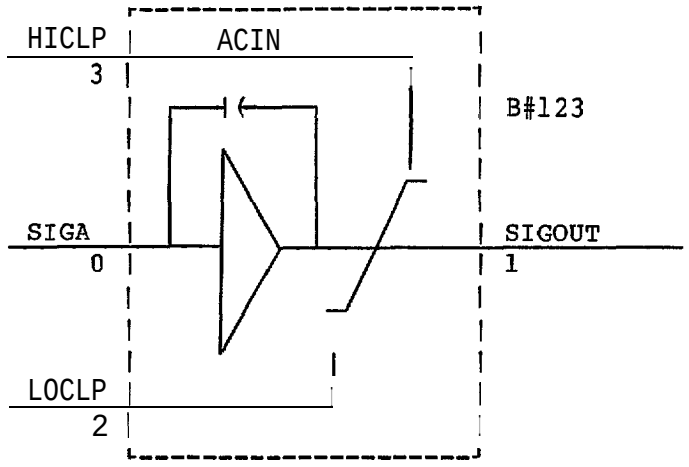


Figure l-3
ACIN, SYMBOLIC AND STATEMENT FORMS

In addition to handling control blocks that represent traditional analog functions, the DMC provides basic logic functions and special operations. Figure 1-4 shows a basic logic operation represented in 'relay ladder diagram form' and in corresponding 'CBL statement form'.

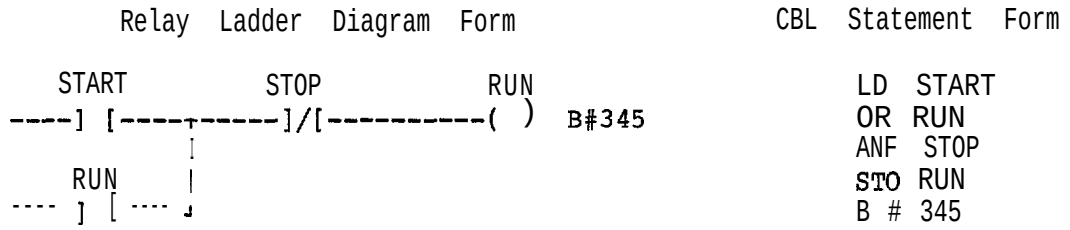


Figure 1-4
LOGIC OPERATION

In Figure 1-4, 'relay ladder rung' 345 (Block # 345), uses a START and a STOP function with a relay RUN, which seals itself up. The corresponding elementary logic statements can be read as follows:

1) LOAD with signal START
2) OR the result with signal RUN
3) AND the false state of signal STOP
4) STORE the result in signal RUN for 'block number' 345

Each program (segment) within the DMC consists of 1 or many of the above blocks and elementary instructions. Each CBL block can be called multiple times from multiple segments, with each call supplying its own set of parameters.

COMMUNICATION WITH EXTERNAL CONTROLLERS AND COMPUTERS

The DMC is often included in systems with Series Six programmable controllers and computers (see Appendix B). The DMC communicates with Series Six systems, which use the C-bus, through a virtual I/O interface board (described later). In a similar way, the DMC communicates with control computers through a virtual I/O port to the Control Signal Highway.

Additionally, the DMC communicates with a VAX computer used by General Electric during system configuration to generate the DMC software. The DMC monitor contains commands for uploading and downloading programs between this computer and the DMC.

NOTES

-- CHAPTER 2 --
DMC OPERATION

HARDWARE STRUCTURE

The typical DMC comprises DIRECTO-MATIC Plus printed wiring boards mounted in a small or medium electronics module (SEM or MEM). Chapter 1 provides an overview of the boards' functions. The following list typifies the boards included in a DMC configuration.

Microprocessor and optional boards:

DS3815PLCA - DS3800HMPH with DMC Version 1 firmware
- Optionally includes DS3800DMPC daughter board

DS3800HXPC - Contains interrupt, counter/timer, logic I/O, and memory expansion

Memory boards (examples):

DS3810RAM - DS3800HUMA memory board with RAM devices, 32k bytes
DS3810EEP - DS3800HUMA memory board with EEPROM devices, 32k bytes
DS3800RAMB - 8K byte memory board with battery back-up

I/O and communication boards (examples):

DS3800HADA - L-bus to F-bus decode and drive
DS3800HBDA - Buffer decode
DS3800__ - Various R-bus input and output boards
DS3800HXCA - C-bus master board
DS3800HXMA - Dual ported memory board for virtual I/O
DS3815PCLA - DS3800HMHA board with CPL master station firmware
DS3815PRBA - DS3800HMHA board with CPL slave station firmware
DS3815PTCA - DS3800HCMA board with terminal firmware

This chapter's "I/O TYPES" section contains block diagrams of typical DMC system configurations.

****
* WARNING *
****

DO NOT PLUG OR UNPLUG THE DIRECTO-MATIC PLUS BOARDS IN THIS DMC MODULE UNDER POWER. SUCH ACTIONS CAN SIGNIFICANTLY AFFECT SYSTEM OPERATION AND MAY RESULT IN PERSONAL INJURY OR MAY ADVERSELY AFFECT ANY EQUIPMENT CURRENTLY BEING CONTROLLED. SUCH ACTIONS CAN INTRODUCE BUS TRANSIENTS THAT MAY CHANGE THE CONTENTS OF ANY RAM IN THE SYSTEM, SUCH AS THOSE CONTAINING THE DMC'S PROGRAM OR DATA.

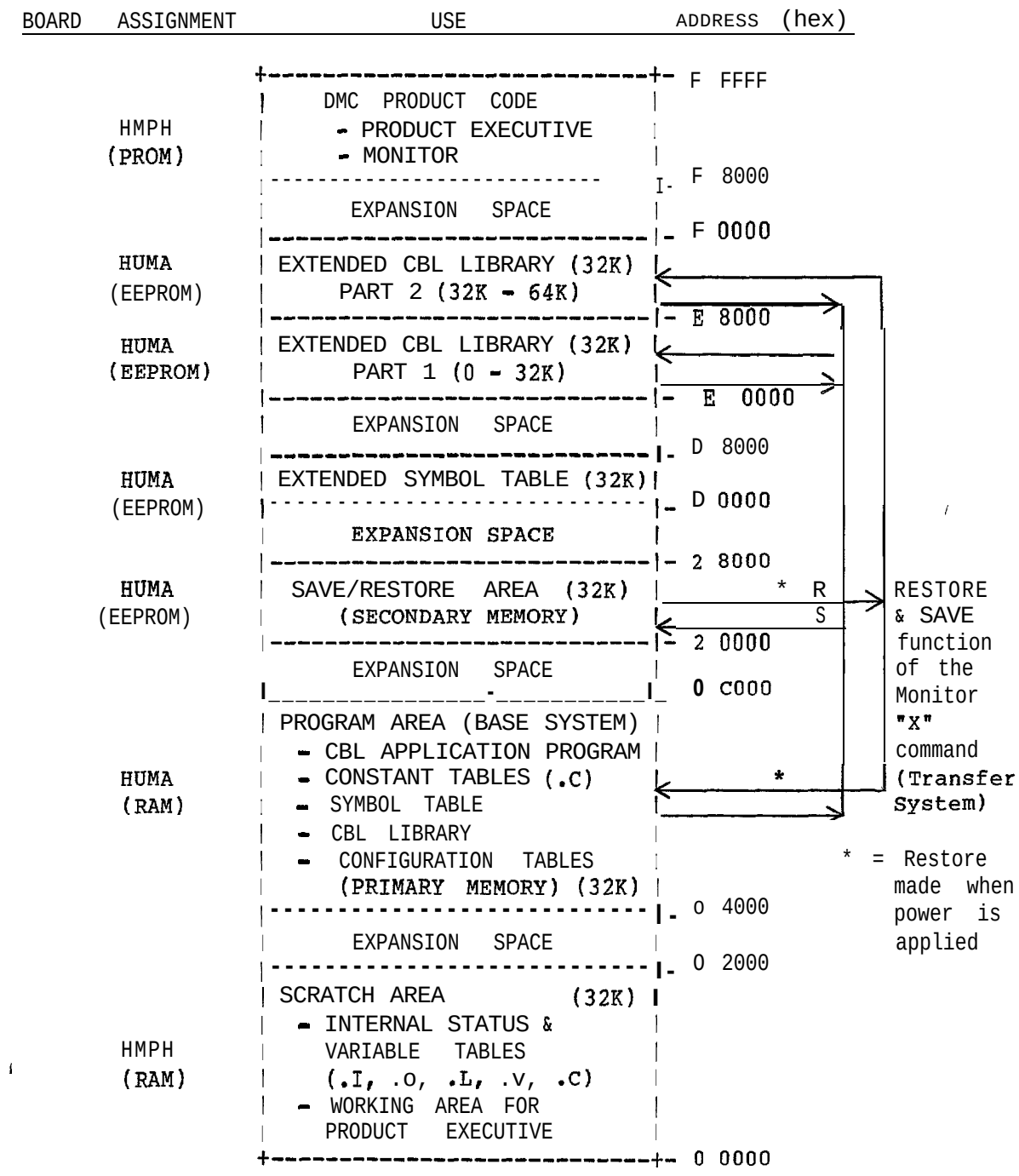


Figure 2-l
SIMPLIFIED DMC MEMORY MAP

MEMORY ORGANIZATION

The DMC's memory provides a variety of functions, as shown by Figure 2-1, a simplified DMC memory map. (For a more detailed memory and processor I/O map, refer to Appendix C.) The following text describes the function of the various memory areas (refer to Figure 2-1).

Program Area - Contains program type information that can be modified but is not dynamically changed. This area resides in RAM (primary memory) for execution, but is shadowed in EEPROM (secondary memory).

- CBL Application Program - Control block instructions partitioned into segments that the CBL system interpreter executes. Each entry is 1 word long. Each elementary instruction requires 1 entry, while a CBL block requires 2 entries to call the control block library routine and 1 entry for each parameter the block requires.

- Symbol Table - This table contains signal names and corresponding signal addresses. Each name is limited to 7 characters, and each entry is 5 words long. The symbol table's size is optional. The table may include a secondary EEPROM that holds up to 3200 symbols.

- Control Block Library - The collection of CBL routines, which are called by the CBL application program during segment execution. These routines are written in 8086 machine language. The particular CBL blocks included in the CBL library are optional and are determined at system configuration. If the total length of all the routines is under 16K, the library resides on the base system EEPROM board and the routines are called directly. When the size exceeds 16K, the library resides on a secondary EEPROM board and can be as large as 64K. With the secondary memory, the interpreter requires 5 additional microseconds to access every routine it calls.

- Configuration Tables

  - Segment Definition Table - This table defines the name of each control block program segment defined to the system, and that segment's execution period. The system uses this table to determine the location of the first instruction of the segment.

  - I/O Definition Table - This table contains the definition of all the external I/O. It is used during system initialization for I/O board self test and configuration.

- Internal Constants Table - This table contains the initial version of the Constant Storage Table.

Scratch Area - RAM areas for dynamically changing data.

- Constant Storage Table (.C Table, or CONST) - This table defines any internal constants. At power up, the DMC loads the initial state of these constants from the Internal Constants Table located in EEPROM. Symbolic names can be assigned to locations in this table. The constants' default names are written in the form ".Cxxxxt", where t is the variable's data type and xxxx is a hexadecimal constant specifying the byte offset into the table. (See "type" discussion below.)

- Variable Storage Table (.V Table, or VARTAB) - This table defines all internal variables, except logicals. At power up, the DMC sets this table zero. Symbolic names can be assigned to any location in the table. The variables' default names are written in the form ".Vxxxxt", where "t" is the variable type and "xxxx" is the byte offset

- Logical Storage Table (.L Table, or LOGTAB) - This table contains all internal logical variables. (An internal logical is any logical without an input or output byte mapped to it.) A symbolic name or a default name can represent a logical variable. A default name is written in the form ".Lxxxx" where "xxxx" is an offset into the table. On power up, the DMC sets the logical variables to false.

- Input Status Table (.I Table, or INTAB) - This table contains the status of all input signals. The I/O Definition Table (described above) defines the I/O. A symbolic name or a default name can represent these variables. The default name is written in the form "Issrbt". (For definition and more information, see the "System I/O Support" section in this chapter.) On power up, the DMC sets all I/O bytes to a logical 0.

- Output Status Table (.O Table, or OUTTAB) - This table contains the status of all output signals, and has the same format as' the input table. The default name for a table entry is written in the form ".Ossrbt". (For definition and more information, see the "System I/O Support" section in this chapter.)

DATA TYPES

The DMC accepts several data types that can be assigned to variables, constants, or external I/O. The following text lists and describes the data types, and Figure 2-2 illustrates their relative size.

- LOGICAL, type I and 0 - The logical element is an entry from LOGTAB, INTAB, and OUTTAB. It represents one bit of data. This data type can be referenced by a symbolic name ".Lxxxx", or by an INPUT or OUTPUT table reference with a data type of 0 - 7. References to the I/O logicals should still specify the I/O tables as their table type (see Note). When the I/O is executed, the DMC automatically expands the I/O table logicals into LOGTAB.

  NOTE: The DMC maintains the logicals, .I and .O, in a separate table from the actual inputs and outputs.

- BYTE or GROUP, type G - The byte variable contains 8 data bits. The byte can have a value of 0 through FF (hexadecimal).

- COUNT (SIGNED INTEGER), type C - The signed integer variable contains 16 bits, with 1 bit reserved for the sign bit. This gives the integer a range from -32,768 to +32,767. All data tables except LOGTAB (i.e. CONST, VARTAB, INTAB and OUTTAB) can define an integer. An integer value is defined by giving a variable a "type" value of "C". The value requires 1 word (2 bytes) of storage within the designated table.

- BINARY CODED DECIMAL, type B - The BCD variable is 16 bits wide with no implied sign bit, requiring 1 word (2 bytes) of storage within the designated table. Thus, the BCD number ranges from 0 to 9999, or 4 decimal digits. The BCD number is entered and displayed as a HEX number (digits A through F is recognized as a valid BCD digit). A BCD value is defined by giving a variable a "type" value of "B". All data tables except LOGTAB can define a BCD digit.

- FLOATING POINT, type R - The binary floating point variable is 32 bits wide and requires 2 words (4 bytes) of storage within a table. The 32 bits include 1 bit reserved for sign, 8 bits reserved for exponent, and 23 bits reserved for normalized fraction. Floating point variables within a system always require the 8087 coprocessor* Any data table except LOGTAB can define a floating point number, which has a "type" descriptor of "R". The DISPLAY AND MODIFY commands (see Chapter 3) display floating point numbers as 8-digit real numbers. This limits these numbers' display range from 10 E-7 to 10 E+7. There *is* no exponential display. Note that the fraction contains 24 bits of significance within the fraction because there is an implied "1" to the left of the decimal point. This value ranges from 1.10 E-38 to 3.37 E38.

- FIXED BINARY POINT, type F - The fixed binary point number is 16 bits wide, requiring one word (2 bytes) of storage within a table. The number can be scaled with a binary scale of from -12 to 15 (binary), with the range limited only for display and modify reasons. The application software must relocate the binary point whenever an arithmetic operation occurs on the value in a control block library routine. The DMC does not store the binary point with the value, but requests it any time the value is displayed or modified. Any table except LOGTAB can define a fixed binary point number, which has a "type" descriptor of "F".

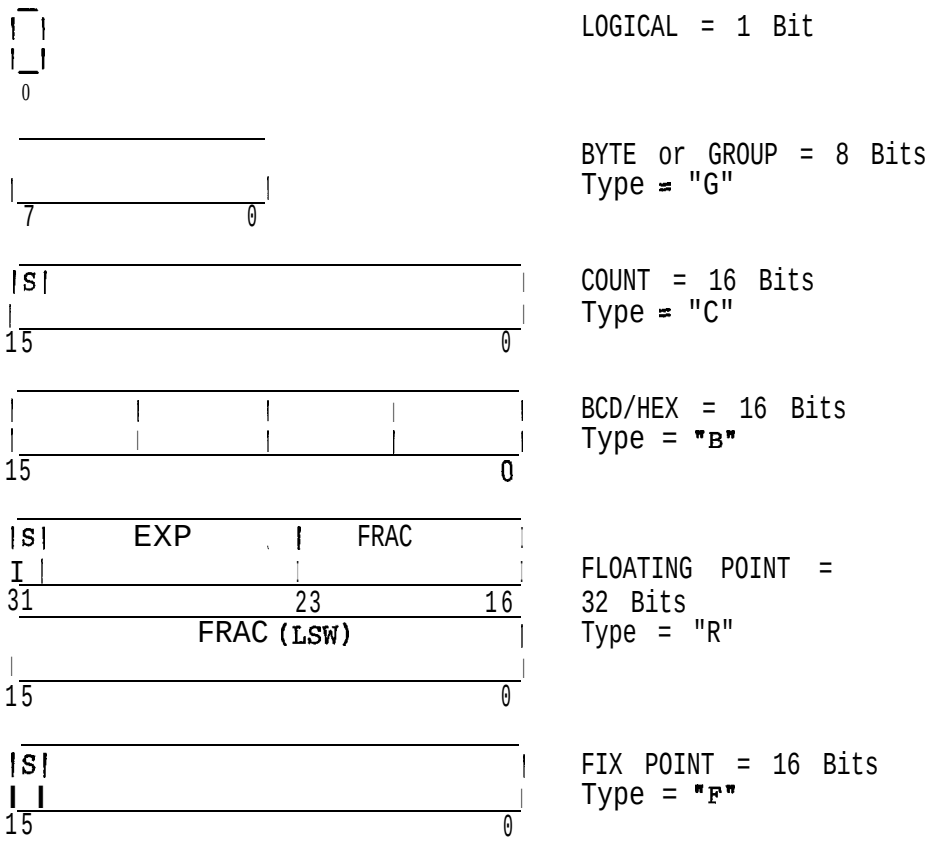


Figure 2-2
DATA TYPES

FLOATING'POINT SUPPORT

The DMC optionally supports floating point binary numbers, which require the 8087 co-processor. The 8087 resides on the DS3800DMPC, the HMPH daughter board, Segments are optionally configured use the floating capability. The floating point function is required for all analog CBL blocks.

CONTROL PROGRAM SEGMENTS

A control program segment (defined in Chapter 1) is a sequence of interpreter instructions within the CBL application program's memory space. A configuration parameter defines its beginning, and an end of segment CBL block terminates it.

Only 1 segment initiates the actual input and output to a particular I/O channel. The other segments can reference the I/O, but cannot initiate it. The segment executes an I/O control block to perform the I/O. A segment can perform I/O on multiple channels, but multiple segments cannot perform I/O on any 1 channel.

Periodic Segments

Each segment's time interval and priority is established during system configuration. The base time interval is a system parameter, and each segment can be run at a multiple of that base interval. Segments can be scheduled with a cyclic rate of once every 1 millisecond up to a every 100 days. A scan rate set at "zero" causes the segment to not be scheduled on a periodic basis.

Multiple segments can' have the same scan interval, but do not always run at the exact scan rate specified. The highest priority segment runs first, and when it completes, the next highest segment runs. The segment priority is set by the order that the segments are specified during configuration.

NOTE: It is important to incorporate within high priority segments the tasks that need to run at very strict intervals.

A lower priority segment is preempted during its execution if a higher priority segment becomes active at that time. A control block library routine can disable the preemption, but should only be used if a control block program cannot be interrupted during execution. The preemption is enabled if the segment terminates or suspends.

Interrupt Segments

Segments can be assigned during system configuration to an external interrupt so that they run only when the associated interrupt occurs. When it receives an interrupt, the DMC disables all other interrupts and executes the associated segment. Once initiated, interrupt segments run until completion. No other segment can run until the interrupt segment completes.

Interrupt segments have no priority, but an interrupt level is assigned to each of these segments during system configuration. To prevent other interrupts from being missed, interrupt segments are designed with minimum execution time. When required, the interrupt segment activates periodic segments to perform extended work. In such a case, the periodic segment is set up with no scan so that it executes only when the interrupt segment starts it.

Sometimes segments must suspend execution of their control block program while waiting for an external event to occur. For example, a control block library routine that is processing console input may suspend operation until it gets an input interrupt.

When an interrupt segment calls a CBL block that requires the segment to be suspended, the DMC aborts the CBL block. This occurs because an interrupt segment cannot suspend itself. An interrupt segment cannot call the channel I/O block, zrooo. Therefore, the DMC executes any needed I/O as part Of direct access CBL blocks within the interrupt segment.

Since the DMC can preempt a segment at any time, unpredictable values can occur in locations shared by periodic and interrupt segments. The CBL application program is designed to compensate to prevent these errors. Since an interrupt segment cannot be interrupted, no special code is required when modifying the memory content in an interrupt segment. However,, since a periodic segment can be preempted while reading or writing to a memory location, the DMC uses CBL block ZDINTOO to disable interrupts during the read or write to shared locations. Interrupts are then enabled after the memory read or write with CBL block ZEINTOO.

The basic DMC system provides 4 external interrupts, which can be increased by adding hardware. (For more information, refer to the "Interrupt Configurations" section in this chapter.)

Control Program Intepreter

The control program interpreter (see Figure 2-3) is the DMC system driver, and is a reentrant routine executed for every active segment. The interpreter uses CODTAB, the interpreter instruction table. Once the interpreter starts, it continues until it reaches an end-of-segment instruction or is preempted. The control program interpreter executes both elementary instructions and control block library routines.

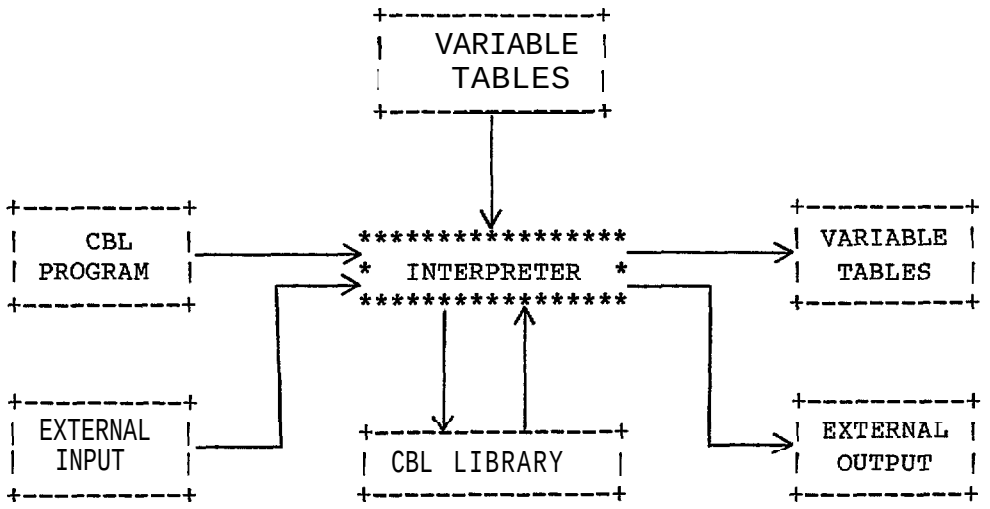


Figure 2-3
CONTROL PROGRAM INTERPRETER

Elementary Instructions

Elementary instructions provide manipulation of logical variables, and are used in relay ladder diagrams. These instructions operate with a specified logic signal and an internal logic variable stack.

The DMC provides the following elementary instructions:

- NOP - No operation.
- LD - Load logic variable onto stack.
- LDF - Load negated logical variable onto stack.
- AN - "AND" logical variable with stack.
- ANF - "AND" negated logical variable with stack.
- ANS - "AND" top two elements of stack.
- OR - "OR" logical variable with stack.
- ORF - "OR" negated logical variable with stack.
- ORS - "OR" top two elements of stack.
- STO - Store top of stack into logical variable.

These elementary instructions implement relay ladder diagrams (described in Chapter 1).

NOTE: The DMC accepts up to 9 LD or LDF instructions within a block and relay rung. Exceeding this limit can cause unpredictable results.

SYSTEM I/O SUPPORT

The DMC supports a variety of external I/O. I/O data is stored internally in input table (.I table) and the output table (.O table). These tables represent the data from the previous input reading on that particular channel, or the data for the next output on that channel. All CBL program references to I/O data other than logicals are taken from these tables.

As mentioned previously, only 1 CBL program (segment) can execute the I/O on a particular channel, since segments run at various time intervals. A segment can perform I/O on multiple channels, but multiple segments cannot perform I/O on any 1 channel. Any segment can reference data brought into the DMC by another segment.

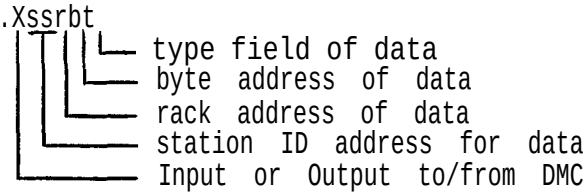
The DMC cannot handle external I/O as logicals directly from the I/O buffers, The I/O byte must be expanded into 8 bytes in the logic storage table (LOGTAB). The DMC performs this expansion and compression at channel I/O time via the logical I/O lookup table. Control for this is created whenever a reference is made to an I/O logical within the editor, or a symbol name is assigned to a logical variable.

The DMC's method of signal identification provides a direct relationsihip between the default name and the physical input or output device. By looking at an I/O reference's default name, a user can tell: (1) whether the reference is for an input or an output, (2) what data type the reference is for, and (3) the physical origin or destination of the signal.

SIGNAL IDENTIFICATION

As previously discussed, the DMC's signal identification scheme mimics the physical structure of I/O hardware found in typical industrial control systems. In most systems, several signals are wired into a board or byte, several bytes are contained in a board rack, several racks are contained in a station, and several stations are coordinated by one DMC.

The DMC default name for an I/O reference is in the form ".Issrbt" for all inputs to the DMC, or ".Ossrbt" for all outputs from the DMC. Figure 2-4 and the following text define the form.



Example:
Default address .ICC023 shows a logical input on station CC, rack 0, byte 2, and point 3 within that byte.

Figure 2-4
I/O REFERENCE FORM

- Station ID address - Limited from 01 to FE hexadecimal. The station is a logical major grouping of signals, and can be either physical or virtual: Physical stations consist of hardware (typically up to 16 racks) mounted in one panel or distributed among several panels; virtual stations exist only within the DMC's memory and are used for transferring data between DMCs or from a DMC to a host computer.
- Rack address of data - A hexadecimal number between 0-F. A rack is typically a board rack (electonics module or field termination module) containing several I/O boards. A rack can contain up to 16 bytes of inputs and 16 bytes of output.

- Byte address of data - A hexadecimal number between 0-F. A byte is usually an I/O board containing 8 input or 8 output points. Some I/O boards contain 16 points and are handled as 2 bytes within the DMC.

- Type field of data - An address between 0 - 7 or one of the data types defined earlier. A type of 0 - 7 specifies one of the 8 I/O points on the byte or I/O board. This I/O point is a logical data type, which can specify a numeric data type. When this is done, several additional consecutive I/O bytes can be used to form the data item (1-4 bytes), dependent on type.

I/O TYPES

The DMC handles the following types of external I/O:

- Module/Local I/O - Channels 0, 1
- DS3800HMHA to DS3800HSHA slave - Channels 2 through 9
- DS3800HMHA to DS3800HIOB slave - Channels 2 through 9
- DS3800HMHA to DS3800HMHA slave - Channels 2 through 9
- c-bus (Virtual I/O) using DS3800HXMA (e.g. Series Six) - Channel 10
- Control Signal Highway (CSH) using DS3800HRMB - Channel 11
- Global I/O - Channel 11
- DS3800HMHA slave to DS3800HMHA master - Channels 12 through 15
- C-bus master using DS3800HXCA - Channels 16 through 19

The channel number associated with the I/O specifies the form of I/O being used. The DMC system requires the channel number when defining the I/O during system configuration. However, when referencing an I/O location, the channel number is not required.

The following text describes these I/O types.

Module/Local I/O (HADA)

Module I/O is I/O transfer between boards within the same rack as the DMC; local I/O is I/O transfer between the DMC and boards in different racks. The DS3800HADA board handles both module I/O and local I/O, although local I/O also requires a DS3800HPBC Parallel Line Buffer board to communicate outside of the same module. Multiple stations can be defined within each channel. Each rack has its own L-bus address associated with it.

HADA Configuration Information. The following information is provided to help the user understand the I/O addressing logic. For more detailed information, refer to the system elementary.

The number of bytes for both input and output are defined at system configuration. All bytes within a rack start at byte address 0, are stored contiguously within the DMC, and are addressed sequentially.

CHANNEL: 0 or 1
STATION: Software selectable; normally set to the DMC's station number.
RACK:Software selectable
BYTE: Number of contiguous bytes on L-bus
TYPE: 0
PORT ADDRESS: Starting L-bus address of byte 0

Figure 2-5 is an example of the CBL Application Program View of the I/O.

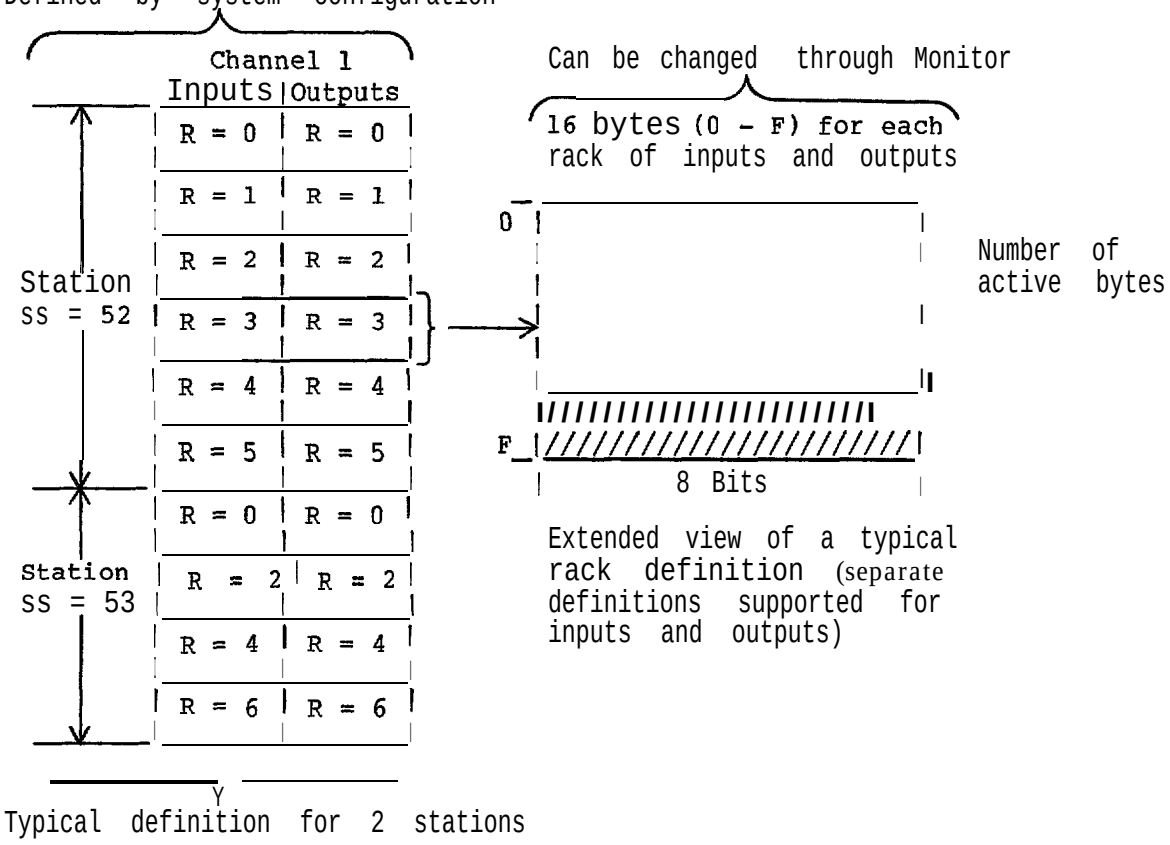


Figure 2-5
EXAMPLE OF CBL APPLICATION PROGRAM VIEW OF I/O

F-bus mapping into L-bus. Figure 2-6 shows how the DMC maps the F-bus into the L-bus address. The port address in the configuration information requires this L-bus address,

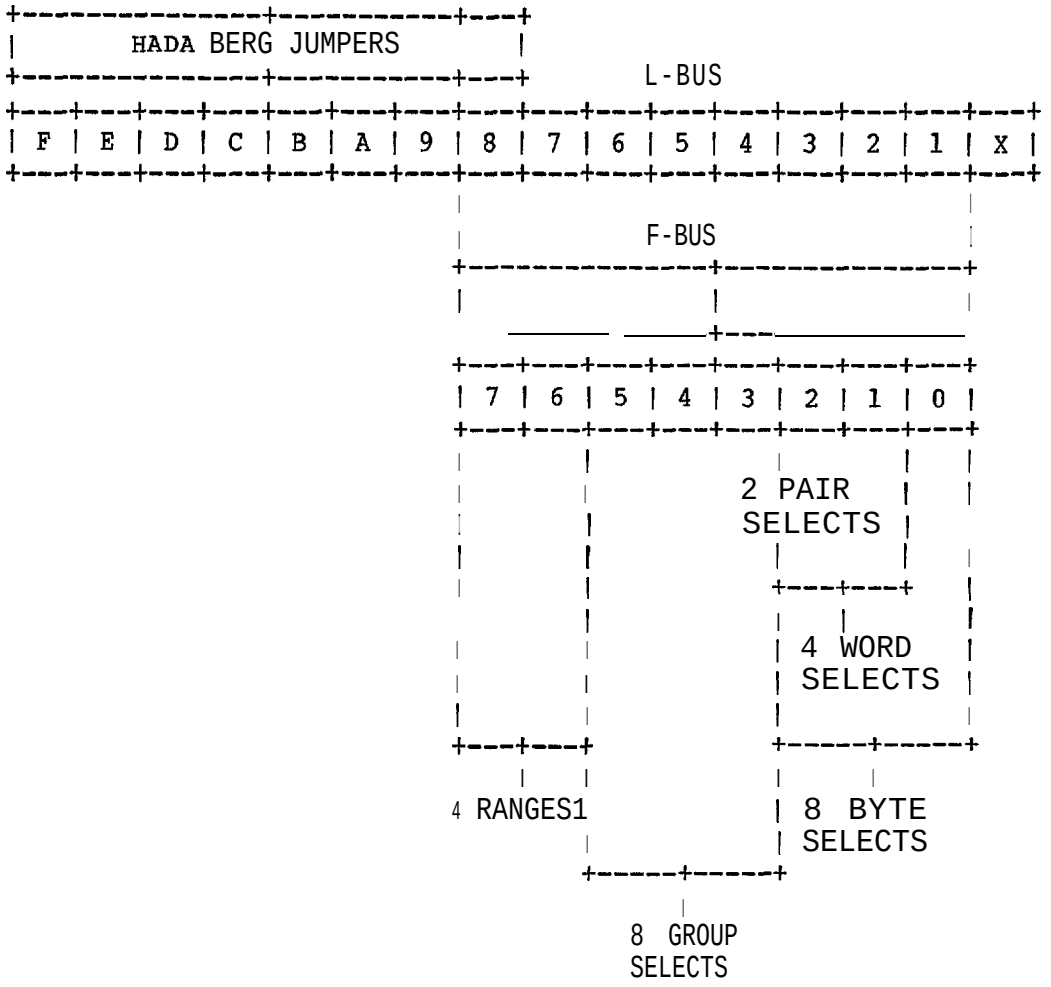


Figure 2-6
F-BUS MAPPING INTO THE L-BUS ADDRESS

F-bus to DMC Address Mapping. Table 2-1 is an example of F-bus to DMC address mapping when transmitting 8 input bytes and 2 output bytes for channel 1, station 40, rack 2, port address 4020H.

Table 2-1
EXAMPLE: F-BUS TO DMC ADDRESS MAPPING

HADA	F-BUS	L-BUS	GROUP	COUNT	REAL
40H*	10H	4020H	.I4020G	.I4020C	.I4020R
40H	11H	4022H	.I4021G		
40H	12H	4024H	.I4022G	.I4022C	
40H	13H	4026H	.I4023G		
40H	14H	4028H	.I4024G	.I4024C	.I4024R
40H	15H	402AH	.I4025G		
40H	16H	402CH	.I4026G	.I4026C	
40H	17H	402AH	.I4027G		

Note that the input bytes on station 40, rack 2 are referenced as .I4020G, .I4021G, .I4022G, and .I4023G by the DMC, although the L-bus hardware address is for even bytes only.

* The suffix "H" indicates a hexadecimal number.

DS3800HMHA Master Highway Board (DS3815PCLA) With DS3800HSHA Slave

The DMC contains a DS3800HMHA Master Highway board, which is a dual-ported memory board that handles all CPL (Control Party Line) I/O. The board functions as either a master or slave, depending on the type of firmware in the board's EPROM. With master station firmware, the HMHA becomes a DS3815PCLA module. The HMHA controls communication with 3 types of slave stations: a DS3800HSHA Slave Highway Interface board, a DS3800HIOB I/O Data Slave board, and another HMHA board containing slave firmware.

The HSHA is the most common type of slave for the HMHA. On-board Berg jumpers set the HSHA station ID. The HMHA functions in parallel with the CPU processor during the time scan. The HMHA handles the slave station input and output simultaneously, and can support 2 channels per board. The system can contain more than 1 HMHA board. Any channel can handle multiple slave stations of multiple type; however, space limitations and time constraints limit the number of stations per channel.

HSHA Configuration Requirements.

CHANNEL: 2,3 4,5 6,7 8,9
STATION: Set by Bergs on HSHA
RACK: Top 4 bits of F-bus address
BYTE: Number of contiguous bytes on F-bus
TYPE: 0
FORT ADDRESS: Not used

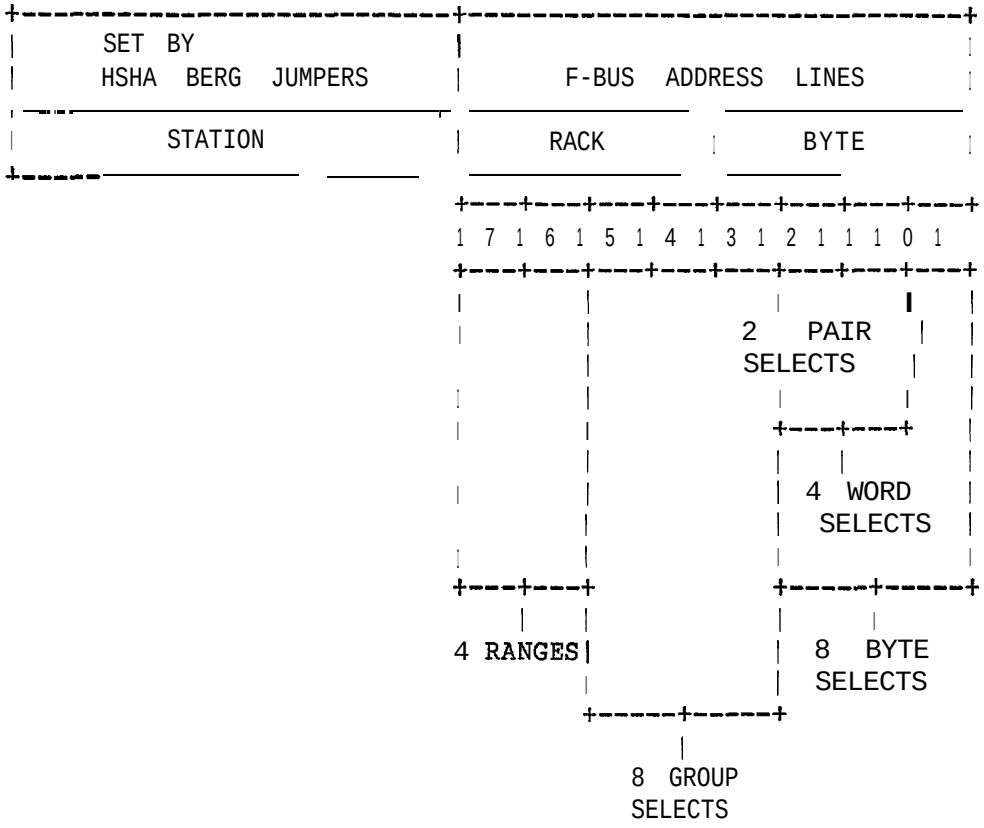


Figure 2-7
F-BUS RELATIONSHIP TO RACK/BYTE FOR DMC

HSHA F-bus Mapping. Table 2-2 is an example of F-bus to DMC address mapping when transmitting 0 input bytes and 8 output bytes for channel 2, station CC, rack 1.

Table 2-2
EXAMPLE: HSHA F-BUS MAPPING

HSHA	F-BUS	GROUP	COUNT	REAL
CCH	10H	.OCC10G	• 0cc10c	.OCC10R
CCH	11H	.OCC11G		
CCH	12H	.OCC12G	.OCC12C	
CCH	13H	.OCC13G		
CCH	14H	.OCC14G	.OCC14C	.OCC14R
CCH	15H	.OCC15G		
CCH	16H	.OCC16G	.OCC16C	
CCH	17H	.OCC17G		

HMHA Dual-ported RAM Strappings. Table 2-3 shows the required dual-ported RAM address strapping for each HMHA channel, assuming that address C000H is the starting address set at system configuration.

Table 2-3
HMHA ADDRESS STRAPPING WHEN USING HSHA SLAVE

ADDRESS	HMHA CHANNEL	DMC CHANNEL
C000H	A	2
C000H	B	3
C800H	A	4
C800H	B	5
D000H	A	6
D000H	B	7
D800H	A	8
D800H	B	9

HMHA Master with DS3800HIOB Slave

The HMHA can also communicate with the HIOB board, which is used to control a General Electric DS3828 SILTRON* Power Conversion module. The HIOB is limited to 8 inputs and 16 outputs, with the bits defined specifically for the SILTRON module.

Configuration Requirements.

CHANNEL: 2,3 4,5 6,7 8,9
STATION: Set by Bergs on HIOB
RACK: 0
BYTE: In =1, Out 2
TYPE: 1
PORT ADDRESS: Not used

HMHA Master With HMHA Slave (DS3815PRBA)

The slave HMHA (DS3815PRBA module) resides in another DMC and allows 2 or more DMCs to communicate via the CPL link. (With slave station firmware, the HMHA becomes a DS3815PRBA.) The I/O interface on the master HMHA is the same as for a HSHA. The slave HMHA is limited to only one station and one channel per board, but multiple HMHA slave boards can exist on a system. The slave station address is software selectable, but both the master and slave HMHA must use the same station ID. I/O transfered between the master and slave HMHA boards is referenced in the same manner as all other external I/O.

Configuration Requirements.

CHANNEL: 2,3 4,5 6,7 8,9
STATION: Same as slave - Software selectable
RACK: Top nibble of 256 byte slave address space
BYTE: Number of contiguous bytes in address space
TYPE: 0
PORT ADDRESS: Not used

* SILTRON is a trademark of General Electric Company.

C-bus Virtual I/O Using DS3800HXMA

The DMC uses the HXMA dual-ported memory board to communicate with another controller (such as the Series Six) via the C-bus. The HXMA can store 32 bytes of data. Series Six I/O allows a maximum of 128 input bytes and 128 output bytes. The DMC accesses this I/O as a single channel and a Single station within that channel.

Configuration Requirements.

CHANNEL: 10
STATION: Software selectable
RACK: Software selectable
BYTE: Number of contiguous bytes on L-bus
TYPE: 0
PORT ADDRESS: Starting L-bus address of byte 0

See "Local I/O" for F-bus to L-bus mapping.

Control Signal Highway (CSH) I/O Using DS3800HRMB

The DMC's CSH interface uses the HMPH daughter board, the DMPC, and the HRMB RAM memory board to communicate as a slave station. The DMPC creates an extended bus (E-bus). The memory board connects with this extended bus (E-BUS) and is mapped into Segment 8 of the DMC's memory space. The DMC divides Segment 8 into 256 stations, each with a capacity of 256 bytes within the station. The last station on the board is reserved for semaphore coordination within the segment. The segment offset's 8 high address bits determine the CSH station addresses. Only memory used for I/O needs to exist within Segment 8. (For example, station EO is mapped to Segment 8 at, addresses from E000H to E0FFH.) All CSH I/O stations reside on a single channel.

Configuration Requirements.

CHANNEL: 11
STATION: High address byte in segment 8
RACK: Top 4 bits of low address byte in segment 8
BYTE: Number of contiguous bytes in segment 8
TYPE: 0
PORT ADDRESS: Starting L-bus address of byte 0

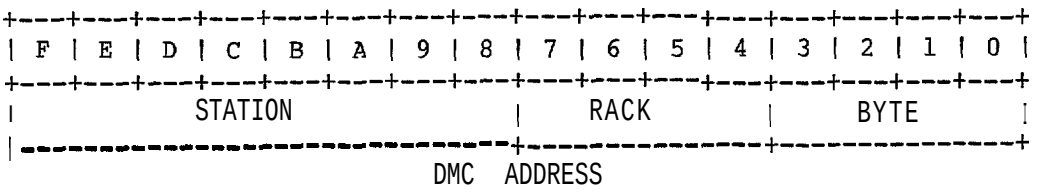


Figure 2-8
SEGMENT 8 TO DMC ADDRESS SPACE

Global I/O

Global I/O provides a second method (CSH is the first) for DMCs to communicate with each other. The DMC-to-global I/O interface is identical to the DMC-to-CSH I/O interface. The DMC uses the same hardware as CSH I/O (DMPC and HXMA boards) to generate an E-bus for global I/O transfers with another DMC.

Configuration Requirements.

CHANNEL: 11
STATION: High address byte in Segment 8
RACK: Top 4 bits of low address byte in Segment 8
BYTE: Number of contiguous bytes in Segment 8
TYPE: 0
PORT ADDRESS: Starting L-bus address of byte 0

See "Control Signal Highway (CSH) I/O" for segment 8 address mapping.

HMHA Slave With HMHA Master

The HMHA *slave* (DS3815PRBA) can transfer up to 256 bytes of I/O to an HMHA master (DS3815PCLA). The HMHA master resides in another DMC and data is transmitted via the CPL link. The slave HMHA contains only 1 channel on-board, with only 1 station on that channel. The master and the slave require the same station address.

Configuration Requirements.

CHANNEL: 12, 13, 14, 15
STATION: Same as master, software selectable
RACK: Top 4 bits of 256 byte slave address space
BYTE: Number of contiguous bytes in address space
TYPE: 0
PORT ADDRESS: Not used

HMHA Dual-ported RAM Strappings. Table 2-4 shows the required dual-ported RAM address strapping for each HMHA channel, assuming that address C000H is the starting address set at system configuration.

Table 2-4
HMHA ADDRESS STRAPPING WHEN USING HMHA SLAVE

ADDRESS	HMHA CHANNEL	DMC CHANNEL
C000H	A	12
C800H	A	13
D000H	A	14
D800H	A	15

C-bus Master Using DS3800HXCA

The DMC uses the HXCA C-bus master board both as C-bus interface and for R-bus data transfers. Each HXCA can handle up to 250 bytes of input and 250 bytes of output. The C-bus can be used to drive Series Six I/O. The board requires 2K of I/O address space on the L-bus. To communicate over long distances, the HXCA requires the DS3800HXTA Series Six Transmitter board and the DS3800NPSR Transmitter Power Supply board.

Configuration Requirements.

CHANNEL: 16, 17, 18, 19
STATION: Software selectable
RACK: Top 4 bits of F-bus address
BYTE: Number of contiguous bytes on F-bus
TYPE: 0
PORT ADDRESS: Starting L-bus address of HXCA

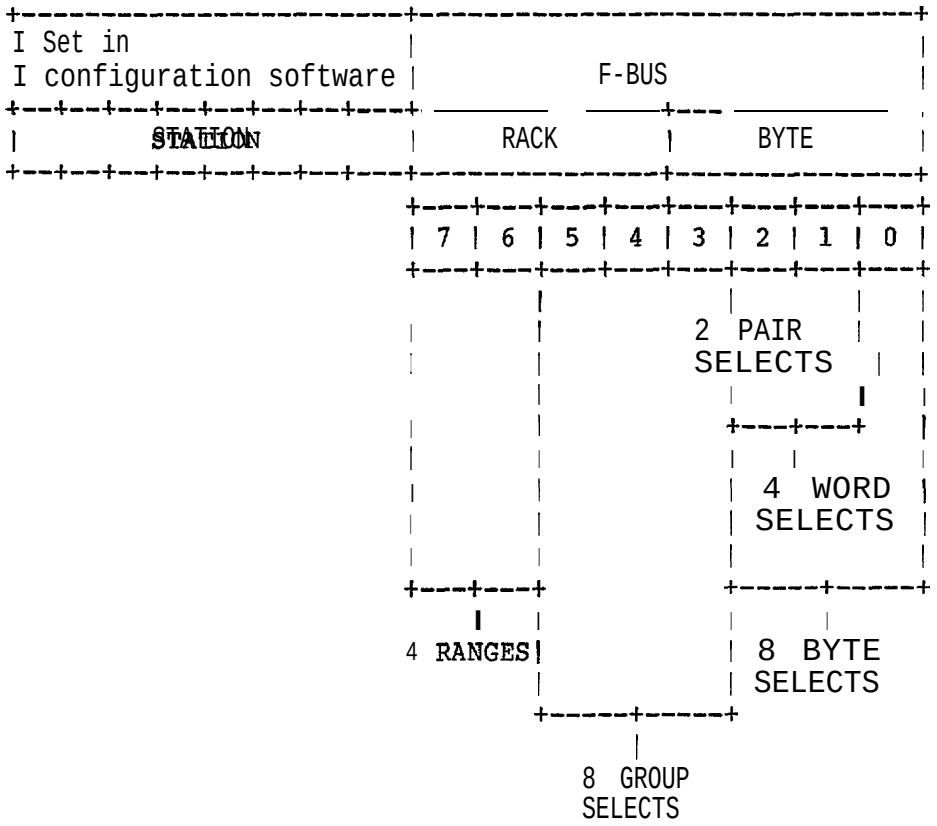


Figure 2-9
F-BUS RELATIONSHIP TO RACK/BYTE FOR DMC

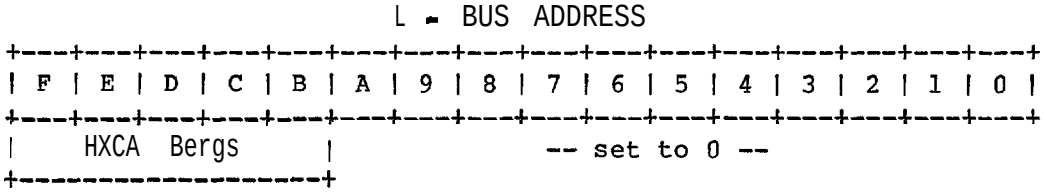


Figure 2-10
HXCA TO L-BUS MAPPING

INTERRUPT CONFIGURATIONS

Interrupts to the HMPH board start-up the DMC's interrupt driven segments. (Refer to the previous section, "Interrupt Segments".) The DMC can use any of the 3 interrupt configurations described below.

Figure 2-11 shows the first configuration, which provides 4 possible interrupts using the HMPH's 8259 interrupt controller chip. The configuration requires no extra hardware. Two of the 4 interrupts, however, may not be usable in most systems: if the system contains an 8087 coprocessor, Interrupt 36 is dedicated to that chip: to provide a design margin in the DMC, Interrupt 39 is a default interrupt and is not used.

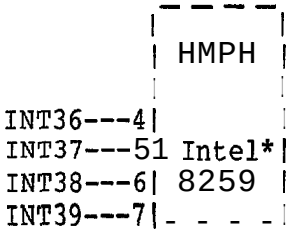


Figure 2-11
SYSTEM WITH 4 INTERRUPTS

Figure 2-12 shows the second possible interrupt configuration, which requires an added DS3800HXPC CPU Expander board. The HXPC contains an 8259 interrupt controller chip which can expand the DMC's interrupt levels by 8. Note that Interrupt 47 is a default interrupt and is therefore not used. When the system contains one HXPC board, it uses port address base 9000H.

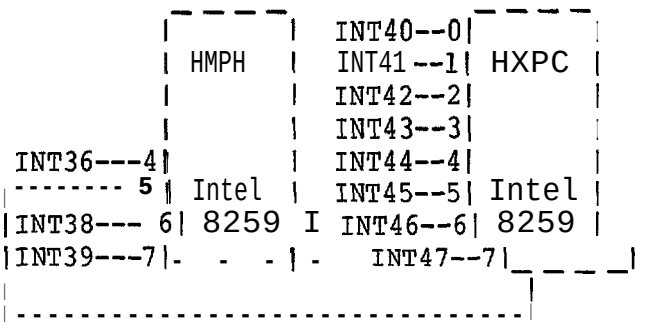


Figure 2-12
SYSTEM WITH 11 INTERRUPTS

* Intel is a registered trademark of Intel Corporation.

Figure 2-13 shows the third possible interrupt configuration, which requires 2 added HXPC boards. (The HXPC also contains 8253 timer chips, digital I/O, RAM, and PROM which can be used in the system.) These boards are strapped for port addresses 9000H and 9100H. As in the second configuration, each HXPC can expand the HMPH's interrupt levels by 8. Both boards also do not use default interrupts 47 and 55.

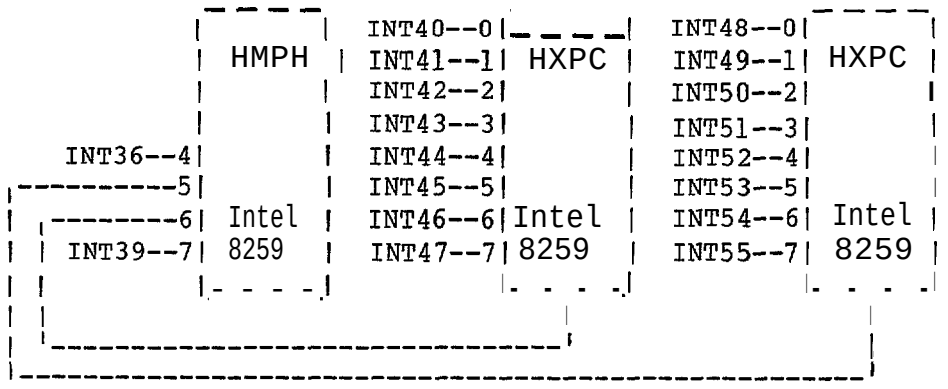


Figure 2-13
SYSTEM WITH 18 INTERRUPTS

NOTE: For all of the above 3 interrupt configurations, the 8087 coprocessor uses INT36.

SYSTEM CONFIGURATIONS

The DMC can be configured in many differant combinations with different types of boards, depending on the system requirements. (Refer to Appendix A for information about specific boards.) All systems contain external I/O. The following text describes some of the possible memory and I/O configurations.

Minimal DMC System

Figure 2-14 shows the minimal DMC system with the peripheral support available to all DMC systems. The peripherals connect to the HMPH via the board's RS-232 port. This port can be used with a Local Terminal for program inspection and modification, with an STR*-Link tape cartridge unit for system copy and load, or with a suitably configured VAX computer for system copy and load. (See Chapter 3, TRANSFER SYSTEM Commands for system backup). All system configurations can use these devices, but this is the only diagram that shows them.

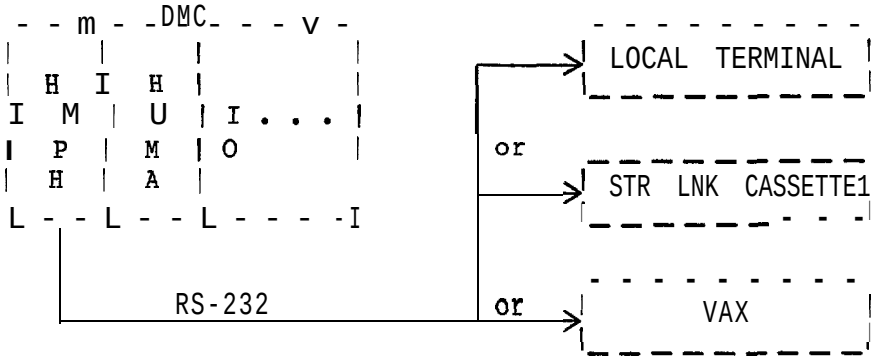


Figure 2-14
MINIMAL MEMORY CONFIGURATION

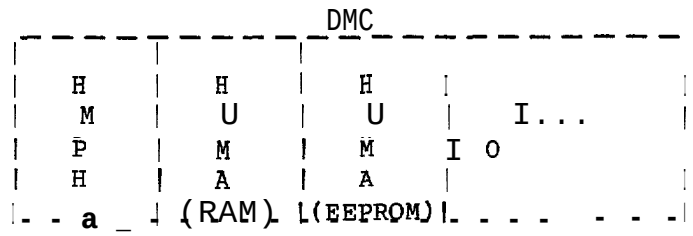
The HUMA memory board contains 32K of EEPROM that provides a run-only system. The DMC accepts changes only on the CONSTANT variable data, and does **not** accept changes to the application logic. All logic changes must be downloaded from the VAX or cassette.

The HUMA is strapped from Segment 0 offset address 4000H to Segment 0 offset address BFFFH.

* STR is a registered trademark of Electronic Processors, Inc.

Standard Memory Configuration

Figure 2-15 shows the standard memory configuration. This configuration contains all the boards in the minimal system plus an extra HUMA board with 8K-128K of RAM. The DMC runs on code stored in RAM on the second HUMA board, and uses the board's 32K of EEPROM for a backup or initial download image. The RAM is initialized from the EEPROM whenever power is applied to the system.



Typical Catalog Number:

DS3810RAM = HUMA with RAM
DS3810EEP = HUMA with EEPROM

Figure 2-15
STANDARD MEMORY CONFIGURATION

The user can modify the system application code at any time and save it on EEPROM using the monitor commands. RAM can be used to expand the DMC data tables and for CBL block private use. The DMC can be loaded from tape at 9600 baud and saved on EEPROM when the system is online.

A HUMA with only EEPROM is strapped from segment 2 offset address 0 to segment 2 offset address 7FFFH. On a HUMA that includes RAM, 1 of its 4 segments is strapped from segment 0 offset address 4000H to segment 0 offset address BFFFH; the strapping of other segments is selected at configuration time and is shown in the system elementary.

Expanded Memory Configuration

Figure 2-16 shows the DMC's expanded memory configuration. This arrangment adds from 1 to 3 more HUMA boards to the standard memory configuration. Each board contains 32K of EEPROM. These HUMAs are used for either an expanded symbol table or an extended CBL library. The user cannot modify the extended tables on the DMC, but can load them from tape or from the VAX.

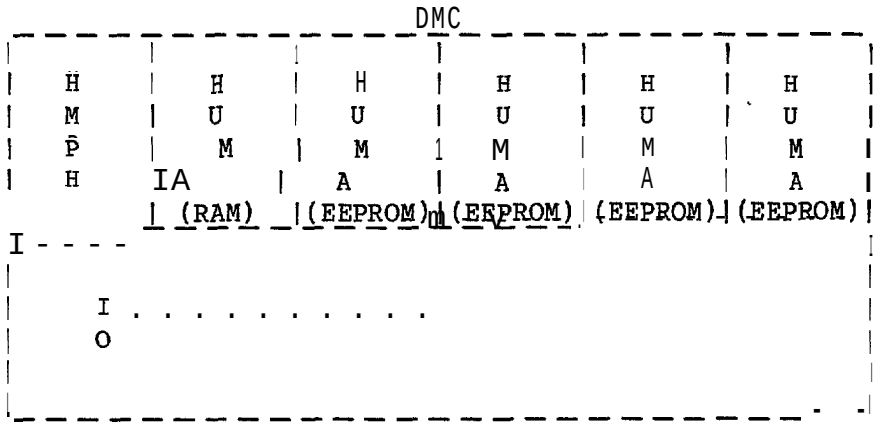


Figure 2-16
EXPANDED MEMORY CONFIGURATION

The expanded symbol table uses 1 HUMA strapped from segment D offset address 0 to segment F offset address 7FFFH. This table resides in EEPROM and allows up to 3200 symbolic definitions in the system.

The extended CBL library uses 1 or 2 HUMAs to provide up to 32K or 64K of code space. This code resides in EEPROM. The boards are strapped from segment E offset address 0 to segment E offset address FFFFH.

Interrupt Expansion

Figure 2-17 shows a DMC configuration for interrupt expansion. The HXPC boards provide additional interrupt capability, as described previously. Each board contains 4K of RAM, strapped to Segment 0 address 2000H and to Segment 0 address 3000H for using DMC data tables.

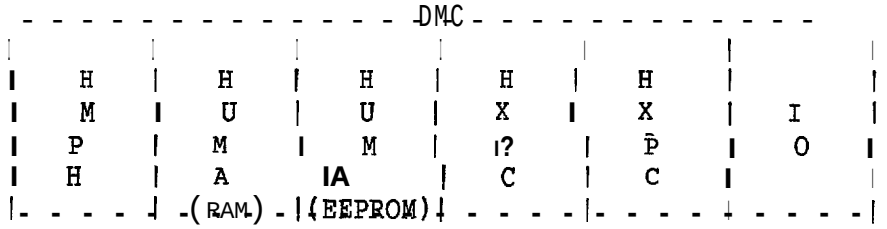


Figure 2-17
INTERRUPT EXPANSION

The first HXPC interrupt control connects to the HMPH's Interrupt 5; the second HXPC interrupt control connects to the HMPH's Interrupt 6. The port address is strapped for addresses 9000H and 9100H.

The the HXPC's PROM is not used and is therefore disabled.

Local I/O Configuration

Figure 2-18 shows a local I/O configuration using the DS3800HADA L-bus to F-bus decoder board. (Refer to Appendix B for bus definitions.) This I/O is limited to a distance of 50 feet. The system requires isolation modules if the DMC and I/O electronics module reside in different control cabinets. The DMC module requires a buffer decoder board, the HPBC. The I/O module requires an HPBC with a DS3800NTEA, a DS3800HBDA Buffer Decoder board, or no board at all, depending on the I/O configuration.

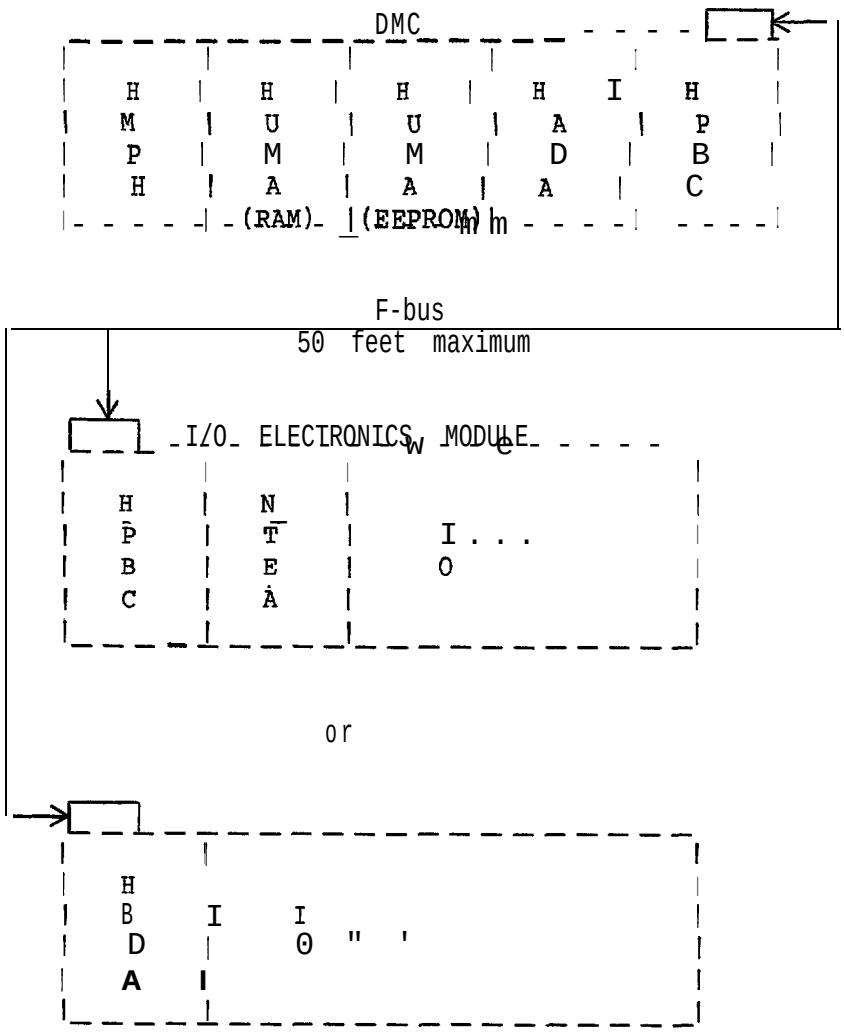


Figure 2-18
LOCAL I/O CONFIGURATION

Master DMC to CPL I/O Slave

Figure 2-19 shows a DMC system configuration using the DMC as a CPL master to an I/O slave. As a CPL master, the DMC requires an HMHA Master Highway Controller board, as discussed previously. The CPL remote I/O can handle I/O at a distance of up to 4000 feet. The system reguires isolation modules on any I/O lines outside of the DMC cabinet.

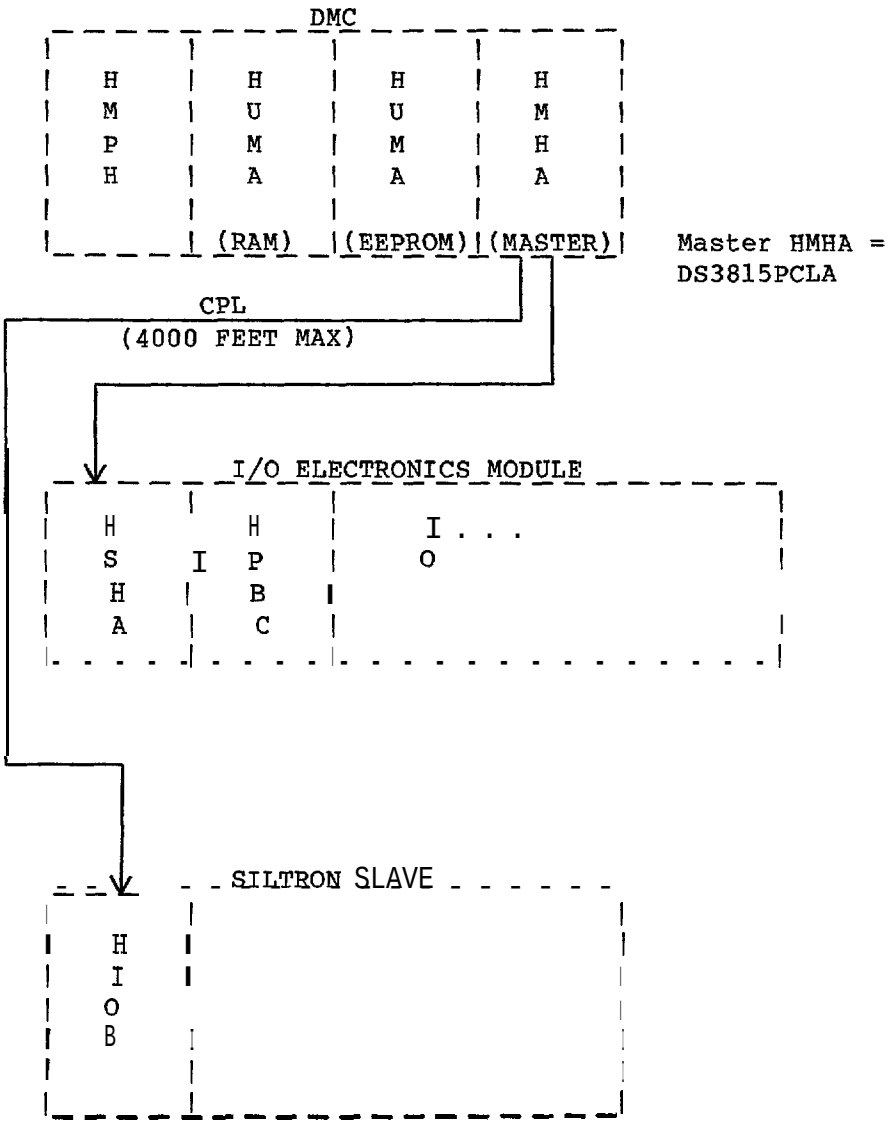


Figure 2-19
CPL MASTER TO I/O SLAVE

The DMC generally controls 2 basic types of I/O slaves. One type is a DS3828 SILTRON drive module (see Appendix B), which requires a HIOB I/O Data Slave interface for control. This I/O is limited to 2 bytes of output and 1 byte of input.

The second type of slave station requires an HSHA Slave Highway Interface board, which is used for general I/O at a remote site. The HSHA can be set up in either of 2 modes: the decode mode provides 64 bytes of I/O: the buffered mode uses either the HBDA or HPBC buffering boards.

Berg jumpers set the station ID for each I/O station. Multiple stations can reside on each HMHA channel, and the HSHA and the HIOB can reside on the same channel.

The HMHA contains dual-ported memory on the L-bus. The memory address must be unique, with strapping in the range of segment 0 address C000H to segment 0 address E7FFH. The selected address determines the remote I/O channel ID. (For more information, including addressing, refer to this chapter's "DS3800HMHA Master Highway Board" section.)

CPL Master DMC to Slave DMC

Figure 2-20 shows a DMC master connected to a DMC slave via the CPL. Both DMC modules contain an HMHA board for CPL communication. However, the HMHA master and HMHA slave differ in their application firmware. Each board contains 2K of dual-ported RAM on the L-BUS. The HMHA slave supports only 1 channel. (For more 'information, including addressing, refer to this chapter's "DS3800HMHA Master Highway Board" section.)

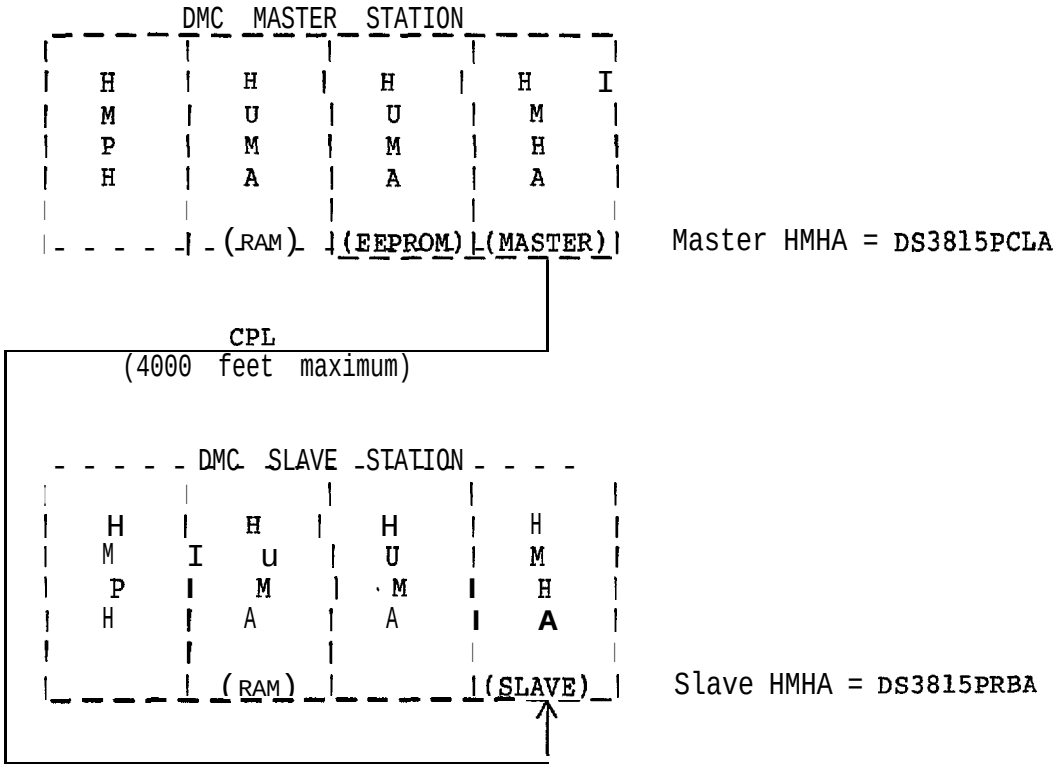


Figure 2-20
CPL MASTER TO SLAVE

CSH Slave Station Interface

Figure 2-21 shows the DMC connecting to a CSH slave electronics module via the slave's HPBC buffer board. The slave provides an interface for the DMC and host computer. The slave module connects to its host computer with a set of high speed data link interface boards (DS3800HLXA and DS3800HLXB) and a set of data link control boards (DS3800HLSC and DS3800HLSD).

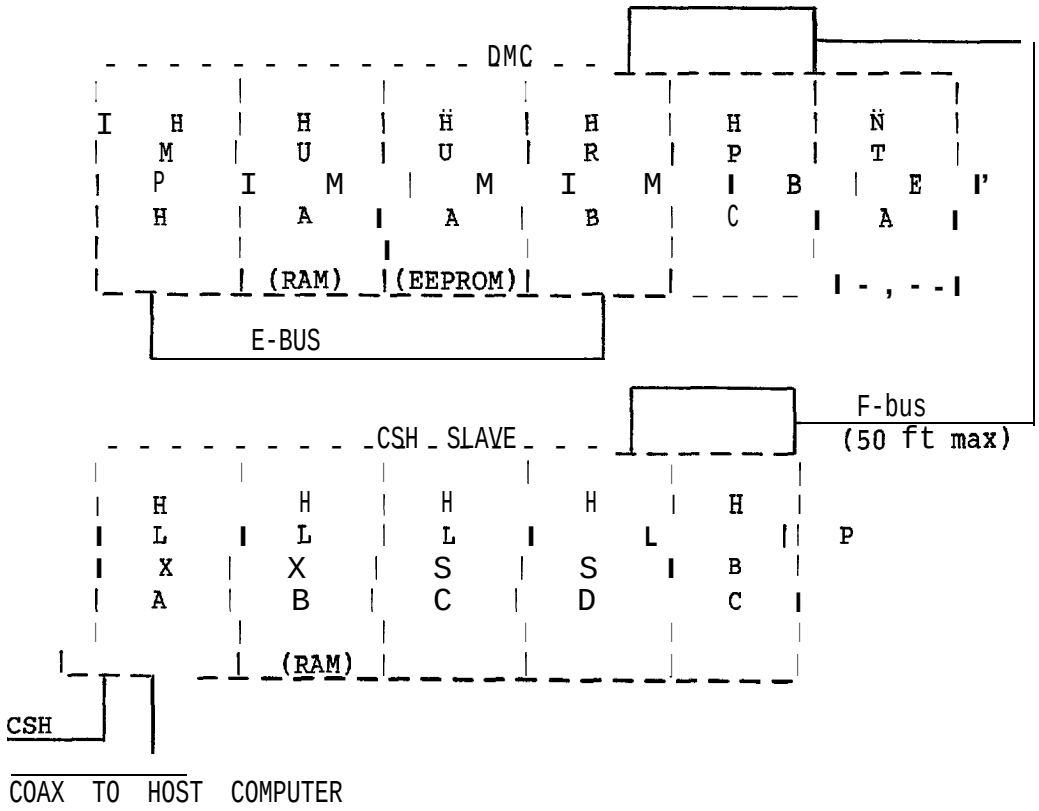


Figure 2-21
CSH SLAVE STATION INTERFACE

The DMC communicates with the slave module via a DS3800HRMB Dual-ported 8K Byte RAM board located on the E-BUS. The HMPH daughter board (DMPF) creates the E-bus for the DMC. The HRMB board is set up for segment 8 on the E-BUS. The top byte of the segment 8 offset address determines the I/O station address.

In some CSH slave interface configurations, the DS3800HXMA is used in place of the HRMB board.

C-bus Virtual I/O To Another Controller

Figure 2-22 shows an T/O connection via the C-bus between the DMC and a controller such as the Series Six. The Series Six controller can transfer UP to 32 bytes of input and/or output to the DMC's HXMA board via the DMC's DS3800HXRA Receiver board. (The diagram does not show the transition module, DS3820STMA.) The HXMA uses port addresses 8000H to 83FFH.

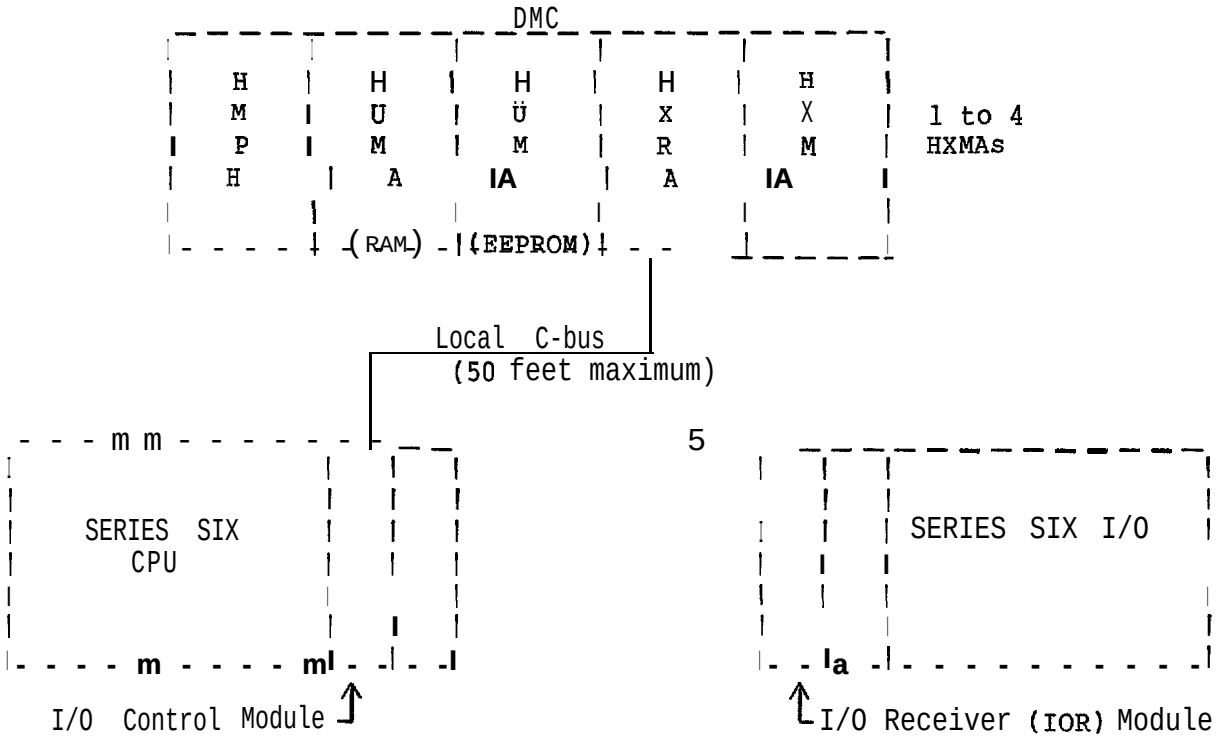


Figure 2-22
C-BUS VIRTUAL I/O

C-bus Master

Figure 2-23 shows a DMC to C-bus configuration. The DMC can function as a C-bus master by using the DS3800HXCA board. The HXCA drives all C-bus supported I/O and can communicate with R-bus I/O. This configuration often contains the HXRA, HXTA, and NPSR boards for long distance (500 feet) communication.

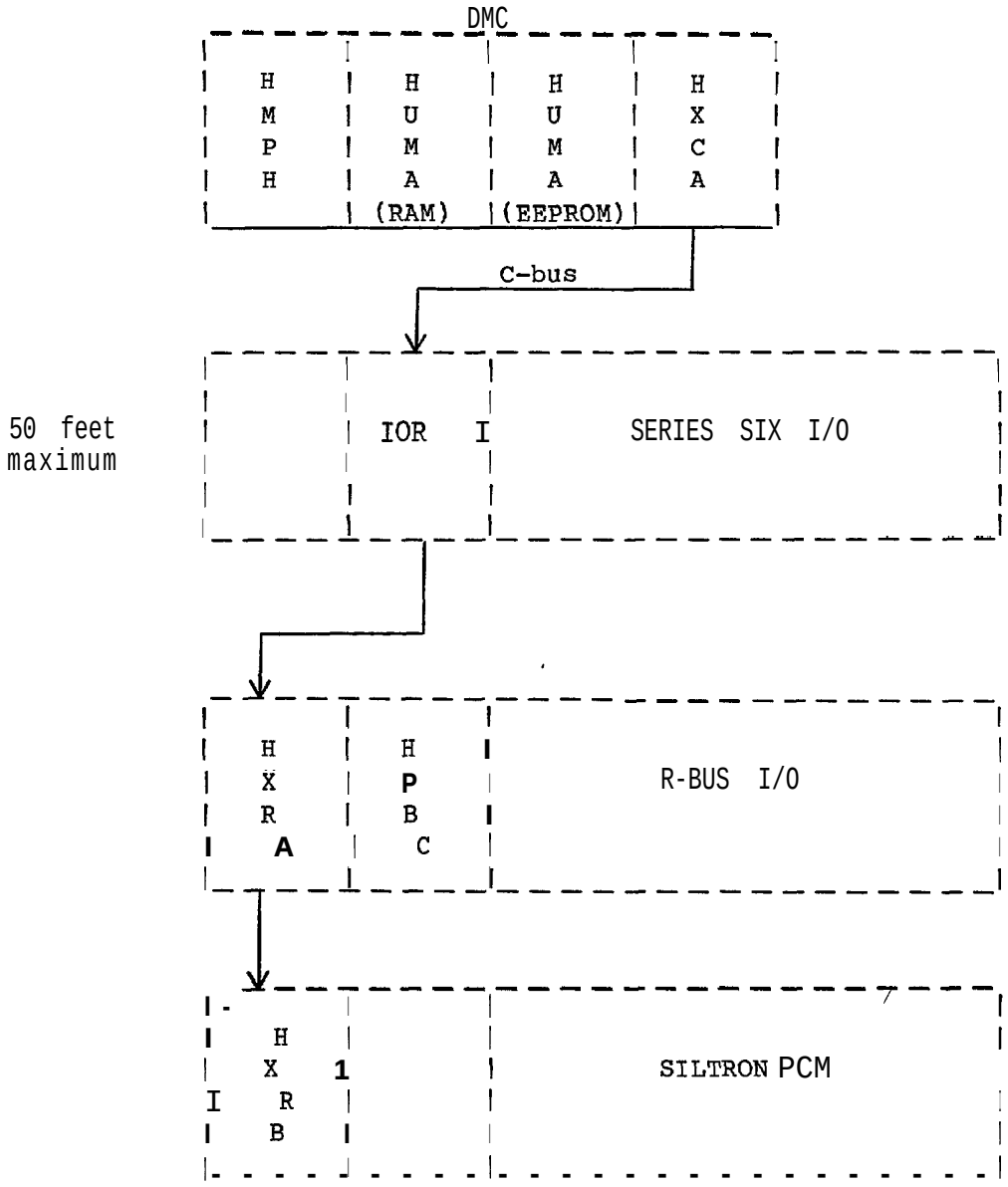


Figure 2-23
EXAMPLE DMC to C-BUS I/O

NOTES

-- CHAPTER 3 --
DMC MONITOR

INTRODUCTION

All DMC systems include a monitor, which is accessed via an RS-232 compatible terminal attached the the DMC's HMPH board. The monitor contains over 100 commands and subcommands that enable the user to inspect and modify the DMC's signals and CBL program.

This section discusses the monitor, including the terminal interface requirements, system start-up, and monitor access. It also describes each command and subcommand and their function within the DMC system.

GENERAL TERMINAL CHARACTERISTICS

The DMC monitor interface must be RS-232 compatible, and can be either a l&character hand-held terminal, a CRT, or a hard copy terminal. The DMC functions in either of 2 display modes:

- Scroll mode (CRT mode) – The standard mode, used for terminals that display all data at the baud rate. A 2-character prompt, "X>", serves as a command request in which "X" is the single letter command that indicates the currently active command level.

- Handheld mode – The alternate mode, used for terminals with a single 16-character display line. This mode leaves the last completion message on the display and does not prompt for the next command. The DMC accepts any subcommand entered while the last command line is displayed. Hitting [RET] (the RETURN key) will display the prompt, although the handheld mode does not require it. When a command prints out more than 1 line, it displays each line for 1 second before displaying the next line. Typing a key during the pause inactivates the delay between line printouts.

Several keyboard characters provide special functions for the **DMC** system:

- Typing the character **"S"** during output halts the output. When another character is entered, the display continues from where it left off.
- Typing the **[RET]** character during output aborts any list and returns the monitor to the subcommand level.
- Typing either a **rubout** key or the delete/backspace key erases input. The **rubout** key **echos** the character to be deleted; the delete key erases the character then backspaces if the monitor is in scroll mode.
- Control-X clears the input buffer, i.e. any characters accumulated.
- Control-R displays the current contents of the input buffer.
- Control-Z aborts any input request.

NOTE: Control-X, Control-R, and Control-Z function only if the terminal being used can generate control characters.

System Set-Up

The following parameters are set when the system is configured:

- Parity - Even parity is required for the local terminal.
- Default baud rate - The system restarts at the default baud rate. Valid baud rates are 110, 300, 600, 1200, 2400, 4800, and 9600. The baud rate can be changed once the system is running, but the system returns to the default baud rate when rebooted.

  NOTE: The system starts an automatic baud (auto-baud) search once the system self-test completes and also after the monitor has been exited. The DMC determines the new baud rate by looking for a **[RET]** character. During either of these states, the user should enter multiple **[RET]** inputs until the monitor is activated.

- **System** terminal type - The terminal type is either “handheld” or “CRT”, depending on the **system.** When booted, the **system** comes up with the selected terminal type.
- Input time-out - Defines the time period (in seconds) that the **system** will wait for an input before it aborts the request. A period of from 1 second to 255 seconds long is set at system configuration. The system jumps to the next higher command level when a time-out occurs. **If** the input times out after reaching the main command input level, the monitor aborts.

SYSTEM SELF-TESTS

At system power-up, the DMC performs self-tests that check for checksum errors, memory failures, and missing hardware. If a terminal is connected (set at the default baud rate), the DMC displays the following messages that indicate completed self-tests when it restarts:

```
LOCAL CONTROLLER
PROM CSUM -- OK
BASE RAM -- OK
8087 CHECK -- OK
AUX RAM -- OK
SYSTEM RAM -- OK
APP CSUM -- OK
SELF TEST DONE
```

These self-tests are defined below.

PROM CSUM – Checksum computed for the product code that resides on the HMPH board. Failure means that the product code in PROM is damaged. If this test fails, replace the product code PROMS and/or the HMPH board.

BASE RAM – Performs a destructive memory test on RAM that resides on the HMPH board. A failure displays the memory address of the failure. If this test fails, replace the HMPH board. When this test completes, it sets the memory to all 0s (zeroes).

8087 CHECK – Executes only if an 8087 coprocessor is configured in the system. Failure indicates that the 8087 is not responding. If this test fails, verify that the DMPC daughter board is attached to the HMPH and that the coprocessor Berg jumper on the DMPC is set in the correct position.

AUX RAM – Performs a destructive memory test on RAM that resides outside the HMPH in address range 2000H to 3FFFH. This test runs only if the system contains memory within this range. When this test completes, it sets the memory to all 0's. The message is not displayed if the RAM size is 2000H or less. A failure prints the memory address of the failure. In case of failure, verify that memory is strapped to the region in which the error was detected and that no other board overlaps the memory space on that board. If the strapping is correct, replace the memory board.

SYSTEM RAM - Runs only if memory used for system variables is not located in EEPROM, The system conducts a non-destructive memory test on the area when it is determined to be BRAM. The error recovery is the same as that stated under AUX RAM.

APP CSUM - Application checksum performed on all systems. The base system, the symbol table, and the CBL library have checksums generated on them. The system recomputes the checksum and compares this checksum on **powerup;** if the checksum was never created (such as after a VAX download or RESTORE) the checksum is flagged as --NEW. Checksum failure indicates any of 3 problems: **1)** that the memory containing this information might be destroyed, **(2)** that a faulty CBL block is modifying data that it should not be addressing, and **(3)** there are **"soft"** memory failures missed by earlier self-tests. When the application checksum test fails, either reload the system or place the memory boards through a more extensive RAM test.

[INITIALIZED] - Displayed only when the system determines that the system memory space is destroyed. The DMC will look at the secondary memory space to see if a valid system can be found. If it finds a backup system, the DMC downloads the secondary system into primary memory. In all other cases, the DMC downloads a default system from product code PROM. This system has no CBL library nor logic defined.

[RESTORED] - Displayed only when the contents of the system memory have been downloaded from the secondary EEPROM. The DMC performs this restore when the it detects that the primary memory is not valid and that a system image exists on the secondary EEPROM.

PIC xxxx BAD - Displayed only when one of the programmable interrupt controllers **(PIC)** does not respond. The "xxxx" represents one of the following PIC port address:

- OBFFE - HMPH master PIC
- 09022 - HXPC slave PIC 1
- 09122 - HXPC slave PIC 2

The DMC tests the slave **PICs** only if the system is configured with **HXPCs.** If either slave PIC fails, verify that the 2 HXPC boards are strapped to addresses **9000H** and **9100H.** If the strappings are correct, replace the HXPC boards . If the master PIC fails self-test, replace the HMPH board.

I/O **INIT** FAILURE - The last self-test performed. The DMC checks all enabled channels on any HMHA master/slave boards, but does not check disabled channels. The DMC prints a failed channel's ID and the system comes up off-line. If the test fails, first check the **HMHA's** IMOK light: if it is turned off, restart the system and check the IMOK light again; if it is already turned on, check that the HMHA board's memory is strapped for the correct channel and that the memory strappings do not overlap. If the board still fails self-test, replace it.

On all the above tests, except for I/O and initialization, the test name prints out when the test begins. The DMC displays the message **"-- OK"** if the test completes successfully, and the message **"-- BAD"** if the test fails. When a test fails, the self-test pauses for about a second then displays the following prompts:

CONTINUE?

CLEAR MEMORY?

The system can still be brought up and the remaining self-tests executed by answering **"Y"** to the CONTINUE prompt. If **"Y"** is entered, the self-test continues, but the DMC comes up off-line. Any other answer displays the CLEAR MEMORY prompt. A **"Y"** entry for the CLEAR MEMORY prompt sets the initialize flag and restarts the system from scratch. The INITIALIZE statement then appears in the test results. Any other entry for the CLEAR MEMORY response spins the system in an idle loop. At this point, the user must manually restart the system.

CONTINUE and CLEAR MEMORY can be forced during self-test by entering a [RET] on the terminal. As soon as the next test completes, the DMC displays the CONTINUE prompt for the user. This method should be used when the user wants to clear memory on power up, even though no self-test failures occur, or when the user wants the system to come up off-line.

Failure Diagnosing Without A Local Terminal

The HMPH board's yellow IMOK light **(CR2)** and red **"Test"** light **(CR5)** can be used to diagnose system failures when no terminal is attached. The IMOK light turns on as soon as the product code gets control of the CPU on restart. The IMOK light should turn off, then turn back on and remain on as soon as the **HMPH's** self-tests complete. The Test light turns on for **1/4** second at the same time the **IMOK** light turns on at restart, and for **1/4** second at the end of each self-test. Whenever a self-test failure occurs, the Test light blinks rapidly until the system is continued or restarted. Note that the lights on the HMPH blink regardless of whether or not a terminal is connected to the system. (Use the terminal messages whenever possible, since they provide more information on how to recover and also allow user feedback.)

After the DMC completes self-test and starts normal operations, it continues performing checksum tests when not performing other operations. The DMC suspends checksums during a system modification, then recomputes them once the modification completes. For checksum failures, there is an internal counter that can be displayed as part of the BREAKPOINT command.

The HMPH board's IMOK light indicates that the system is finishing all active segments at least every second. It turns off if this condition is not met. When this occurs, the monitor can be activated and the scan times modified, although the monitor responds slower than normal. The IMOK light turns on as soon as a segment scan rate is modified.

Bus Time-Outs

The DMC continually checks for bus time-outs. This time-out occurs when the L-bus does not respond within 80 microseconds after an access. When the error occurs during any command, the system transmits the following message to the local terminal port:

=== BUS HOG ===
CS:IP xxxx:yyyy

The **"xxxx"** represents the current code segment register at the time of the error, and **"yyyy"** represents the current offset within the segment.

ACTIVATING THE MONITOR

As soon as the system completes self-testing, the monitor can be activated by typing several **[RET]s** on the terminal. The monitor determines the terminal's baud rate, and responds with the following message:

```
LOCAL  MONITOR
REV 1.2 10/25/83
*>
```

The monitor is now available. By entering commands through the monitor, the user can display and modify external **I/O** variables, display and modify CBL application programs, and upload/download DMC systems. The **"*>"** prompt indicates that the monitor will accept a command input from the user.

COMMAND STRUCTURE

The monitor's command structure is a 2-level hierarchical tree: the main command level consists of "commands"; the next level, the subcommand level, consists of **"subcommands"** under that main level command. The monitor provides 18 commands and over 100 subcommands.

All commands are single letter entries to the monitor, accepted only at the **"*>"** prompt. The system responds to a command as soon as it receives the character, and does not need a **[RET]** after the character. The recognized commands and their keyboard entries are listed below.

COMMAND	ENTRY	COMMAND	ENTRY
ASSIGN	A	MODIFY	**M**
BREAKPOINT	**B***	ONLINE STAT	**O**
CBL BLOCKS	C	PASSWORD	**P**
DISPLAY/MOD	D	QUIT MONITOR	**Q**
EDITOR	E	SEGMENTS	**S**
FORCE	F	TERMINAL	**T**
HELP	H	USER MONITOR	**U**
I/O	I	WHERE USED	**W**
LOOK	L	XFER SYSTEM	**X**

*NOTE: The BREAKPOINT command is designed only for factory use by General Electric. For information about this command, refer to Appendix D.

Entering a HELP **(H)** command into the system displays the list of commands for the user's reference.

When the system receives a command character, it enters the **subcommand** level. This level displays a prompt **"X>"**, in which **"X"** is the letter entered to get to the current subcommand level. To exit the subcommand level and re-enter the main command level (so that another command can be entered), the user must type a QUIT **(Q)** subcommand. Every command has a HELP **(H)** subcommand available that lists the subcommands available at with that command.

Memory Access Levels

To protect the system memory, the monitor functions at different access levels to limit the commands available to the user. There are 3 levels available:

1) Level 0 access - The first access level accepts only commands that read memory locations. The monitor enters Level 0 at start-up, and returns to this level when terminated.

2) Level 1 access - The second access level allows memory modification from any monitor command other than BREAKPOINT. This is the most common access level and provides all the memory access that is usually necessary. Level 1 access requires use of the PASSWORD command.

3) Level 2 access - The third access level allows memory modification from any monitor command, including the BREAKPOINT command. This is the highest access privilege granted, and requires use of the PASSWORD command.

****
* CAUTION *
****

THE USER CAN DESTROY THE SYSTEM WITH THE BREAKPOINT COMMAND BY MODIFYING ABSOLUTE SYSTEM MEMORY. DO NOT USE LEVEL 2 ACCESS UNLESS IT IS ABSOLUTELY REQUIRED. WHEN USING LEVEL 2 ACCESS, BE SURE TO EXERCISE EXTREME CARE.

The commands allowed at Level 0 are a subset of the commands allowed at Level 1; the commands allowed at Level 1 are a subset of the commands allowed at Level 2. If the user types in a command without having the necessary access, the DMC displays the error message '*INVALID PASSWORD**, and aborts the subcommand.

If the primary memory resides in EEPROM, the monitor will not modify memory in the primary system. Instead, the DMC prints an error message and aborts the command. To modify the memory in this type of system, the user must replace EEPROM with RAM, transfer the EEPROM's contents into RAM using the RESTORE command, make the change, then save the RAM's contents on the EEPROM.

SIGNAL REFERENCES

Signal Address

The basic method for referencing signals or variables is by using the "Signal Address". The signal address always starts with a "." followed by a letter indicating the kind of signal, a code indicating the specific signal, and then a final code indicating the type of signal. The signal address consists of the following 2 major categories:

- Inputs and Outputs
  - Kind identifiers - Uses "I" for input and "O" for output.
  - Specific signal identification - Four hexadecimal digits indicate the station, rack, and group or byte of the interface hardware.
  - Type identification - Can use a "G" to identify the entire group or a "0" through "7" to identify a specific point or logical signal.
- Internal or Register Signals:
  - Kind identifiers - Uses "L", "V", and "C" for logical, variable, and constant, respectively
  - Specific signal identification - Four hexadecimal digits, indicate the offset within an internal table where the signals are stored.
  - Type identifications - Include "G", "C", "B", "R", and "F" for group, count, BCD, real, and fixed point, respectively.

    (For further detail, refer to Chapter 2, "SIGNAL IDENTIFICATION**.)

In a DMC system, all signals can be referenced by using the signal address. Additional signals can be added using the signal address reference to the limit of the I/O and internal tables, as established at configuration.

Signal Name

Some DMC systems include an optional mechanism that allows referencing signals by a "signal name" or symbol. For example, the signal address ".V0244R" can be assigned an equivalent signal name, "MOTSPD". A signal name consists of 1 to 7 characters, and can start with any letter or number besides "." (period).

Signal addresses are normally assigned signal names at system configuration. However, the monitor's ASSIGN SYMBOLS (A) command (described below) enables the user to modify existing assignments and to assign as many new symbols as the symbol table's allocated memory space allows. The size of the symbol table is an optional feature established at system configuration.

MONITOR COMMANDS AND SUBCOMMANDS

The following text lists and describes all commands and subcommands available with the DMC monitor. Each command is designated by a main heading, and is described below it; each subcommand is designated by a subheading following the associated command, and is described below it. Note that in the examples of displays, the user response is underlined and **[RET]** indicates the Return key. In general, [RET] is required after data entry, but not after command entries.

A – ASSIGN SYMBOLS Command

The ASSIGN SYMBOLS **(A)** command adds, deletes, or displays symbolic names from the system symbol table. A symbol is a mnemonic name that represents a physical table location and a data type. The symbol table is initially defined at the system configuration, and downloaded into the system. Symbolic names can be any name that does not start with a **"."**, and can be equated to physical names anywhere in the system. All signal address or physical names are in the form **".lxxxxt"**, in which **"l"** is the data table identifier **(L,** I, 0, **V, C); "xxxx"** is the hexadecimal offset of the data item within the table, and **"t"** is the data type **(G,** B, C, R, F, described in Chapter **2).**

The following sequence initiates the ASSIGN command from the main command level:

```
*> A
A>
```

H – HELP INFORMATION. The HELP **INFORMATION (H)** subcommand displays the following list of subcommands available within the ASSIGN command:

```
A> H
A -- ASSIGN
D -- DISPLAY
H -- HELP
L -- LIST
N -- NEXT
P -- PREVIOUS
Q -- QUIT
R -- RELEASE
A>
```

A - ASSIGN SYMBOL. The ASSIGN SYMBOL **(A)** subcommand assigns a new symbolic name to a physical name.

The following example shows a display for the A subcommand:

```
A> A
ADDRESS? .C0010C [RET]
SYMBOL? COUNTER [RET]
COUNTER = .C0010C
A>
```

The A subcommand accepts any valid physical name for the address and any undefined symbol for the symbol. Multiple symbols can be assigned to a single memory location, but only 1 memory location can be assigned to any symbol. A symbol must be released (deleted) before it can be modified. The DMC prints out a successful assignment, but produces an error message if the symbolic name is defined or the physical address does not exist. The DMC requests the binary scale if the address contains a data type of **"F"** (fixed point). There are no defaults allowed in this subcommand.

D - DISPLAY SYMBOL OR ADDRESS. The DISPLAY SYMBOL OR ADDRESS **(D)** subcommand prints out the symbolic and physical address of the name requested. The following example shows a display for the D subcommand:

```
A> D
DISPLAY? COUNTER [RET]
COURTER = .C0010C
A> D
DISPLAY? .C0010C [RET]
COUNTER = .C0010C
A>
```

The DISPLAY subcommand prints out the current assignment for a symbol or memory location, **or** an UNDEFINED SYMBOL error if the location is not found. A **[RET]** entry after the display prompt prints out the last symbol displayed.

L - LIST ALL SYMBOL ASSIGNMENTS. The LIST ALL SYMBOL ASSIGNMENTS **(L)** subcommand alphabetically displays all the current symbols in the system. The list can be stopped or aborted by typing a [RET] character during output. The following example shows a display for the L subcommand:

```
A> L
4002 = .C0070B
SCALE = .V0020F
FIX POINT B11
SLOPE = .IlOOOR
TRUE = .L0001
A>
```

N - DISPLAY NEXT SYMBOL. The DISPLAY NEXT SYMBOL (N) subcommand displays the next symbol (next in alphabetical sequence) after the last displayed symbol. (The "last" symbol means the last symbol referenced in any ASSIGN subcommand.) The table wraps around when it reaches the end. The following example shows a display for the N subcommand:

```
A> N
SLOPE = .I1000R
A> N
TRUE = .L0001
A>
```

P - DISPLAY PREVIOUS SYMBOL. The DISPLAY PREVIOUS SYMBOL (P) subcommand displays the previous symbol (previous in alphabetical sequence) after the last displayed symbol. (The "last symbol" means the last symbol referenced in any ASSIGN subcommand.) The table wraps around when it reaches the end. The following example shows a display for the P subcommand:

```
A> P
SPEED = .C012CR
A> P
MOTOR = .V0244C
A>
```

R - RELEASE SYMBOL. The RELEASE SYMBOL (R) subcommand deletes a symbol from the symbol table. The symbol must exist in the symbol table or else the DMC prints an error message. A symbol must be released before it can be redefined. The DMC deletes the last symbol displayed if the user does not enter a symbol for the SYMBOL? prompt.

The following example shows a display for the R subcommand:

```
A> R
SYMBOL? COUNT [RET]
COMMAND  COMPLETE
A>
```

NOTE: The deletion occurs immediately-- there is no confirmation sequence.

Q - QUIT. The QUIT (Q) subcommand exits the ASSIGN command and returns to the main command level. The following example shows a display for the Q subcommand:

```
A> Q
*>
```

B – BREAKPOINT Command

The BREAKPOINT (B) command is a combination of debug and diagnostic aids intended primarily for factory use during system configuration. This command checks for checksum errors, displays and modifies absolute memory, tests memory, and sets breakpoints in the interpreter. Refer to Appendix D for information about this command.

C – CBL BLOCK LIBRARY Command

The CBL BLOCK LIBRARY **(C)** command displays the names of the CBL blocks and the number of parameters required for each block. This information is read only, and can only be modified by reconfiguring the system.

The following sequence initiates the CBL BLOCK LIBRARY command from the main command level:

```
*> C
C>
```

H – HELP INFORMATION. The HELP INFORMATION **(H)** subcommand displays the following list of subcommands available for use via the CBL CONTROL BLOCK LIBRARY command:

```
C> H
D -- DISPLAY
H -- HELP
Q -- QUIT
C>
```

D – DISPLAY CBL BLOCKS. The DISPLAY CBL BLOCKS **(D)** subcommand prints out the name and the number of parameters required for either a specific CBL block or for all CBL blocks. The following example shows a display for the D subcommand:

```
C> D
DISPLAY? KDLY00 [RET]
KDLY00, 2 PRMS
C> D
DISPLAY? [RET]          (Note that entering only a [RET] to the DISPLAY? prompt
ZEXIT00, 0 PRMS          prints the entire list.1
21000, 1 PRMS
EINW00, 3 PRMS
EOTW00, 3 PRMS
KDLY00, 2 PRMS
KLCH00, 1 PRMS
KPLS00, 1 PRMS
ZJMP00, 1 PRMS
C>
```

Q – QUIT. The QUIT **(Q)** subcommand exits the CBL BLOCK LIBRARY command and returns to the main command level. The following example shows a display for the Q subcommand:

```
C> Q
*>
```

D - DISPLAY AND MODIFY VARIABLE Command

The DISPLAY AND MODIFY VARIABLE (D) command examines and changes variables within the system. The system accepts both symbolic names and default variable names. Note that when the monitor receives the D command input, it immediately enters into the D (DISPLAY VARIABLE) subcommand (described below). This is done automatically for convenience to the user. A [RET] terminates the D subcommand and returns to the command level prompt "D". At this time the user can enter another subcommand.

The D subcommand prompts for a variable name. The display format depends upon the variable's data type.

The following example shows a display for entry into the D command:

```
*> D
DISPLAY/MOD
DISPLAY?  COUNT
COUNT  =  1200
COUNT  =  1240
COUNT  =  1280
COUNT = 1320 [RET]
D>
```

H - HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the DISPLAY AND MODIFY VARIABLE command:

```
D> H
A --  ADJUST  VALUE
D -- DISPLAY
E --  ENTER  MULTI
F --  FAST  DISPLAY
H --  HELP
M --  MODIFY
Q --  QUIT  COMMAND
R --  RUN  MULTI
S --  STATION  I/O
+ --  ADD  ADJUST
- -- SUBTRACT ADJUST
D>
```

A - ADJUST VALUE FOR REAL. The ADJUST VALUE FOR REAL **(A)** subcommand sets the value for real numbers for the + and - subcommands. The + and - subcommands modify a count variable specified by 1 unit. However, the increment must be set for real numbers.

The A subcommand allows the user to set the desired value for real numbers. The number must be entered in real format (a number with an embedded decimal point and an optional "-" sign for negative numbers). The system sets the value to 0 upon restart; once set, the value remains the same until changed by the user. The current value of the adjustment remains unchanged if a **[RET]** is entered in response to the value request.

The following example shows a display for the A subcommand:

```
D> A
VALUE? 12.5
ADJUST = 12.5
D>
```

D - DISPLAY VARIABLE. The DISPLAY VARIABLE **(D)** subcommand repeatedly prints the value for a specified variable at the rate of one line per second in the format type specified. There are 5 data types: logical, **BCD/hex,** count, fixed, or real. A [RET] during the display stops the display. The DMC displays the symbolic name if a default variable name is used and a symbol exists for it. Note that the subcommand requests a binary point if a default variable with a fixed point data type is requested. The symbolic fixed point variables do not require a binary point since the information resides in the symbol table.
The D subcommand uses the last symbol displayed or modified if a **[RET]** is entered for the display prompt.

The following example shows a display for the D subcommand:

```
D> D
DISPLAY? .V0020R
MTR-SPD = 120.29999
MTR-SPD = 125.5
MTR-SPD = 140.81250
MTR-SPD = 141.99999
MTR-SPD = 142.00001
MTR-SPD = 142.0
MTR-SPD = 142.00001  [RET]
D>
```

E - ENTER MULTIPLE DISPLAY VARIABLES. The ENTER MULTIPLE DISPLAY VARIABLES (E) subcommand enters into the system the variables to be shown by the MULTIPLE VARIABLE display. (Refer to the the R subcommand's description for further information.) The system prompts the user for up to 4 variables. The user can abort the input sequence at any time by entering an asterisk (*) or Control-Z. A [RET] entered to the prompts provides the regular means of using the previously defined variables. A blank space input at the prompts deletes the current displayed entry for the prompted variable.

The following examples show displays for the E subcommand:

```
D> E
VAR 1? HOURS [RET]
VAR 2? MINUTES [RET]
VAR 3? SECONDS [RET]
VAR 4? MILLS [RET]
D>
```

```
D> E
VAR 1? HOURS
VAR 2? _ [RET]
VAR 3? [RET]
VAR 4? SECONDS [RET]
D>
```

```
D> E
VAR 1? DAYS [RET]
VAR 2? * [RET]
D>
```

F - FAST DISPLAY. The FAST DISPLAY (F) subcommand is the same as the DISPLAY VARIABLE (D) subcommand except that the F subcommand displays the data as fast as the terminal allows. Note that using the F subcommand for a terminal with a fast baud rate can cause excessive DMC loading.

M - MODIFY VARIABLE. The MODIFY VARIABLE **(M)** subcommand changes a variable's current value. The variable specified has a data type associated with it and the new variable entered must be of the same type. The following table lists the valid characters for each data type:

LOGICAL - 0, 1
GROUP - **0, 1,** 2, 3, 4, **5,** 6, 7, **8, 9,** A, **B, C, D, E, F**
BCD/HEX - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
REAL - **0, 1,** 2, 3, 4, 5, 6, 7, 8, **9, ., -**
FIX - **0, 1,** 2, 3, 4, 5, 6, 7, 8, **9, -**

The system rejects the new value if it contains illegal characters. Real numbers can only be modified on systems with the 8087 co-processor. The modified value remains the same if no new value is entered. If no name is entered, the variable to be displayed defaults to the last modified or displayed variable. An attempt to modify a fixed point number using its default name will request the binary point. This binary point is limited between -12 and **+16.** The following example shows a display for the M subcommand:

```
D> M
MODIFY? .C0010B [RET]
.C0010B=1234
VALUE? AA55
.C0010B= AA55
D> M
MODIFY? SPEED [RET]
SPEED =109243.0
VALUE? [RET]
SPEED =109243.0
D>
```

R - RUN MULTIPLE VARIABLE DISPLAY. The RUN MULTIPLE VARIABLE DISPLAY **(R)** subcommand begins and continues to display the variables previously entered via the ENTER MULTIPLE DISPLAY VARIABLES **(E)** subcommand. A **[RET]** stops the display. The DMC prints the display at the rate of 1 line per second, unless the space bar is held down. In that case, the display updates as fast as possible, but has the same effect on processing time as the F subcommand.

The following example shows a display for the R subcommand:

```
D> R

HOURS = 10     MINUTES = 58     SECONDS = 33     MILLS = 350
HOURS = 10     MINUTES = 58     SECONDS = 34     MILLS = 350
HOURS = 10     MINUTES = 58     SECONDS = 35     MILLS = 350
```

S – STATION I/O DISPLAY. The STATION I/O DISPLAY (S) subcomtnand prints an entire rack of I/O input or output. The DMC displays the data in group format, with the I/O configuration determining the number of groups. Only active I/O groups are shown. The S subcommand defaults to the last displayed variable if the variable is an I/O type. Note that the logical points within a group differs from the group data until an I/O execution on that rack.

The following example shows a display for the S subcommand:

```
D> S
DISPLAY?  .OCC00G
.OCC00G = 24
.OCC01G = 38
IN-DATA = 55
.OCC03G = 10
D>
```

"+" – ADD ADJUSTMENT VALUE. The ADD ADJUSTMENT VALUE (+) subcommand provides incremental adjustment to the last displayed variable. The variable increments by 1 unit each time the "+" subcommand is issued, unless it is a real variable. In that case, the current value defined by the ADJUST VALUE FOR REAL (A) subcommand is added to the variable. Logical data types are not valid in this subcommand. The only case in which a single unit value does not equal an integer 1 is for a fixed data type. The unit for fixed point data types vary with the binary point assignment. The value is not permitted to overflow on fix and count data types.

The following example shows a display for the "+" subcommand:

```
D> +
LIMIT = 1026
D> +
LIMIT = 1027
D> +
LIMIT = 1028
D> +
LIMIT = 1029
D> +
LIMIT = 1030
D>
```

"-" - SUBTRACT ADJUSTMENT VALUE. The SUBTRACT ADJUSTMENT VALUE **(-)** subcommand is the same as the ADD ADJUSTMENT VALUE **(+)** subcommand, except that it subtracts from the display variable. All other restrictions are the same.

The following example shows a display for the **"-"** subcommand, assuming the previously executed ADJUST VALUE FOR REAL **(A)** subcommand set the adjust value to 0.15:

```
D> -
.C0030R = 20.3
D> -
.C0030R = 20.15
D> -
.C0030R = 20.0
D> -
.C0030R =  19.85
D> -
.C0030R = 19.7
D>
```

Q - QUIT. The QUIT **(Q)** subcommand exits the DISPLAY command and returns to the main command level. The following example shows a display for the Q subcommand:

```
D> Q
*>
```

E - TARGETEDITORCommand

The DMC's TARGET EDITOR (E) command is a line editor that adds, deletes, modifies, and displays the CBL program. The editor handles elementary instructions and CBL library blocks. It does not provide graphic representation of the application code. The following list defines the elementary instructions for the E command:

- LD - Load logical onto bit stack
- LDF - Load inverted logical onto bit stack
- AN - And logical value with top of stack
- ANF - And inverted logical with top of stack
- ANS - And top two elements of the stack
- OR - Or logical value with top of stack
- ORF - Or inverted logical with top of stack
- ORS - Or top two elements of the stack
- [illegible] - No operation
- STO - Store stack top into logical value
- BLK - CBL library block

All elementary instructions, except for STO and BLK, require 1 line in the segment instruction. The STO instruction takes 2 segment lines: 1 for the instruction and 1 for the block number. The BLK instruction takes a variable number of lines: 1 for the instruction, 1 for the block number, and 1 for each parameter the block requires.

Multiple elementary instructions form a block of application logic. Either a STO or BLK instruction, or both, terminates a block. Multiple STO instructions can reference a block if the block number portion of the instructions are the same. Multiple STO instructions do not require intervening instructions between them. The next block starts with the first instruction that is not a BLK or STO. The blocks are numbered in ascending order from 1 to 9999.

The editor addresses 1 segment at a time. A segment is created at system configuration, and the user cannot add or delete a segment. A segment consists of multiple blocks, and is terminated by an end of segment mark (EOS). Instructions cannot be inserted after the EOS.

Logical input parameters reside on the bit stack, which transfers to the CBL library block. The logical input parameters transfer in reverse order. (Reverse order means that the first logical bit to the stack is the last input to the block; the last logical input to the stack is normally the enable logical bit.) Block parameters are located after the BLK call and block number.

Most CBL blocks return a logical output status on the bit stack when they exit. A STO instruction saves the status if it is needed later.

The DMC requires an 8087 coprocessor for segments that contain parameters with real data types. The editor does not accept BLK instructions with real parameters if the segment was not created with 8087 access during system configuration. The DMC aborts the insert if this is attempted.

Most elementary instructions require a logical operand. This operand can originate from the logical table or an I/O bit address. No logicals can be defined from the variable or constant table.

System configuration determines the total amount of space available for CBL programs. Each line of code entered by the editor uses 1 entry in the code table for the CBL program. The editor cannot insert any more logic in any segment if the code table is full. The LOOK command displays the the code table's current usage.

Many editor commands use the line pointer's current value during execution. The current line pointer points at the last line displayed in an operation: a block display request sets the line pointer to the last line of a block; an insert sets the current line pointer to the last line inserted. To display the current line, enter the LINE (L) subcommand and a [RET].

The editor requests a segment edit as soon as it is set active. The user can edit only this segment until the editor is terminated and recalled. The segment can be defaulted and the last edited segment put into the edit mode. When the editor starts, it displays the segment's first line.

The following example shows a display for entering the E command:

```
*> E
SEG? USRSEGP
0001: LD TRUE
E>
```

H - HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the TARGET EDITOR command:

```
E> H
B--BLOCK
D--DELETE
H--HELP
I--INSERT
L--LINE
M--MODIFY
N--NEXT BLOCK
P--PREV BLOCK
Q--QUIT  COMMAND
S--SEGMENT
+ NEXT LINE
- PREV LINE
E>
```

B - BLOCK DISPLAY. The BLOCK DISPLAY (B) subcommand displays the specified application block. If the block specified does not exist, the subcommand prints the next block after the specified block number. This allows the user to access the segment's first block by specifying block 0, and to access the end of segment (EOS) marker by specifying block 9999. The DMC displays the current block if a block number is not specified. When the subcommand completes, the DMC sets the current line pointer as the block's last line.

The following example shows a display for the B subcommand:

```
E> B
BLOCK? 100
0044:LD   RESET
0045:AN .I10004
0046:LD   SET
0047:BLK   KLCHOO
0048:B # 100
0049:P-O TEMP
0050:ST0   LATCH
0051:B # 100
E>
```

D - DELETE INSTRUCTION, The DELETE INSTRUCTION (D) subcommand deletes the current instruction from the segment. The current line must be the first line of a multiple line instruction. The D subcommand deletes the entire instruction, but cannot delete a block number or a BLK parameter. The DMC echoes the lines to be deleted so that the user can abort the request before the deletion executes. After the deletion completes, the DMC renumbers all lines following the instruction.

The D subcommand cannot be used when the primary system resides in EEPROM. The subcommand can execute while the controller is on-line, but all segments halt during the deletion. To prevent potential control problems, this delay should be considered.

The following example shows a display for the D subcommand:

```
E> D
0047:BLK   KLCHOO
0048:B # 100
0049:P-O TEMP
OK   - (Y/N) Y
E>
```

I - INSERT INSTRUCTION. The INSERT INSTRUCTION (I) subcommand inserts an instruction into the segment. The system requires a single space between an elementary instruction and its operand. Any other spacing produces an error.

A [RET] aborts the I subcommand at any time when the subcommand prompts for the next line. The DMC asks if the inserted instruction is OK before it inserts the instruction into the segment. It then renumbers all lines following the insert.

The I subcommand verifies that the block number and parameter data type are valid as the user enters these fields. If the system detects an error, it ignores the input and requests the line again.

The I subcommand prompts for the line number that the next instruction should follow. Code cannot be inserted after an EOS instruction. To insert code before the segment's first instruction, the user must specify line 0. If a line number is not specified, the DMC inserts the line after the current line.

The I subcommand follows the same restrictions as the DELETE (D) subcommand: the DMC cannot execute the I subcommand when the system is in EEPROM, and all segments halt momentarily if the DMC is on-line while the subcommand inserts the logic.

The following example shows a display for the I subcommand:

```
E> I
LINE? 0 [RET]
0001: LD PICKUP [RET]
OK? - (Y/N) Y
E> I
LINE ? [RET]
0002: BLK KDLY00 [RET]
BLOCK #? 10 [RET]
P-0: .L0010 [RET]
OK? - (Y/N) Y
E> I
LINE ? [RET]
0005: STO DELAY [RET]
BLOCK #? 10 [RET]
OK? - (Y/N) Y
E>
```

L – DISPLAY LINE. The DISPLAY LINE **(L)** subcommand displays a specific line within the segment. When the L subcommand completes, it sets the current line pointer to this line. The DMC displays an error message if the current line exceeds the end of the segment. To display the current line, enter a **[RET]** for the line number.

The following example shows a display for the L subcommand:

```
E> L
LINE ? [RET]
0049:P-0 TEMP
E> L
LINE? 1
0001:LD  TRUE
E>
```

M – MODIFY LINE. The MODIFY LINE **(L)** subcommand changes 1 line within the segment. When entered, the subcommand displays the current line number and requests the modification. The M subcommand does not allow the user to modify the first line of a **STO** or BLK instruction. To change these instructions, the user must use a DELETE **(D)** subcommand followed by an INSERT **(I)** subcommand. The user can modify block numbers and parameters using the same restrictions as for the INSERT INSTRUCTION **(I)** subcommand. Entering a **[RET]** for the modification prevents a change to the line.

The M subcommand cannot execute when the system resides in EEPROM. The subcommand executes with the segments halted momentarily, but the time is shorter than with a D or I subcommand.

The following example shows a display for the M subcommand:

```
E> M
0008:  ANF  ALARM
MOD – OR FIRBEL [RET]
E> L
LINE? 10 [RET]
0010: B # 30
E> M
0010: B # 30
BLOCK #? 35
E> +
0011:  P-0  LIMIT
E> M
0011:  F-0  LIMIT
MOD – .C0044C
E>
```

N – DISPLAY NEXT BLOCK. The DISPLAY NEXT BLOCK **(N)** subcommand displays the next block in the segment after the current block. Entering the subcommand after the end of segment generates an error. The subcommand sets the current line pointer to the last line of the block displayed.

The following example shows a display for the N subcommand:

```
E> N
0004:  LDF FLIPPER
0005:  STO FLIPPER
0006:  B # 20
E> N
0007:  LD TRUE
0008:  BLK  21000
0009:  B # 30
0010:  P-O  2
0011:  STO .LOO2O
0012:  B # 30
E> N
0013:  EOS
E>
```

P – DISPLAY PREVIOUS BLOCK. The DISPLAY PREVIOUS BLOCK **(P)** subcommand displays the segment's previous block before the current block. Entering the subcommand at the beginning of the segment generates an error. The current line pointer is set to the last line of the block displayed. The following example shows a display for the P subcommand:

```
E> P
0007:  LD  TRUE
0008:  BLK  21000
0009:  B # 30
0010:  P-O  2
0011:  STO .LOO2O
0012:  B # 30
E> P
0004:  LDF FLIPPER
0005:  STO FLIPPER
0006:  B # 20
E> P
0001:  LDF  ENABLE
0002:  BLK  ZEXITOO
0003:  B # 10
E>
```

S - DISPLAY THE ENTIRE SEGMENT. The DISPLAY THE ENTIRE SEGMENT (S) subcommand displays the entire segment and sets the line pointer set to the end of segment (EOS) instruction upon completion. To abort the display, enter a [RET] while the list is printing.

The following example shows a display for the S subcommand:

```
E> S
0001:  LDF ENABLE
0002:  BLK ZEXITOO
0003:  B # 10
:
0004: LDP FLIPPER
0005:  STO FLIPPER
0006:  B # 20
:
0007: LD TRUE
0008:  BLK ZIOOO
0009:  B # 30
0010:  P-O 2
0011:  STO .L0020
0012:  B # 30
:
0013: EOS
E>
```

"+" - DISPLAY NEXT LINE. The DISPLAY NEXT LINE (+) subcommand displays the next line in the segment. Entering the subcommand after the end of segment generates an error.

The following example shows a display for the + subcommand:

```
E> +
0007:  LD  TRUE
E> +
0008:  BLK  ZIOOO
E> +
0009: B # 30
E>
```

"–" – DISPLAY PREVIOUS LINE. The DISPLAY PREVIOUS LINE (–) subcommand displays the previous line in the segment. Entering the subcommand at the beginning of the segment generates an error. The following example shows a display for the – subcommand:

```
E> -
0008:  ELK  ZIOOO
E> =
0007:  LD  TRUE
E>
```

Q – QUIT. The QUIT subcommand exits the EDITOR command and returns to the main command level.

The following example shows a display for the Q subcommand:

```
E> Q
*>
```

F - FORCE LOGICAL SIGNALS Command

The FORCE LOGICAL SIGNALS **(F)** command allows the user to force a logical signal to either a high state (logic **1)** or to a low state (logic **0).** The signal stays forced until the system restarts or the system goes off-line then back on-line. The forced bit is used in internal logic and for output to external I/O.

Forcing any number of logical signals adds 6 microseconds to the execution time of all **STO** instructions.

The system ignores any attempt to force temporary logic variables for CBL blocks.

The following sequence initiates the FORCE LOGICAL SIGNALS command from the main command level:

```
*> F
F>
```

H - HELP INFORMATION. The HELP INFORMATION **(H)** subcommand displays the following list of subcommands available within the FORCE LOGICAL SIGNALS command:

```
F> H
C--REMOVE  ALL
D--DISPLAY
F--FORCE
H--HELP
N--NEXT
P--PREVIOUS
Q--QUIT  COMMAND
R--REMOVE
F>
```

C - CLEAR ALL FORCED POINTS. The CLEAR ALL FORCED POINTS **(C)** subcommand clears all outstanding forced **logicals.** The command removes all force indicators and the logical points remain in the currently forced state. If there are no outstanding forced signals, the DMC prints an error message.

The following example shows a display for the C subcommand:

```
F> C
REMOVE ALL? Y
COMMAND  COMPLETE
F> C
NO  FORCED  POINTS
F>
```

D - DISPLAY LOGICAL. The DISPLAY LOGICAL (D) subcommand displays a logical variable and indicates whether or not it is forced. A [RET] after the prompt displays all forced signals if forced points exist; if forced points do not exist a message is printed. The following example shows a display for the D subcommand:

```
F> D
DISPLAY?.L0001 [RET]
.L0001 = 1
F> D
DISPLAY? [RET]
.L0020 = 0 F
.L0030 = 1 F
.ICC003 = 1 F
INPBIT = 0 F
.O10000 = 1 F
F>
```

F - FORCE LOGICAL. The FORCE LOGICAL (F) subcommand forces a logical signal to either a high state (logic 1) or to a low state (logic 0), and ignores any other value. The F subcommand displays the current value of the logical before the monitor requests the new value. The F subcommand can force previously forced logicals to their opposite state. (Remember that forcing does cause some overhead.) If a new value is not specified, the forced logical defaults to the last displayed logical.

The following example shows a display for the F subcommand:

```
F> F
FORCE? INP BIT [RET]
INP-BIT = 0
STATE = ? 1 [RET]
INP-BIT = 1 F
F>
```

N - NEXT FORCED STATE. The NEXT FORCED STATE (N) subcommand lists the next forced logical in the logical table. The list wraps around when it reaches the end.

The following example shows a display for the N subcommand:

```
F> N
.L0020 = 0 F
F> N
.L0030 = 1 F
F> N
.ICC003 = 1 F
F>
```

P - PREVIOUS FORCED STATE. The PREVIOUS FORCED STATE (P) subcommand lists the previous forced logical in the logical table. The list wraps around when it reaches the beginning.

The following example shows a display for the P subcommand:

```
F> P
.L0030= 1 F
F> P
.L0020= 0 F
F> P
.O10000 = 1 F
F>
```

R - RELEASE FORCED STATE. The RELEASE FORCED STATE (R) subcommand removes the force state from a logical variable. The R subcommand defaults to the last displayed logical if a variable is not entered at the prompt. To release all forced logicals, use the CLEAR ALL FORCED POINTS (C) subcommand.

The following example shows a display for the R subcommand:

```
F> R
REMOVE? .L0020 [RET]
.L0020 = 0
F>
```

Q - QUIT. The QUIT (Q) subcommand exits the FORCE command and returns to the main command level. The following example shows a display for the Q subcommand:

```
F> Q
*>
```

H – HELP Command

The HELP **(H)** command lists the names of each main command provided in the system. While in the HELP command, typing in a command letter prints out the list of subcommands available for that command. This is the same information displayed by the HELP subcommand within the command itself. For the HELP listing for a particular command, refer to the HELP subcommand description in the particular command.

The following sequence initiates the HELP command from the main command level:

```
*> H
A- -ASSIGN
B- -BREAKPOINT
C- -CBL BLOCKS
D- -DISPLAY/MOD
E- -EDITOR
F- -FORCE
H- -HELP
I- -I/O
L- -LOOK
M- -MODIFY
o- -ONLINE STAT
P- -PASSWORD
Q- -QUIT MONITOR
s- -SEGMENTS
T- -TERMINAL
U- -USER MONITOR
W- -WHERE USED
X- -XFER SYSTEM
H>
```

Q – QUIT. The QUIT **(Q)** subcommand exits the HELP command and returns to the main command level. The following example shows a display for the Q subcommand:

```
H> Q
*>
```

I - I/O Command

The I/O command displays the channel, stations, and racks defined to the system. This information was set during system configuration and cannot be modified, except for the number of active input or output groups on a rack. For any other I/O modification, the system must be reconfigured. (Refer to Chapter 2, Figure 2-5.)

The following sequence initiates the I/O command from the main command level:

```
*> I
I>
```

H - HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the I/O command:

```
I> H
C--CHANNEL
H--HELP
M--MODIFY
S--STATION
Q--QUIT   COMMAND
I>
```

C - DISPLAY CHANNEL INFORMATION. The DISPLAY CHANNEL INFORMATION (C) subcommand displays the stations and racks on the channel. A [RET] entered for the channel prompt displays information for all channels. An error results if a requested channel does not exist.

The following example shows a display for the C subcommand:

```
I> C
CHANNEL? 0 [RET]
CHANNEL # 00:
STN #10, RACK: 0
I> C
CHANNEL? [RET]
CHANNEL # 00:
STN #10, RACK: 0
CHANNEL # 02:
STN #CC, RACK: 0
STN #20, RACK: 0
STN #21, RACK: 2
STN 821, RACK: 4
CHANNEL # 13:
STN #88, RACK: 0
STN #88, RACK: 2
I>
```

M - MODIFY RACK INFORMATION. The MODIFY RACK INFORMATION (M) subcommand changes the number of active groups within a specific rack and station. This subcommand requires that the the controller be off-line. The active group count can range from 0 to 16. The default for station and rack are the last defined values for the parameters. If a new value is not entered for the group, the value stays at the previous one.

The following example shows a display for the M subcommand:

```
I> M
STATION? 10 [RET]
RACK? 0 [RET]
IN: 4, IN? 8 [RET]
OUT: 4, OUT? [RET]
RACK:O, I: 8,0: 4
I>
```

S - STATION DISPLAY. The STATION DISPLAY (S) subcommand displays the number of input and output groups associated with each rack within the station. If a station is not specified, the subcommand provides this information for all stations. The DMC displays an error if the station does not exist.

The following example shows a display for the S subcommand:

```
I> S
STATION? 88 [RET]
STATION #88:
RACK:O, I: 6,0: 6
RACK:2, I: 10,0: 0
I>
```

Q - QUIT. The QUIT (Q) subconunand exits the I/O command and returns to the main command level. The following example shows a display for the Q subcommand:

```
I> Q
*>
```

L – LOOK Command

The LOOK (L) command prints out system statistics, which are listed and defined below:

- ID – The system identification (ID) is set during 'system configuration. The ID consists of a g-character ASCII string selected by the system engineer.

- DATE – The date indicates the date of the most recent system configuration. The date changes everytime the system is reconfigured.

- SYS SIZ – The system size represents the total RAM, BRAM, and EEPROM defined at configuration. The memory need not be contiguous. The memory represented by this value may not exist.

- CODE – The system's code size provides statistics on the amount of space left in the system for editing segments. The first value represents the number of lines of CBL code being used by all segments in the system; the second value represents the maximum number of lines that this system can support. The only way to change the maximum table size is to reconfigure the system. The current usage size can vary from adding and deleting lines through the TARGET EDITOR command.

- SyMBOL – The symbol table size provides statistics on the number of symbol assignments in the system. The first value represents the number of symbols currently defined; the second value represents the maximum number of symbols the target controller can hold. As in code size, the only way to change the maximum number is to reconfigure the system. By assigning and releasing symbols, the ASSIGN (A) command changes the current usage value.

- I/O OVERRUN – The I/O overrun counts the times that an HMHA or HXCA board's I/O channel failed because its I/O had not completed when addressed by an Execute I/O command (ZIOOO). The board returns an error logical status to the segment each time an I/O operation fails during a scan. The overrun count accumulates as the system runs. The system resets this value when the system goes on-line, and the value should normally be 0. To eliminate I/O overruns, the user should increase the scan time of the segment performing the I/O operation.

- IDLE TIME – The idle time value indicates the amount of time spent in either the monitor or the idle loop. This value can include some interrupt processing time if the interrupt occured during the monitor or idle loop execution time. The value is a rolling average over the last 60 seconds. This means that the idle time changes gradually as segments are enabled and disabled. The value does reflect scheduling overhead and segment start up and termination time. Because of this, the idle time may not equal 100% even when the controller is off-line. Note that if this value reaches 0, the HMPH's IMOK light turns off.

The displayed information is read-only, so there are no subcommands for the L command. When the L command terminates, control passes back to the monitor's main command level.

The following example shows a display for the L command:

```
*> L
ID : CRNZ 1.0
DATE : AUG 19 83
SYS SIZ = 32 K
CODE = 2164/ 2500
SYMBOL = 195/ 200
IO OVERRUN: 0
IDLE TIME = 37%
*>
```

Note that the L command returns directly to the main command level; there are no subcommands.

M - MODIFY VARIABLE Command

The MODIFY VARIABLE **(M)** command provides an alternative way to access the DISPLAY/MODIFY **(D)** command. All the commands are the same except for the initial entry. The M command starts out by requesting a variable to be modified, which is the same format as the D command.

Note that when the monitor receives the M command input, it immediately enters into the D command's M (MODIFY VARIABLE) subcommand (described previously). This is done automatically for convenience to the user. A [RET] terminates the M subcommand and returns to the command level prompt **"M".** At this time, the user can enter another subcommand. The M command provides the same subcommands as the DISPLAY/MODIFY **(D)** command. For information about these subcommands, refer to the previous section, **DISPLAY/MODIFY Subcommands."

The following example shows a display for the M subcommand:

```
*> M
DISPLAY/MOD
MODIFY? COUNT [RET]
COUNT = 2400
VALUE? 1400 [RET]
COUNT = 1400
M>
```

Note that the MODIFY VARIABLE command **prompt,M>,** is displayed. Otherwise, the operation is the same as when the function is entered via the DISPLAY/MODIFY **(D)** command.

O - ONLINE STATUS Command

The ONLINE STATUS (O) command enables and disables I/O channels or segments, and also controls the DMC's on-line/off line status. The 0 command helps bring up the system when all the hardware is not available.

The following sequence initiates the ONLINE STATUS command from the main command level:

```
*> O
O>
```

H - HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the ONLINE STATUS command:

```
O> H
C--CONTROLLER
D--DISPLAY
H--HELP
I--IO
O--OUTPUT
Q--QUIT  COMMAND
S--SEGMENT
O>
```

C - CONTROLLER STATUS. The CONTROLLER STATUS (C) subcommand controls the DMC's on-line status, including the execution of all segments and all I/O. The C subcommand toggles between on-line and off-line status.

When going from off-line to on-line, the system must start up all of its I/O channels and initiate all of the segments. To do this, the system executes the following steps:

1) Sets all data tables to 0
2) Resets all forced logicals
3) Initializes all I/O channels
4) Sets initial status flag for segments
5) Executes initialization segment
6) Turns on the HMPH's red "TEST" LED

The system now comes up with the system standard outputs disabled for the first second of execution. The DMC then sets outputs to the output status that was specified when the system went off-line. This l-second pause enables the outputs to stabilize before being broadcast.

The CONTROLLER ONLINE STATUS (C) command requires several seconds to execute because of I/O delays during initialization, The monitor does not receive control until the entire initialization process completes. Any I/O initialization error keeps the DMC off-line. The DMC displays the I/O channel that failed initialization. If a channel does not exist, the user should disable that channel placing the DMC on-line.

When going from on-line to off-line, the system executes the following steps:

1) Resets all HMHA channels
2) Sets all other I/O channel outputs to 0
3) Suspends segment activation
4) Turns off the HMPH's red **TEST** LED

Segments can no longer restart when the controller is off-line. The DMC allows segments that were active at the time of the call to complete. However, the segment cannot restart itself or another segment.

The DMC's state changes if a "Y" is entered at the modify prompt.

The following example shows a display for the C subcommand:

```
O> C
CONTROLLER = OFF
MODIFY? Y
CONTROLLER = ON
O> C
CONTROLLER = ON
MODIFY? [RET]
CONTROLLER = ON
O>
```

D - DISPLAY CURRENT ONLINE STATUS. The DISPLAY CURRENT ONLINE STATUS (D) subcommand lists the DMC's current on-line status, output status, I/O channels, and segments. The following example shows a display for the D subcommand:

```
O> D
CONTROLLER = ON
OUTPUT = ON
POWERUP = ON
USRSEG1 = OFF
USRSEG2 = ON
CHANNEL 00 = ON
CHANNEL 02 = OFF
CHANNEL 03 = OFF
CHANNEL OF = ON
```

I - I/O CHANNEL STATUS. The I/O CHANNEL STATUS (I) subcommand disables an I/O channel, and therefore disables all stations on the channel. A disabled channel cannot be initialized, and it ignores any attempted system I/O operation (ZIOOO). The I subcommand is used when an I/O channel does not exist in the system, but will be included later.

Disabling the I/O with the ONLINE (O) command's I/O CHANNEL STATUS (I) subcommand resembles setting I/O groups to 0 using the I/O (I) command's MODIFY RACK (M) subcommand. The major difference between the two is that the ONLINE command's I/O (I) subcommand disables channel initialization. The general rule is that if the entire channel needs to be disabled, use the ONLINE command's I subcomtnand. If only 1 station or rack within a channel is missing, use the I/O command's M subcommand to set the I/O groups to zero.

Note that if a channel needs special initialization to get started, such as with all HMHA channels, the controller must be turned off-line and back on-line before it can respond. The HMHA I/O can only be initialized when the controller is placed on-line.

To modify the current state of the channel, the user must enter a "Y" at the *'MODIFY?** prompt. The channel ID then defaults to the last channel specified.

The following example shows a display for the I subcommand:

```
O> I
CHANNEL? 02 [RET]
CHANNEL 02 = ON
MODIFY? Y
CHANNEL 02 = OFF
O> I
CHANNEL? [RET]
CHANNEL 02 = OFF
MODIFY? [RET]
CHANNEL 02 = OFF
O>
```

0 - OUTPUT STATUS CONTROL. The OUTPUT STATUS CONTROL (O) subcommand disables outputs to all standard output hardware. The channel inputs continue to be enabled.

The following example shows a display for the 0 subcommand:

```
O> O
OUTPUT = OFF
MODIFY? Y
OUTPUT = ON
O>
```

S - SEGMENT STATUS CONTROL. The segment status control enables or disables segments. All segments (interrupt, run, and initial) can be disabled with this subcommand. The S subcommand can be used to disable segments during debugging so that the system contains only 1 active segment.

If a segment name is not entered, the segment name defaults to the previous name entered. To modify the current state of the segment, the user must enter a Y at the MODIFY? prompt.

The following example shows a display for the S subcommand:

```
O> S
SEGMENT?  USRSEGP [RET]
USRSEGP = OFF
MODIFY? Y
USRSEG2 = ON
O> S
SEGMENT? [RET]
USRSEGP = ON
MODIFY? [RET]
USRSEG2 = ON
O>
```

Q - QUIT. The QUIT subcommand exits the ON LINE command and returns to the main command level. The following example shows a display for the Q subcommand:

```
O> Q
*>
```

P – PASSWORD Command

The PASSWORD (P) command provides both Level 1 and Level 2 memory access to the system user. For more information about levels of access, refer to the previous description under "Memory Protection Levels'*.

To obtain Level 1 or Level 2 access, type in the P command, and then enter a password at the prompt. The required password is set for each access level during system configuration and is identified in the requisition documentation. Password inputs are not echoed. Following the password entry and a [RET], the monitor displays the access level.

The system aborts all commands attempted without the required access level, and displays the error message,'INVALID PASSWORD'.

The following example shows a display for the P command:

```
*> P
PASSWORD
PASSWORD? _____ [RET]    (entered but not echoed)
LEVEL1GRANTED
*> P
PASSWORD
PASSWORD? _____ [RET]    (entered but not echoed)
LEVEL2GRANTED
*>
```

Note that the PASSWORD command contains no subcommands.

Q - QUIT MONITOR Command

The QUIT MONITOR **(Q)** command terminates the monitor and returns the monitor to access Level 0. Once the monitor has been terminated, it enters into the auto-baud detect mode. Several **[RET]** keys need to be entered to restart the monitor once it has terminated.

Monitor input time-outs can automatically terminate the monitor. When no keyboard inputs are entered during the monitor time-out time, the monitor aborts the current input and jumps to the next highest command level. The monitor terminates when a time-out occurs at the top command level. The time required to cause an input time-out is set during system reconfiguration.

The following example shows the display for the Q command:

```
*> Q
QUIT  MONITOR
```

S - SEGMENT CONTROL Command

The SEGMENT CONTROL (S) command lists all the segments and their run type. This command also allows changes to the cycle time of "run" segments.

The following sequence initiates the SEQUENCE CONTROL command from the main command level:

```
*> S
S>
```

H - HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the SEGMENT CONTROL command:

```
S> H
D--DISPLAY
H--HELP
M--MODIFY SCAN
Q--QUIT  COMMAND
S>
```

D - DISPLAY SEGMENTS. The DISPLAY SEGMENTS (D) subcommand displays the segment name and its cycle time or interrupt number. A run time of *INITIAL* denotes the initialization segment. A run segment not executed on a cyclic basis is typed 'NO SCAN'. The subcommand prints this information for all segments within the system. It lists the segments in order of priority, with the highest priority segment at the beginning.

The following example shows a display for the D subcommand:

```
S> D
POWERUP, INITIAL
USRSEG1, 320 MS
USRSEG2, 50 MS
HCMAINT, INT 37
S>
```

M - MODIFY RUN SEGMENT CYCLE. The MODIFY RUN SEGMENT CYCLE **(M)** subcommand allows the user to change the scan cycle of a periodic segment. The user, however, cannot modify the interrupt or initial status of segments, nor change segments from a periodic segment to an interrupt segment. The following units can be designated for the periodic segment:

- M - Milliseconds
- S - Seconds
- M - Minutes
- H - Hours
- D - Days

The segment's cyclic scan rate can be set from 1 millisecond to 100 days, or without a scan rate. A scan rate set to **'0'** sets up the segment without a scan rate. However, a segment without a scan rate can only be activated by a second segment issuing a start segment command.

A periodic segment without a scan rate resembles a segment disabled by the ONLINE command. However, a segment with no scan rate runs if started by another segment, and a disabled segment does not. Additionally, a segment set with no scan rate causes less overhead in the system than if it were disabled.

The M subcommand displays the segment name and interrupt level of interrupt segments, but does not prompt for any changes, If the segment name is not entered, the system defaults to the last segment specified. The segment's scan rate stays at the set value until a new value is entered. The following example shows a display for the M subcommand:

```
S> M
SEGMENT? USRSEG2 [RET]
USRSEG2,  10  MS
SCAN? 0M [RET]
USRSEG2, NO SCAN
S> M
SEGMENT?  [RET]
USRSEG2, NO SCAN
SCAN? 5S
USRSEG2  5 S
S>
```

Q - QUIT. The QUIT **(Q)** subcommand exits the S command and returns to the main command level. The following example shows a display for the Q subcommand:

```
S> Q
*>
```

T-TERMINAL SUPPORT Command

The TERMINAL SUPPORT (T) command allows the user to change an RS-232 terminal's baud rate and the mode in which it displays data to the screen.

The following sequence initiates the TERMINAL SUPPORT command from the main command level:

```
*> T
T>
```

H – HELP INFORMATION. The HELP INFORMATION (H) subcommand displays the following list of subcommands available within the TERMINAL SUPPORT command:

```
T> H
B--BAUD RATE
H--HELP
L--LINE
Q--QUIT COMMAND
S--SCROLL
T>
```

B – SET TERMINAL BAUD RATE. The SET TERMINAL BAUD RATE (B) subcommand changes the terminal's baud rate. The initial baud rate is set during system configuration, and is used every time the system is restarted. The B subcommand allows the user to change the baud rate to 300, 600, 1200, 2400, 4800 and 9600 baud. The new baud rate takes effect upon entry. The user must then set the terminal to the same baud rate and enter a [RET] to get back to the system.

The following example shows a display for the B subcommand:

```
T> B
BAUD RATE? 9600 [RET]
COMMAND COMPLETE
T>
```

The current baud rate can be set at the default baud rate by entering a [RET] only in reponse to the B subcommand's "BAUD RATE?" prompt. This selection then displays the power-up self-test messages. Note that the auto-baud detect only takes effect after the DMC self-test completes.

L – LINE MODE (HANDHELD). The LINE MODE **(L)** subcommand changes the display mode to 'line' mode for handheld terminals. (Use this mode for handheld terminals only.) The line mode assumes a single line, 16 character display. The mode does not display the prompt character unless it receives a **[RET].** All displayed lists delay 1 second between lines. For most subcommands, the output line remains displayed on the terminal. Normally, single letter subcommand entries are not echoed.

The following example shows a display for the L subcommand:

```
T> L
COMMAND   COMPLETE
T>
```

S – SCROLL MODE (CRT). The SCROLL MODE **(S)** subcommand changes the display mode to 'scroll' mode for a multiple line display screen. (Use this mode for multiple line terminals only.) The scroll mode provides prompt characters and displays all lists at the selected baud rate.

The following example shows a display for the S subcommand:

```
T> S
COMMAND   COMPLETE
T>
```

Q – QUIT. The QUIT **(Q)** subcommand exits the TERMINAL command and returns to the main command level. The following example shows a display for the Q subcommand:

```
T> Q
*>
```

U - USER MONITOR Command

The USER MONITOR (U) command is used to call up special monitor commands if they are included in the system. This command requires the system to contain a special user monitor segment, which must be added during configuration. The user monitor takes over control of the RS-232 terminal and provides extended monitor commands. When the user monitor terminates, it reactivates the system monitor. If no user monitor exists, the DMC displays an error message when it receives the U command.

The following example shows a display for the U command:

```
*> U
USER MONITOR
(User monitor prompts and displays)
*>
```

If a special user monitor segment is included in your system, refer to the requisition documentation for information.

W – WHERE USED Command

The WHERE USED **(W)** command finds where variables are referenced within a segment. The command displays the segment and the block number of the desired signal, and its access type (if it is written, read or both).

The W command finds different symbols that are defined to start at the same table address, but it cannot tell if another data item is defined in the middle of the data structure. For example, if there is a real number defined and at the same location that 2 count symbols are defined, the W command finds the real variable and the first word variable because they start at the same address. It cannot find the second word variable because its start address is different.

The W command always displays the symbol name or address data type of the way the symbol was referenced.

The following sequence initiates the WHERE USED command from the main command level:

```
*> W
W>
```

H – HELP INFORMATION. The HELP INFORMATION **(H)** subcommand displays the following list of subcommands available within the WHERE USED command:

```
W> H
F--FIND   VARIABLE
H--HELP
N--NEXT
P--PREVIOUS
Q--QUIT   COMMAND
w>
```

F - FIND. The FIND (F) subcommand requests, finds, then displays the first reference to a variable address in the system, The variable defaults to the previous one used if no new value is entered at the prompt. The display prints out the variable name, access type, segment name and block number.

The following example shows a display for the F subcommand:

```
W> F
FIND? COUNT [RET]
COUNT , READ
USERSEG1 : 10
w>
```

N - NEXT WHERE USED LOCATION. The NEXT WHERE USED LOCATION (N) subcommand is used after the FIND (F) subconunand to find successive references to the variable address. The N subcommand continues through all segments and terminates when it reaches the end of all segments. The display format is the same as the F subcommand.

The following example shows a display for the N subcommand:

```
W> N
COUNT,  WRITTEN
USERSEGl  :  100
w> N
COUNT, R/W
USERSEG2 : 90
W> N
COUNT , READ
USERSEG2  :  1000
W> N
*END  OF  CODE*
w>
```

P - PREVIOUS WHERE USED LOCATION. The PREVIOUS WHERE USED LOCATION (P) subcommand displays previous 'where used' locations. The P subcommand is used after the FIND (F) and NEXT (N) subcommands to redisplay positions that have already been printed.

The following example shows a display for the P subcommand:

```
W> P
COUNT , READ
USERSEG2 : 1000
W> P
COUNT, R/W
USERSEG2 : 90
W>
```

Q - QUIT. The quit subcommand exits the WHERE USED command and returns to the main command level. The following example shows a display for the Q subcommand:

```
W> Q
*>
```

X – TRANSFER SYSTEM Command

The TRANSFER SYSTEM **(X)** command loads or backs-up the system to EEPROM, TAPE or VAX. Most X subcommands execute with the controller off-line. These subcommands are used to load new systems into system memory. The system restarts itself after all load system operations, and the new system starts execution at that time.

The following sequence initiates the TRANSFER SYSTEM command from the main command level:

```
*> X
X>
```

H - HELP INFORMATION. The HELP INFORMATION **(H)** subcommand displays the following list of subcommands available within the TRANSFER SYSTEM command:

```
X> H
C--SAVE .C TABLE
D--VAX   DOWNLOAD
H--HELP
Q--QUIT   COMMAND
R--RESTORE BRAM
S--SAVE   EEPROM
T--TAPE
U--VAX   UPLOAD
X>
```

C - SAVE .C TABLE. The SAVE .C TABLE (C) subcommand copies all the current values of the constants to the saved memory base in the primary BRAM/EEPROM space. The subcommand can execute with the DMC on-line if the saved memory area is not EEPROM. EEPROM can only be updated with the DMC off-line.

When powered up, the system copies constants from the primary system memory to RAM memory. All changes to constants modify only the RAM image of the constants. The user must enter the C subcommand to save any constants in nonvolatile memory. The old copy of the constants is downloaded if the SAVE CONSTANTS subcommand is not executed before the next time the system restarts.

****
* CAUTION *
**********at

TO SAVE THE MODIFIED CONSTANT VALUES IN EEPROM, THE SAVE .C TABLE (C) SUBCOMMAND MUST BE EXECUTED BEFORE THE SAVE MEMORY (S) SUBCOMMAND IS ENTERED.

The following example shows a display for the C subcommand:

```
X> C
SAVE .C TABLE
COMMAND  COMPLETE
X>
```

D - VAX DOWNLOAD. The VAX DOWNLOAD (D) subcommand copies a system stored on a suitably configured VAX into the target DMC memory. The DMC communicates with the VAX through either a direct line to the VAX or a modem. The user logs onto the VAX, and at the prompts, selects the correct JOB name and MODULE name (discussed below). The user then transfers the corresponding system image down to the DMC. The system restarts with the new image once the transfer completes.

The DMC assumes that the system image is located within a special file maintained by an "MSS" routine on the VAX. This file is identified by a job name and a module name. The job name is the name associated with the DMC system image on the VAX. It is selected when the job is created and identified in the requisition documentation. The module name is always "SEGLINK".

The user logs onto the VAX as part of the D subcommand. The DMC requires a valid USERNAME and PASSWORD input before it logs onto the VAX. If the download is tranferring via modem from the General Electric factory in Salem, Virginia, the user also needs the VAX REMOTE PASSWORD.

The DMC communicates to the VAX through the HMPH board's RS-232 port. This is the same port used by the monitor for a terminal. The cable that attaches the modem to the DMC requires a standard 25 pin straight through connection with one male and ***one*** female connector. No null modem is required.

The HMPH board's red Test light (LED) indicates the transfer state at the present time. The light turns off as the user enters the various prompts for the subcommand. The Test light starts blinking at a slow rate when the last prompt is answered and the message **---CONNECT VAX---" is printed to the console. At this time, the user should disconnect the console terminal and connect the cable from the VAX. The red LED continues to blink at a slow rate until the DMC logs onto the VAX. Once logged on, the light blinks at a faster rate.

During this time, the DMC selects the job and the module that were specified. The Test light stays on once the download file is located and the file transfer starts. The light turns off when the download completes. The VAX cable should then be disconnected and the terminal connected. A **[RET]** entered on the system console displays a termination status message and restarts the **system.**

The Test light turns off any time the DMC detects an error in the above process. To find the problem, disconnect the VAX, connect the system terminal, then enter a [RET]. A summary of the steps is listed below.

ACTION	TESTLED
1) Answer monitor prompts	off
2) Connect VAX - VAX log in	slow blink
3) JOB and MODULE select	fast blink
4) **System** being transferred	on steady
5) VAX logged off - connect CRT	off

The transfer speed is determined by both the baud rate of the VAX link and whether the transfer is an EEPROM write operation. (Writing to EEPROM requires 10 milliseconds per word.)

The user can abort the download after answering all the questions. Continue to enter [RET] to each question until the abort prompt is given. If an error occurs, reenter the responses to the questions.

The following example shows a display for the D subcommand:

```
X> D
USERID: > KRATERG
PASSWORD: > _______ [RET]   (entered but not echoed)
REMOTE PASSWORD: > _______ [RET]   (entered but not echoed)
MSS JOB #: MILLXYZ
MSS MODULE NAME: > SEGLINK [RET]
OK? - (Y/N) Y
BAUD RATE? 4800 [RET]
---CONNECT VAX---
(user connects cable to DMC)
(Test light displays phase of operation, as described above)
(user reconnects terminal)
READY? Y/N
READY? Y/N
READY? Y/N Y
VAX - NO ERRS
SYSTEM RESTART
(system starts self-test procedure)
```

R - RESTORE MEMORY FROM EEPROM. The RESTORE MEMORY FROM EEPROM (R) subcommand copies a 32K block of memory from the secondary memory board to primary memory. The command can restore from 1 to 4 of the primary memory sections as described below

Copy from secondary memory:

2 0000 - 2 7FFF

Copy to primary memory, depending on the selection:

0 4000 - 0 BFFF: Base system memory
E 0000 - E 7FFF: Extended CBL library (0 - 32K) - Part 1
E 8000 - E FFFF: Extended CBL library (32 - 64K) - Part 2
D 0000 - D 7FFF: Extended symbol table

The R subcommand copies as much of the 32K memory as used by the system. When the system receives the subcommand, it prompts for the section of memory to restore (only those memory sections defined to the system).

The R subcommand must be executed off-line. After loading the base system memory, the system is rebooted, which installs the new system. The system comes up off-line. The R subcommand determines the size of the extended memory boards from information stored on the base system. Because of this, the base system memory should be loaded first when multiple sections exist.

NOTE: Do not place the system on-line until the entire system is restored. The system requires all of its memory to operate correctly. When multiple primary segments exist, the power should be turned off and the secondary HUMA boards replaced before the next R subcommand is executed.

The base system memory is the only memory that needs to be restored or saved once the system has been downloaded to the DMC. A monitor command cannot change the CBL library, and only the ASSIGN command can modify the symbol table. (All symbols are typically defined during system configuration.) In most cases, the 3 extended memory boards can be write protected and never modified once they have been downloaded from the VAX.

At power-up, the DMC automatically restores the base system if the primary memory is invalid and the base system memory exists in secondary memory. Note that at power-up, if the base system is in RAM and the base system saved in EEPROM is in segment 2, the system automatically performs a "restore" from EEPROM to the base system. (For more information, refer to this chapter's SYSTEM SELF-TEST section.)

The following example shows a display for the R subcommand:

```
X> R
RESTORE  BRAM
BASE SYSTEM? N
SYMBOL TABLE? N
CBL (0-32K)? N
CBL (32K-64K) N
X> R
RESTORE BRAM
BASE SYSTEM? Y
--WORKING---
COMMAND  COMPLETE
(system will be restarted)
```

S - SAVE MEMORY ON EEPROM. The SAVE MEMORY ON EEPROM (S) subcommand provides the opposite function of the RESTORE subcommand. It transfers memory from the primary memory system to secondary memory, segment 2 addresses 0-7FFFH. This memory usually resides on EEPROM. The S command saves the same 4 memory sections defined in the RESTORE (R) subcommand.

****
* CAUTION *
****

TO SAVE THE MODIFIED CONSTANT VALUES IN EEPROM, THE SAVE .C TABLE (C) SUBCOMMAND MUST BE EXECUTED BEFORE THE SAVE MEMORY (S) SUBCOMMAND IS ENTERED.

The S subcomtnand can be executed on-line. The system requires 10 milliseconds to save each word in EEPROM. However, it does not rewrite a word if the new value is the same as the old value. Since each memory area is 32K bytes (16K words) long, saving an entire board requires 160,000 milliseconds (2 and 2/3 minutes). If no changes are made, the subcommand comes back within a second.

If a memory failure occurs, the system stops the transfer and prints the address of the error. The sections can be saved in any order, but as in the RESTORE subcommand, only the base system memory normally needs saving.

The following example shows a display for the S subcommand:

```
X> S
SAVE  EEPROM
BASE SYSTEM? Y
---WORKING---
COMMAND  COMPLETE
X>
```

T - STR-LNK II OR III TAPE BACKUP. The STR-LNK II OR III TAPE BACKUP (T) subcommand controls the load, dump, and compare tape subcommands. All tape subcommands must be executed off-line. The tape can be run at 300, 600, 1200, 2400, 4800, or 9600 baud, but the type of tape unit limits the transfer speed. The DMC requires a 25-pin RS-232 cable to connect the tape unit to the HMPH board's RS-232 port.

X - XFER SYSTEM **(contd.)**

The **HMPH's** Test light indicates different phases of the transfer:

- The light blinks after the user answers all prompts and connects the tape cable
- The light stays on when the tape unit is set to **remote** mode and while the system transfers
- The light turns off when the operation completes+ At this point, the user reconnects the system terminal then enters a **[RET].** The system then displays the status of the operation.

All tape operations operate on the entire memory space of the system that needs to be saved at power loss. The DMC can copy onto tape a system as large as 140K bytes, although most systems do not exceed 32K bytes. Table 3-l lists the approximate transfer times at several baud rates for different system sizes.

Table 3-l
TAPE STORAGE TRANSFER TIMES

SYSTEM SIZE	300 BITS/SEC	1200 BITS/SEC	9600 BITS/SEC
32K	17.6 **MIN**	4.3 MIN	33 SEC
64K	35.2 MIN	8.6 MIN	66 SEC
96K	52.8 MIN	12.9 MIN	1.6 MIN
140K	76.5 MIN	19.1 MIN	2.4 MIN

The tape can operate at full speed if loading data into RAM. However, when loading into EEPROM, the tape operation requires a slower baud rate than the tape unit allows. (EEPROM can only be updated at a rate of 1 byte every 10 milliseconds. This requires that the tape speed be set to 600 baud [bits per second1 or less).

To prevent transfer problems when loading to EEPROM, first load from the tape into RAM. Then copy the data to EEPROM using the SAVE MEMORY ON EEPROM **(S)** subcommand rather than going directly to EEPROM. Note that the tape operation must be executed off-line while the SAVE subcommand can be executed on-line.

The following example shows a display for the T subcommand:

```
X> T
LOAD/DUMP/COMP? D
BAUD RATE? 9600 [RET]
---CONNECT TAPE---
(attach tape and wait for LED to turn off)
TAPE - NO ERR
X> T
LOAD/DUMP/COMP? C
BAUD RATE? 9600
---CONNECT TAPE---
(attach tape and wait for LED to turn off)
TAPE - NO ERR
0 COMP ERRORS
X> T
LOAD/DUMP/COMP? L
BAUD RATE? 9600
---CONNECT TAPE---
(attach tape and wait for LED to turn off)
TAPE - NO ERR
SYSTEM RESTART
(system restarts)
```

U - VAX UPLOAD. The VAX UPLOAD (U) subcommand copies the current DMC to the MSS maintenance file on a suitably configured VAX. The VAX environment is the same as was defined in the DOWNLOAD (D) subcommand. The procedure for answering the prompts and transferring the data are also the same as those described in the (D) subcommand. Refer to the VAX DOWNLOAD subcommand (D) section for further details.

The U subcommand transfers the entire system back to the VAX for storage. Depending on the VAX configuration, the user can create documentation of the DMC in a CBL block listing and/or can reconfigure the DMC.

The following example shows a display for the U subcommand:

```
X> U
USERID: KRATERG [RET]
PASSWORD: _______ [RET]   (entered but not echoed)
REMOTE PASSWORD: _______ [RET]   (entered but not echoed)
MSS JOB #: MILLXYZ [RET]
MSS MODULE NAME: SEGLINK [RET]
OK? - (Y/N) Y
BAUD RATE? 4800 [RET]
---CONNECT VAX---
(user connects cable to controller)
(Test light displays phase of operation,
    as described with the VAX DOWNLOAD (D) subcommand)
(user reconnects terminal) [RET]
READY? (Y/N)
READY? (Y/N) Y
VAX - NO ERRS
X>
```

Q - QUIT. The QUIT (Q) subcommand exits the XFER command and returns to the main command level. The following example shows a display for the Q subcommand:

```
X> Q
*>
```

MONITOR ERROR MESSAGES

This section defines each error message that the monitor can display, and describes the action required to correct the error. In most cases, the user must re-enter the command from the beginning after receiving an error message.

ALREADY AT EOS - Occurs in the BREAKPOINT command mode. This error message is caused by trying to single step a segment past its last statement using the SINGLE STEP subcommand. This error removes all breakpoints. To regain control, set a new breakpoint with the BREAKPOINT, EXECUTE subcommand.

ALREADY DEFINED - Occurs in the ASSIGN command mode. This error message is caused by using the ASSIGN SYMBOL subcommand to redefine a symbol that already has a value. A symbol must be deleted with the RELEASE subcommand before it can be redefined.

BAD DATA TYPE - Occurs in the DISPLAY AND MODIFY VARIABLE command mode. This error message is caused by using the STATION I/O DISPLAY subcommand to display a data item that is not in the I/O tables. Use the normal DISPLAY VARIABLE subcommand for this variable.

BLOCK UNDEFINED - Occurs in the CBL BLOCK LIBRARY command mode. This error message indicated that a block specified during the DISPLAY CBL BLOCKS subcommand does not exist in the CBL library. When this error occurs, display the entire list of CBL block names (by hitting [RET] at the "DISPLAY?" prompt) to find out if the requested block was included in the DMC configuration. If not shown, the only way to include the missing block is to reconfigure on the VAX.

BRKPNT ABORTED - Occurs in the BREAKPOINT command mode. This error message is caused by [RET] input before the system reaches a breakpoint. Use the EXECUTE BREAKPOINT subcommand to set the breakpoint again.

BRKPNT INACTIVE - Occurs in the BREAKPOINT command mode. This error message is caused by a SINGLE STEP SEGMENT subcommand when the system is not at a breakpoint, Use the EXECUTE BREAKPOINT subcommand to set the first breakpoint, then use the SINGLE STEP SEGMENT subcommand.

BUS HOG - Caused by an L-bus time-out, which occurs when trying to access memory or I/O that is not present on the bus. Refer to the "SYSTEM SELF-TEST** section for more information.

DATA IN EEPROM - Caused by any command that tries to modify non-volatile memory, which is in EEPROM. To modify this memory, first load the system into RAM or BRAM.

ERROR-ON LINE - Indicates that the attempted command must be executed with the controller off-line, Use the ONLINE STATUS command, CONTROLLER STATUS subcommand to take the controller off-line, then reissue the failed command.

INSERT ABORT - Occurs in the TARGET EDITOR command mode. The error message is caused when the user aborts the INSERT INSTRUCTION subcommand. When this message prints, no change has been made to the segment.

INVALID BLOCK # - Occurs in the TARGET EDITOR command mode during the MODIFY LINE and INSERT INSTRUCTION subcommands. This error message indicates that the block number specified is smaller than the previous block number or is larger than the next block number. To determine an appropriate number, check the surrounding block numbers. A **[RET]** following the error message aborts the subcommand.

INVALID LINE NO. - Occurs in the TARGET EDITOR command mode during the DISPLAY LINE, DISPLAY NEXT LINE, and DISPLAY PREVIOUS LINE subcommands. The error message indicates an invalid line number request. The DISPLAY NEXT LINE and DISPLAY PREVIOUS LINE subcommands display the error message when used after reaching the segment end or beginning. The DISPLAY LINE subcommand displays the error if entered after the end of the segment marker, or if the line requested is 0. To position the editor at the segment end, use the BLOCK DISPLAY subcommand for block 9999.

INVALID PASSWORD - This error occurs when the user attempts an operation without the correct access level. Use the **PASSCODE** command to obtain the proper access level, then reissue the command.

INVALID REQUEST - Occurs in the TARGET EDITOR command mode when the MODIFY LINE subcommand is used to change an EOS, BLK, or **STO** instruction. The EOS instruction can never be altered. The user must delete and insert the BLK and **STO** instructions to modify their current operands because they are multiple line instructions.

INVALID TYPE - Occurs in the TARGET EDITOR command mode when the MODIFY LINE or the INSERT INSTRUCTION subcommand is used to define a parameter with a bad data type. The error message indicates a logical value used where a data variable *was* expected, or vice versa. Check the data type required for the block, then enter the appropriate data type.

I/O INIT FAILURE - Occurs at system restart or when the system is placed on-line. The error message indicates that an HMHA I/O channel is incorrectly initialized. Refer to the "SYSTEM SELF-TEST" for more information to recover from this error.

MEMORY WRITE ERR - Occurs during the TRANSFER SYSTEM command mode when the memory it referenced is not RAM or EEPROM. The write operation cannot complete. To correct this error, verify that the target memory is available and that all the board strappings are correct. Use the BREAKPOINT command, TEST MEMORY subcommand to determine the memory type and location.

NO 8087 FOR SEG - Occurs in the TARGET EDITOR command mode during the MODIFY LINE and INSERT INSTRUCTION subcommands. This error message indicates that a block requires a real data type parameter, but the segment cannot access the 8087. The **segment** receives 8087 access at DMC configuration time. The system must be edited and reconfigured on the VAX before the segment accepts the CBL block.

NO 8087 AVAILABLE - Occurs when the user tries to display or modify a real data type variable, but the entire system was created without 8087 support. To display or modify real data, the system must be configured with the 8087 option enabled.

NO FORCED POINTS - Occurs in the FORCE LOGICAL SIGNALS command mode. This error message indicates that there are no forced **logicals** in the system when the DISPLAY LOGICAL, NEXT FORCED STATE, or PREVIOUS FORCED STATE subcommand is executed. (All forced points are cleared when going on-line.)

NO ROOM - Occurs in the TARGET EDITOR command mode. This error message indicates that no room exists in the CBL code table during the INSERT INSTRUCTION subcommand. The user can recover space for the table by deleting lines of code from any segment. The table can be expanded only by reconfiguring the DMC. Refer to the LOOK COMMAND for information about the CBL code table current usage.

NO SUCH CHANNEL - Occurs in the I/O command mode during the DISPLAY CHANNEL INFORMATION subcommand, and in the ONLINE STATUS command mode during the I/O CHANNEL STATUS subcommand. The error message indicates that the specified channel does not exist. To correct the error, use the **display** subcommand for all channels in either command to determine what channels exist on the system. (The channels available on a system are determined at DMC configuration time.)

NO SUCH RACK - Occurs in the I/O command mode during the MODIFY RACK INFORMATION subcommand. The message indicates that a rack within a station does not exist. Use the STATION DISPLAY subcommand to list the set of racks defined for this station. (Racks are defined at DMC configuration time.)

NO SUCH SEGMENT - Occurs in the ONLINE command mode during the SEGMENT STATUS CONTROL subcommand. This error message indicates that a requested segment does not exist. Enter the DISPLAY CURRENT ONLINE STATUS subcommand to get the complete list of segments. (All segment names are defined at DMC configuration time.)

NO SUCH STATION - Occurs in the I/O command mode during the STATION DISPLAY subcommand. The message indicates that the requested station does not exist. Use the DISPLAY CHANNEL INFORMATION subcommand to list all defined stations within the system. **(All** stations are defined at DMC configuration time.)

NO USER MONITOR - Is displayed when the USER MONITOR command is called, but a user monitor segment was not defined at DMC configuration time.

SYMBOL UNDEFINED - Occurs in the command mode when the user requests a symbol or address that does not exist. Use the ASSIGN command to check if a symbolic name is defined or not. Verify that a default name is valid by checking the size limitations of each table. Also verify that the table and data type are correct and that the name is preceded by a "."

TABLE OVERFLOW - Occurs in the TARGET EDITOR command mode during the INSERT INSTRUCTION subcommmand. The error message indicates that the user creates a new logical I/O reference, but there is no space in the logical I/O group table. To expand this table, the system must be reconfigured on the VAX.

The error also occurs during the ASSIGN command mode during the ASSIGN SYMBOL subcommand when the symbol table runs out of room. Symbols can be released to recover some room in this table. Use the LOOK command to find out how full the symbol table is. To expand this table, the system must be reconfigured on the VAX.

TAPE - BAD CSUM - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. This message is displayed after a tape load or compare operation if the checksum character disagrees with the data on the tape. To correct the error, the user can recreate the tape. However, the system that was loaded may be causing the problem.

TAPE - DATA ERR - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. This message is displayed after a tape load or compare operation when the system cannot find the start of the tape's message marker. Verify that a new tape is not being used. Also, the tape may be defective or the wrong baud rate set on the tape handler.

TAPE - FRAME ERR - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. This message is displayed if a HMPH framing error occurs in any tape operation. To correct the error, try the command again. Also, check the hardware manuals for more details on framing errors.

TAPE - LOST LINE - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. The message is displayed after a tape command if the cable between the tape unit and the HMPH was disconnected during the middle of a tape operation. Check the cable and retry the operation.

TAPE - MEMORY ERR - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. The message indicates that the memory to be transferred does not exist. If the message occurs on a load, all the memory does not exist in the system. Refer to the memory error message list above for more details on recovery.

TAPE - TIME OUT - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. This message occurs if the cable between the tape and the HMPH is not connected within 60 seconds, or if the tape does not respond within 30 seconds after the operation starts. If the tape was not connected in time, reissue the command. If the error message occurs in the middle of the operation, verify that the tape is not jammed or broken.

TAPE - PARITY ERR - Occurs in the TRANSFER SYSTEM command during the STR-Link II OR III TAPE BACKUP subcommand. This message is displayed if the HMPH detected a parity error from the tape unit. Try to do the tape operation again.

UNDEFINED - Occurs in the TARGET EDITOR COMMAND during the INSERT INSTRUCTION and MODIFY LINE subcommands. This message indicates that the new instruction does not include a valid elementary instruction. Refer to the TARGET EDITOR command section to find the list of valid instructions. Note that an error occurs if there is other than 1 blank space between the 2 parts of the instruction.

VAX - DATA ERR - Occurs in the TRANSFER SYSTEM command during a VAX transfer when the checksum is invalid. The error message indicates an error in the data transmission, or that the file on which the information is working may be damaged.

VAX - LOGIN ERR - Occurs in the TRANSFER SYSTEM command during a VAX transfer when the user input is invalid. The error message indicates either an invalid USERNAME or PASSWORD. Verify that the log in information is correct, then retry the command.

VAX - MEMORY ERR - Occurs in the TRANSFER SYSTEM command when a VAX subcommand cannot access the memory required to execute the operation. Refer to the TAPE - MEMORY ERR section for more details.

VAX - MSS ERR - Occurs in the TRANSFER SYSTEM command when a VAX subcommand cannot locate the job or module specified in the parameters. Verify that the JOB name and MODULE name are correct, then retry the command.

VAX - NO ANSWER - Occurs in the TRANSFER SYSTEM command when a VAX subconunand cannot access the VAX. Either the line is not connected to the VAX or the VAX is not responding. Verify that the VAX is up and that the HMPH and the VAX are connected correctly.

VAX - TIME OUT - Occurs in the TRANSFER SYSTEM command when the VAX times out during the middle of a VAX transfer subcommand. The VAX subcommand waits from 1 to 2 minutes for the VAX to respond before it times out. Verify that the VAX is up and running and that the line is connected.

WRONG! - The universal error message generated by an invalid user input. The most common cause of this error is an invalid command or subcommand. Verify that the input provided is correct and try again.

NOTES

-- APPENDIX A --
PRINTED WIRING BOARD DESCRIPTIONS

This appendix lists and describes DIRECTO-MATIC Plus boards and modules currently associated with the Distributed Micro Controller. Boards are catalogued with a "DS3800" prefix, and are grouped into 2 categories: "controller" boards associated with the DMC operation, and "I/O" boards that interface the DMC and various types of control signals. Each category lists the boards and modules alphabetically, except that daughter boards directly follow the mother board.

The "DS3810" and "DS3815" prefixes represent general purpose boards with added components that redefine the boards as modules. These added components include special combinations of pluggable chips or PROM chips containing product firmware. The "DS3820" prefix indicates several boards configured into a module.

Each description contains only the board's or module's main features. For more detailed information, refer to the individual documentation for the particular board or module.

CONTROLLER BOARDS

HACA The DS3800HACA Address Customizing board is an optional board that provides user address customization in I/O slave stations that use the R-bus. Inputs consist of address lines and group/byte/word control lines from boards such as the HPBC and HSHA. HACA outputs are the select inputs for the actual I/O boards, such as HLIC and NADA. The HACA contains wire-wrap pins for connecting address lines to up to 18 selected I/O cards in an electronics module. The board contains no active electronic components.

HACB The DS3800HACB Address Customizing board contains wire wrap pins for connecting address lines to up to 8 selected I/O cards in a field termination I/O module. The HACB has contains no active electronic components, but contains Berg jumpers to define Sense, Freeze, Reset and I/O or Slave mode logic for the module.

HADA The DS3800HADA L-bus to F-bus Decoder board provides an interface between the L-bus and the F-bus. The HADA is used in the DMC for both module and local I/O. The board maps an F-bus into a 200H page region of the L-bus I/O map.

HBDA The DS3800HBDA Buffer Decoder board is an interface for address, data, and control signals between an F-bus source and I/O boards on the R-bus. The data bus can access 7 logic inputs and outputs. The HBDA is strapped into 1 of 8 address ranges, and can handle up to 16 work addresses in the range.

HCMA The DS3800HCMA RS-232 Dual Communications Interface board is an 8085 microprocessor-based, dual channel, serial communication subsystem. The HCMA contains two RS-232C full or half duplex serial communication channels that can be programmed for baud rates up to 19.2 K bits/second, synchronous or asynchronous. The HCMA interfaces to the L-bus via a 2K byte dual-ported RAM. The board accepts either a 2K or 4K byte EPROM to program communication protocol.

HIMA The DS3800HIMA CPL Isolation Module board is used in the Link Isolation Module (DS3820LIMA). The HIMA includes a 28VDC to 5VDC isolated power supply, and contains a single channel RS-422 channel which operates in a full duplex mode. Signals are optically coupled for isolation, with common mode signal isolation up to 200 volts. The HIMA can drive up to 4000 feet of twisted-pair cable.

HIOB The DS3800HIOB I/O Data Slave board provides a serial interface, logic input buffers, logic output latches, a +105VDC logic input circuit, a relay output circuit, and programmable address and termination Berg jumpers. The board functions as a data link interface between the CPL and the DS3827 or DS3828 SILTRON Power Conversion Modules, providing a means to coordinate several PCMs from a higher level controller. The HIOB is treated as a slave station on the CPL.

Controller Boards (contd.)

HIOH The DS3800HIOH I/O Processor with daughter board DS3800DIOH provides microprocessor control of I/O signals between a DS3828 SILTRON Power Conversion Module and a DS3800HXRB Receiver board for the C-bus interface. The HIOH/DIOH performs analog prescaling, A/D conversion, D/A conversion, digital filtering, drive diagnostics, and simple sequencing. The board supplies 2 relays, jumper selectable analog I/O (including 2 reference outputs), and 6 indicating LEDs. The HIOH contains circuitry specific to the SILTRON module.

HLSC The DS3800HLSC Data Link Slave Control A board provides data control functions and clocking for a CSH slave station. The HLSC is used with the HLSD, the HLXA, and the HLXB boards.

HLSD The DS3800HLSD Data Link Slave Control B board provides an interface with I/O boards for a CSH slave station. The HLSD creates an F-bus, generates and checks parity, and processes interrupts. Berg jumpers select the rising or falling edge for interrupt sampling. The HLSD is used with the HLSC, the HLXA, and the HLXB boards.

HLXA The DS3800HLXA High Speed Data Link Repeater board with an HLXB board provides an interface to the CSH for either a CSH master or slave station. The HLXA receiver function converts incoming message bits to a TTL compatible bit stream and extracts clocking; the transmitter function sends CSH messages back onto the loop. The HLXA contains both primary and redundant coaxial cable connections and circuitry to select the active cable. Relays enable the coax signals to bypass the HLXA circuitry upon power loss or manual switching.

HLXB The DS3800HLXB High Speed Data Link Interface board is used with the HLXA board to a provide an interface to the CSH for both master and slave stations. The HLXB: 1) identifies the station address, 2) delineates the bit stream into separate message fields, 3) inverts and checks the CRC characters for data receipt, and 4) creates new CRC characters for data transmission.

HMHA The DS3800HMHA Master Highway board is an 8085 microprocessor-based, dual channel serial communication subsystem with high speed DMA. The HMHA provides an interface with the Control Party Line, and can be used as either a master (DS3815PCLA) or a slave (DS3815PRBA), depending on the on-board firmware. The board contains 2 SDLC/RS-422 party line serial channels that can be programmed for baud rates up to 48K bits/second, synchronous. The HMHA interfaces to the L-bus via a 2K byte dual-ported RAM, and accepts either a 2K or 4K byte EPROM to program communication protocol.

Controller Boards (contd.)

HMPH The DS3800HMPH Microprocessor is the CPU board for the DMC and contains the product software in PROMs. The CPU can be expanded to handle floating point numbers by including an 8087 coprocessor on the DMPC daughter board. The HMPH consists of an 8086 microprocessor, programmable communications interface, a programmable interval timer, a programmable 8-level interrupt controller, mapped I/O, and on-board system control consisting of a bus hog timer, stall timer, and software reset. The HMPH includes 32K of product code in four 2764 PROMS and 8K of static RAM. All the memory is used by the DMC system and must be present. The HMPH has dedicated 8086 bus, L-bus, and E-bus signal formats.

DMPC The DS3800DMPC Microprocessor Daughter Board is the DS3800HMPH Microprocessor's daughter board. It provides an E-bus and contains the 8087 coprocessor, when required.

HNMA The DS3800HNMA Null Modem board provides two 25-pin female connectors for RS-232 connections between the HMPH microprocessor and a data terminal or pocket terminal. The HNMA is also used to connect terminals with the HCMA board.

HPBC The DS3800HPBC Parallel Line Buffer board interfaces address, data, and control signals between an F-bus source and I/O boards. The HPBC provides 2 modes of operation: the "buffer drive" mode passes l-for-l all of the F-level signals to the R-level, making a new F-level driving source; "buffer decode" mode provides full decoding for up to 32 word addresses. The board also checks all of the R-level signals for shorts to P5 or DCOM.

HRMA The DS3800HRMA Dual Ported 16K RAM board provides 16K bytes of static RAM. It is dual-ported to the L-bus and the E-Bus, and addressable by either byte or word. Data is lost when power is turned off.

HRMB The DSS3800HRMB Dual Ported 8K Byte RAM board is a battery backed-up RAM board, dual-ported to the L-bus and the E-bus, with either byte or word access. Data in memory is valid up to 8 days after power is removed. The HRMB contains 8192 bytes of CMOS RAM, an on-board battery, and provision for connecting an external battery. On-board circuitry recharges the battery.

HSHA The DS3800HSHA Slave Highway Interface board links the CPL and I/O boards, either R-bus or F-bus. The board operates as a CPL slave station when connected to a CPL master station via twisted-pair cable. The HSHA contains an 8085 microprocessor, ROM, RAM, and a Synchronous Data Link Controller (SDLC).

DSHA The DS3800DSHA daughter board provides LEDs, thumbwheel switches, and toggle switches for monitoring data flow through the HSHA mother board and for performing off-line testing of any I/O boards connected to the HSHA. Berg jumpers select scan range, decode or buffer mode, memory type, terminate, and slave address.

Controller Boards (contd.)

HUMA The DS38HUMA Universal Memory board contains 4 "quads", each providing four 28-pin IC sockets for memory devices. The board supports either 64K RAM or 32K EEPROM on the DMC for use on the L-bus. (The board will also support PROM, which is not used on the DMC.) Both the memory segment and the base address are selected independently for each quad via Berg jumper connections.

HXCA The DS3800HXCA C-bus Master board provides an interface for 250 bytes of I/O via the C-bus. Multiple HXCA boards can be used to drive several C-bus lines.

HXMA The DS3800HXMA Dedicated Dual-Ported Memory board allows communication between the L-bus and the F-bus through 32 bytes of on-board RAM. Berg jumpers select memory addresses and boundaries, and internal or external bus-swap control. The HXMA is frequently used with the HXRA board to communicate with a Series Six Programmable Controller.

HXPC The DS3800HXPC CPU Expander board serves as an interrupt expander for the DMC. The board provides 4K RAM, 8 additional interrupt levels, 6 counter/timers, and two 8-bit logic I/O ports. Berg jumpers provide memory map options.

HXRA The DS3800HXRA Receiver with the DS3800DXRA daughter board converts C-bus differential signals into F-bus singular signals, and F-bus singular signals into differential C-bus signals. The boards create write and read strobes along with address and data, enabling transfer of information to I/O boards along the F-bus. The HXRA/DXRA allows page addressing with Berg jumper options. The boards can be linked with other HXRA/DXRA's in a chain up to 50 feet long, or up to 500 feet through an HXTA board. The HXRA/DXRA provides a compatibility with the Series Six local I/O chain and is functionally similar to the Series Six "parallel receiver".

HXRB The DS3800HXRB C-bus Receiver is designed specifically as an interface between the DS3800HIOH board, which requires single-ended data, and the Series Six system, which requires differential data. Both boards are integrated into the SILTRON Power Conversion Module.

HXTA The DS3800HXTA Transmitter is a parallel driver board for the C-bus. It drives signals over cable lines greater than 50 feet, and up to 500 feet. An NPSR board must be used with the HXTA, and the HXTA must be driven by an HXRA. Locally (within 50 ft), the HXTA drives up to 10 HXRA boards, and is needed when more than 10 HXRAs are used on the same chain. The HXTA/NPSR is functionally similar to the Series Six "parallel transmitter".

NPSR The DS3800NPSR Transmitter Power Supply generates +5VDC isolated power from an incoming +28VDC. The NPSR is required by the HXTA board.

I/O BOARDS

HLIA The DS3800HLIA Logic +5VDC Input board accepts 16 logic level signals as non-isolated inputs to the R-bus.

HLIC The DS3800HLIC Logic Input board provides 5VDC isolated signal conditioning for 5VDC inputs. The 8 front-edge inputs, after conditioning, are available as 8 logic output signals for direct use, or can be read via the R-bus interface.

HLOA The DS3800HLOA Logic Output board provides buffering, parity check, and LED indication of 16 logic inputs to produce 16 non-isolated outputs. The outputs can be enabled (from the high impedance state) by address decoding logic from the R-bus.

HLOB The DS3800HLOB Logic Output provides R-bus interface and buffering for 8 non-isolated logic outputs. Discrete logic inputs are also available to trigger the outputs.

HPCA The DS3800HPCA Pulse Counter board is a 16-bit binary up/down pulse counter. The HPCA accepts logic level, +5VDC pulses through an externally controllable gate at rates up to 200kHz with a count range of 0 to 65,535, then transmits the total to the controller via the R-bus. The board can be externally cleared and preset to an externally supplied count value. Contents of counter are buffered and can be read by the R-bus.

HRRB The DS3800HRRB Reed Relay Output board interfaces either direct data inputs or R-bus data inputs (as determined by mode selection) and 8 relay contact outputs. These relay outputs are normally open reed relay contacts used as isolated outputs. The HRRB can be used in low level or analog circuits, or in 115AC circuits.

HSCA The DS3800HSCA Signal Conditioning Input board is a 115VAC or 105VDC isolated signal conditioning board consisting of 8 high-level optically-isolated input circuits, logic output drivers, R-bus interface, and a bus isolation circuit.

HSCB The DS3800HSCB Signal Conditioning Input board optically isolates and conditions eight 125VDC inputs to provide 8 low-level logic output signals. The 8 inputs can be gated to the R-bus.

HSCC The DS3800HSCC Signal Conditioning Input board conditions sixteen 28VDC inputs to provide R-bus input and 16 low-level, non-isolated logic output signals.

HSCD The DS3800HSCD Signal Conditioning board conditions sixteen 105VDC inputs to provide 16 non-isolated outputs. The board contains logic output drivers, a R-bus interface, and a bus isolation circuit.

I/O Boards (contd.)

HSCE The DS3800HSCE Signal Conditioning board conditions eight 105VDC inputs to provide 8 non-isolated outputs. The HSCE includes 8 logic output drivers and an R-bus interface.

HSCF The DS3800HSCF Signal Conditioning board is a 28VDC non-isolated signal input conditioning board. The HSCF includes 8 logic output drivers and an R-bus interface.

HSDA The DS3800HSDA Solenoid Driver board provides 4 fused 115/230VAC solenoid output signals from either direct data inputs or R-bus data lines (as determined by mode selection). The outputs are isolated from logic level signals, with one side of each triac switch connected in common. The switches are rated for 1 amp maximum continuous, fused at 1.5 amps. The board contains 4 status LEDs.

HSDC The DS3800HSDC Solenoid Driver board is the same as the HSDA, except that the HSDC does not include fuses.

HSDD The DS3800HSDD Solenoid Driver board provides 8 fused 115/230VAC solenoid output signals from either direct data inputs or R-bus data lines (as determined by mode selection). The board contains 8 status LED's, 8 removable fuses, and 8 neon lamps that indicate blown fuses. The outputs are isolated from logic-level signals with 1 side of each triac switch connected common in the 2 groups of 4. Each output is rated at .75 amperes.

NADA The DS3800NADA 12-bit A/D Converter board contains 8 differential, filtered analog input channels. The board continually converts the inputs to 12-bit (2 byte) digital values, 12.5 volts full scale. The most recent converted value for each channel is accessible via the R-bus.

NTEA The DS3800NTEA Termination board contains standard terminators for all F-bus signals. The board buffers 8 signals as they pass through.

NDAC The DS3800NDAC Digital to Analog Converter provides 4 addressable analog outputs from 12-bit bipolar digital inputs. Each output is loaded by sending 2 consecutive bytes of data to the board via the R-bus, the first byte being the 4 least significant bits required by the D/A, and the second byte being the 8 most significant bits. The outputs are bipolar, $\pm$10V full scale.

MODULES

EEP The DS3810EEP consists of an HUMA board with 4 quads of EEPROM, totalling 32K bytes of EEPROM.

LIMA The DS3820LIMA Link Isolation Module functions to terminate and isolate signals on the CPL. The LIMA consists of a HIMA board and a standard telephone jack in a special electronics module designed for panel mounting and terminal board connection of cable signals.

PCLA The DS3815PCLA CPL Master Station module consists of an HMHA board plus a PROM chip that enables the board to act as a CPL master station. The PCLA interfaces remote I/O devices via HSHA boards to the DMC via dual twisted-pair termination lines.

PRBA The DS3815PRBA consists of an HMHA board plus a PROM chip that enables the board to act as a CPL slave.

PTCA The DS3815PTCA Terminal Interface board includes the HCMA and terminal communication firmware. The PTCA provides an interface for an L-bus host processor to communicate with a variety of terminals.

RAM The DS3810RAM consists of an HUMA board with 4 quads of RAM, for a total of 32K bytes of RAM.

STMA The DS3800STMA C-bus Termination module consists of the DS3800NTBD Termination board and mounting hardware. The module provides an interface between the round cable used for the Series Six I/O chain and the ribbon cable used with the C-bus in DIRECTO-MATIC Plus hardware configurations. The STMA also provides line-terminating resistors for the C-bus and a connector that can be used for the PDT when on a Series Six I/O chain.

-- APPENDIX B --
GLOSSARY OF TERMS

Application Software - Customer-specific software resident within the Distributed Micro Controller (either in EEPROM and BRAM or RAM) used to control the customer's process. The Control Block Language displayed on the user terminal statement format is a visible representation of application software.

Backplane - Signifies connectors, busses, and other components located to the rear of an electronics module. Printed wiring boards plug into the backplane of an electronics module.

Base System Memory - The portion of memory present in all DMC systems. The base system memory includes the CBL application program, the CBL library, the .C table, the symbol table, and the configuration tables which define additional parameters of the specific target system. (A portion of the CBL library and/or the symbol table may have been assigned to other memory blocks in specific system configurations.)

Bit - Binary digit. A unit of digital information represented as either 0 or 1, corresponding to the opening or closing of a switch. The DMC stores and manipulates data as series of bits.

Board - See Printed Wiring Board.

BRAM (Battery-Backed Random Access Memory) - Semiconductor read/write memory within the DMC, used in cases where dynamic variables must be retained in event of power failure. In case of power failure, the memory is kept powered-up and therfore valid by a board-mounted battery.

Byte - Eight contiguous bits of memory. Logic variables are stored in the byte array.

C-bus - A bus structure in which signals are in differential form (1 positive and 1 negative). This bus is compatible with the Series Six I/O system "local I/O chain".

Card Rack - A mechanical assembly for holding and interconnecting DMC printed wiring boards.

CBL (Control Block Language) - A General Electric control programming language where traditional control functions, such as integration, are treated as functional blocks.

Checksum - In the DMC, this is the sum of the contents of all of the bytes within a specified block of memory. A checksum is used for verifying memory contents.

Channel - A port through which a specified type of I/O signals is transferred to and from the DMC. A channel defines the block of I/O that is transferred at a specific time based on a CBL block within a segment. Channels are defined as part of the configuring process.

Configuring - The process of creating the resources and memory allocations of a DMC system. Configuring requires a special software system available at the General Electric factory, Salem, Virginia, (can also be installed on customer VAX computer systems). Configuring establishes the CBL library (selects the specific control blocks to be incorporated in the target DMC) and the configuration tables which define and allocate memory space to items such as: the types and quantity of I/O; the number, type, and priority of segments; the size of internal tables, including the .V, .C, .L; symbol tables; and a number of other parameters.

Constant - A numerical value that remains the same during operation. This value is initialized at power-up and stored in the .C table.

Control Block - See CBL.

Control Party Line (CPL) - A bit serial multi-drop communications link connecting a DMC with remote input/output boards.

Control Signal Highway (CSH) - A high speed communications link connecting a control computer to remote input/output modules. A DMC can be tied into the CSH as a "virtual" I/O module.

Default Name - See Signal Address.

E-bus (Extended Bus) - Frontplane data bus used in certain cases for communications between a DMC processor and other modules. This bus is created on the DMPC board and is typically used for communications between the HMPH board and a dual-ported memory as an interface to the Control Signal Highway.

EEPROM - Similar to PROM, but can be altered. Each write operation to an EEPROM takes 10 milliseconds. During this time, the memory cannnot be read. In the DMC, EEPROM is used to save the application code between power failures.

Electronics Module - A module consisting of a card rack, specific printed wiring boards, and (when appropriate1 product software. There are 3 types of electronics modules: the Tiny Electronics Module (TEM), the Small Electronics Module (SEM), and the Medium Electronics Module (MEM).

F-bus (Full Bus) - DMC I/O bus containing all address, data, and control lines. This bus is used frequently between modules and must be decoded to the R-bus to drive I/O boards.

Field Termination Module - An assembly containing up to 32 I/O boards in 4 rows. The design provides heavy-duty terminal boards and wire troughs between I/O boards for simplifying the running and terminating of I/O cables in the field.

Firmware - Software that exists in non-volatile, read-only memory, typically in PROM. Firmware, unlike software, is not generally user-changeable. DMC product software is example of firmware.

Frontplane - Signifies connectors, switches, LEDs, and other components located on the front edges of boards in an electronics module. For instance, the IMOK light is located on the frontplane of a DMC module.

Group - An addressable group of 8 consecutive physical inputs or outputs (called points) that generally exist on one printed wiring board. Most boards contain one group of 8 points, although some can handle 16 points (2 groups). When the DMC addresses a group, it is actually addressing a single board.

Hexadecimal - A numerical system commonly used in computer systems with a base of 16 rather than a base of 10 (as in the decimal system). Single digits represent the decimal numbers 1 through 15 as follows: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10.

IMOK Light - A yellow LED located on the front edge of a board. When the IMOK light is turned off, a fault has been detected on the board.

Input Signal (I) - A signal received by the DMC, but originating externally.

Interrupt - A hardware initiated modification to the system operation. Normal operation continues from the point at which the interrupt occurred as soon as the DMC services the interrupt. The source of the interrupt can either be external or internal.

L-bus (Local Bus) - Bus contained on the backplane of a DMC module. Through this bus, the microprocessor board communicates with memory, module I/O, communication boards, and similar units.

Local I/O - I/O boards located in the same panel lineup as the DMC module (within 50 feet of the module), but not located within the module. I/O within the DMC is considered "module" I/O, not "local" I/O. (See Module I/O.) .

Logic Symbol - A one to 8 character alphanumeric name, the first character of which must be alphabetic. This name is associated with an element of .L table by its offset from the start of the table.

Local Terminal - A portable terminal used to inspect, change, or troubleshoot DMC hardware and software systems. A variety of general purpose ASCII RS-232 terminals can be used, including a CRT with keyboard, teleprinter (KSR type with keyboard and printer), or a number of handheld terminals.

MEM (Medium Electronics Module) - A DIRECTO-MATIC Plus electronics module with a capacity of 24 boards (2 rows of 12 boards each).

Module I/O - I/O boards located within the DMC electronics module.

MSS - A special software system used at the General Electric factory, Salem, Virginia, to support the configuring and programming of DMC systems. The system operates on a VAX computer. Selected portions of this software system may be configured for use on a customer computer. See the TRANSFER SYSTEM (X) command, VAX DOWNLOAD (D) subcommand in Chapter 3.

Off-Line - A condition in which the DMC does not process output signals and does not execute the application program.

On-Line - The normally-operating condition of the DMC during which the DMC processes input and output signals and executes the application program.

Output Signal (O) - A Signal originating within the DMC and destined for external hardware such as solenoids, indicator lights, displays, and motors (e.g. speed references).

Physical Address - See Signal Address.

Point - A single input or output signal wired into an I/O board and individually addressable by the DMC. An I/O board can typically handle a group of 8 points, although some boards can handle 16 points.

Point Address - The single digit identifier within a signal address that specifies the point where the signal originates or terminates. (See "Signal Address".)

Primary Memory - The DMC's working memory, which includes storage for the CBL application program, the CBL library, the constant table, the symbol table, configuration tables, etc. The monitor's TRANSFER SYSTEM (X) command provides the capability to copy primary memory to secondary memory for back-up, and to load primary memory from secondary memory to restore a system. The primary memory can exist as 1 to four 32K blocks containing a "base system memory" (always present) and optional blocks for an extended symbol table and extended CBL library. (See "Secondary Memory".)

Product Software - Software resident within a General Electric product (in PROM) that controls the operation of that product only, not a customer application. Product software is essentially transparent to the customer and cannot be modified by the customer. The DMC operating system is an example of product software, which is also called firmware.

PROM (Programmable Read Only Memory) - Semiconductor memory used within the DMC to store product software or firmware. Unlike BRAM, PROM is not user-changeable. (See BRAM.)

R-bus (Reduced Bus) - A partial decode of the F-bus used to communicate with I/O boards.

RAM (Random Access Memory) - Semiconductor read-write memory that is not protected by a battery back-up system. (See BRAM.)

Relay Ladder Diagram - A method of displaying application software (relay logic) in visual form, using contacts, coils, and horizontal and vertical connections. The DMC software elementary shows strings in relay ladder diagram form.

Remote I/O - I/O boards located outside of the panel line-up containing the DMC module (more than 300 feet from the module). Communication with remote I/O boards requires a Control Party Line with isolation modules (LIMAs).

Rung - A portion of a relay ladder diagram terminated by a 'coil".

Secondary Memory - Back-up memory boards. The secondary memory is mapped in one 32K block of address space, which is assigned to 1 card slot. This allows individual HUMA (EEPROM) boards to be successively plugged-in so that each of the "primary memory" blocks can be copied to individual HUMA (EEPROM) boards. See "Primary Memory",

Segment - A functional portion of the CBL application program. A segment is executed as an entity and can include the 3 phases typically found in the sweep of a programmable controller (i.e. read inputs, execute logic, send outputs). A DMC includes a multiplicity of segments that can be executed concurrently. Segment types include periodic (executed on a fixed scan cycle time basis), interrupt (executed based on an external event), and initial (executed on power applied). The DMC product executive handles control of segment execution, which is based on definitions of type, priority, and scan cycle established at configuration. The DMC monitor's SEGMENT (S) command can change the scan cycle of periodic segments.

SEM (Small Electronics Module) - A DIRECTO-MATIC Plus electronics module with a capacity of 12 boards (2 rows of 6 boards each).

Signed Integer Constant - A one to 5 digit decimal number located in the range of - 32767 to + 32767.

Signal Address - The identification of a control signal within the system. This identification is related to the physical address of the signal. For an Input or Output signal, the signal address is the hardware address of the input or output wire carrying the signal, and is in the .Issrb or .Ossrb form. For an internal signal, the signal address is the physical location within a DMC register table, and is in the form .Vnnnnt, .Cnnnnt, or .Lnnnn. The signal address is also referred to as "default name" or "physical address". (See "Signal Name".)

Signal Name - A symbolic name assigned to a signal. The signal name is usually representative of the function of the signal. A signal can be identified by the signal name or the equivalent signal address. The name can be from 1 to 7 characters long (but must not start with "."). Most signal names are assigned when the DMC system *is* designed, but new names can be assigned through the ASSIGN SYMBOLS (A) command in the monitor. The DMC's symbol table stores the correlation between the signal name and the signal address. The signal name is also referred to as "symbol". See "Signal Address". The capacity for signal name Storage is optional.

SILTRON Module - A General Electric power conversion module (DS3827 and DS3828) that uses silicon controlled rectifiers (SCRs) to convert 3-phase AC power to adjustable DC power, which in turn is used to power a DC motor. The module also provides excitation for the motor and includes electronics for the SCR firing circuit, protective circuitry, drive regulators, and single drive logic.

Station - Used in signal addressing to specify a combination of up to 16 racks. A station can be a physical station or a virtual station. Since a rack is typically a card rack, a physical station is usually a combination of card racks, sometimes in a single panel enclosure. A virtual station exists only within DMC software. A single DMC can address up to 254 stations (hexadecimal 01 to FE).

Station Address - A 2-character identifier (hexadecimal number) in the signal address that specifies a particular station.

Sweep A sweep can involve 3 phases: (1) reading input signals from the input boards, (2) processing the CBL application programs (for example, integrating a signal opening or closing software contacts), and (3) sending output signals from the DMC to output boards (for example, closing a physical relay. The DMC organizes the CBL application program into segments, which can include phases 1, 2, and 3, or only phase 2.

Symbol - See Signal Name.

TEM (Tiny Electronics Module) - A DIRECTO-MATIC Plus electronics module with a capacity of 6 single boards in a row.

Unsigned Integer Constant - A one to 5 digit decimal number in the range 0 to 32767.

User Terminal - See Local Terminal.

Virtual Station - A station that exists only within the DMC memory, and is used for transferring data between DMCs or from the DMC to a process computer.

Word - Sixteen contiguous bits (2 bytes) of memory.

-- APPENDIX C --
DMC MEMORY AND I/O LAYOUT

The HMPH microprocessor can access up to 16 segments of memory with 64K bytes per segment. The DMC memory is mapped to define certain sections of that memory to specific functions, as shown in Figure C-l on the following page.

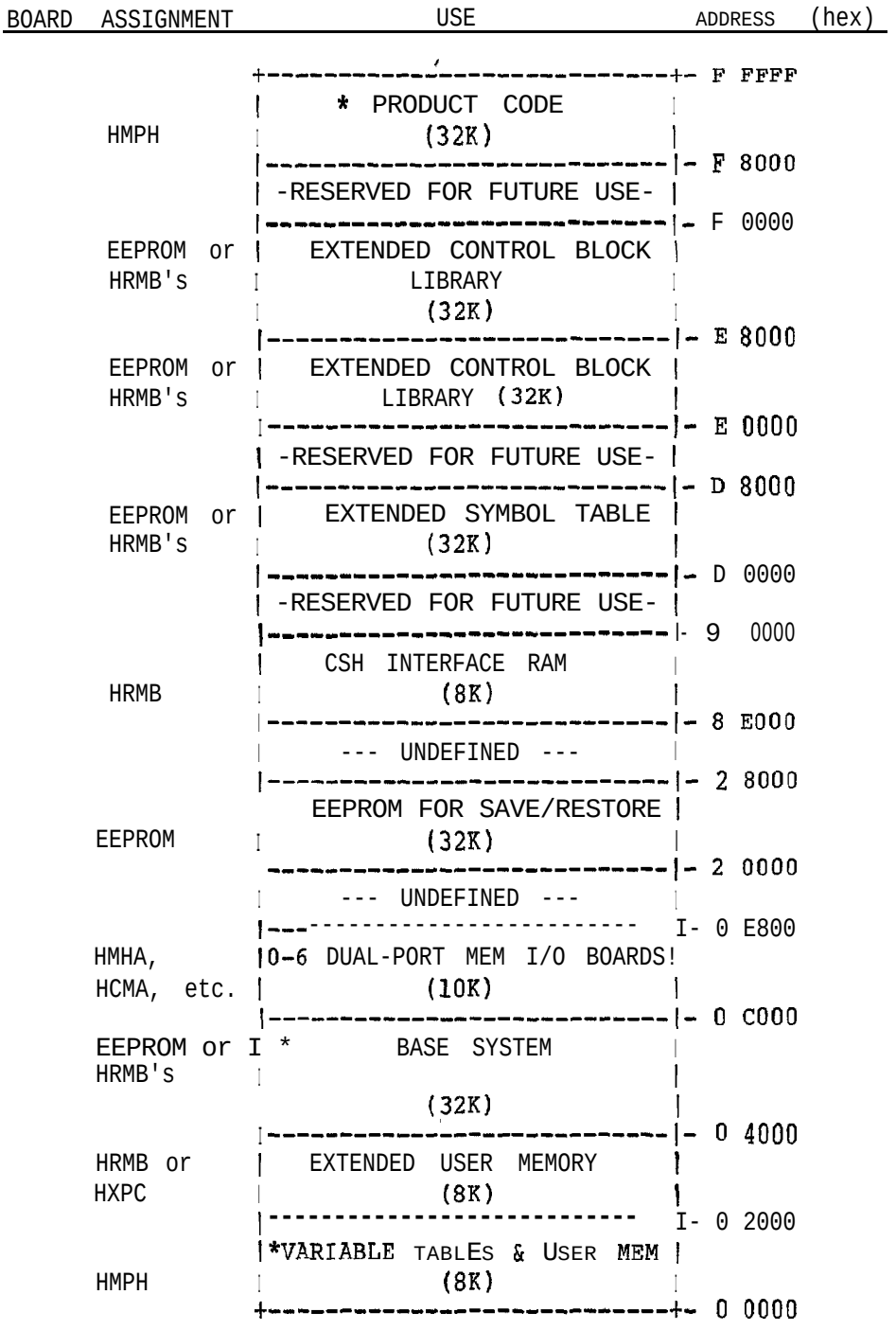


Figure C-l
DMC MEMORY MAP

* INCLUDED IN ALL SYSTEMS

The HMPH microprocessor can access up to 64K of 8-bit I/O registers. The processor I/O is mapped to define certain address groups to specific boards, as shown in Figure C-2.

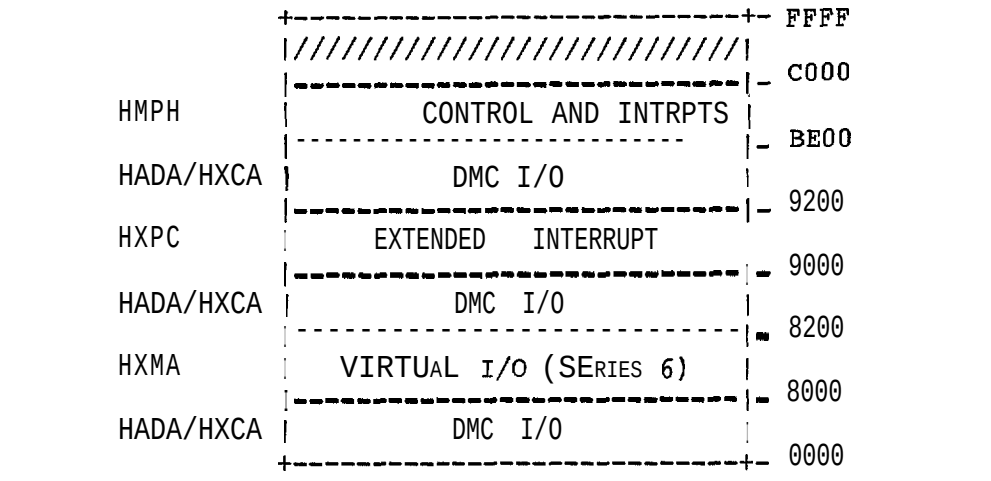


Figure C-2
PROCESSOR I/O

NOTES

-- APPENDIX D --
BREAKPOINT COMMAND

The BREAKPOINT **(B)** command is a combination of debug and diagnostic aids intended primarily for factory use during system design. This command checks for checksum errors, displays and modifies absolute memory, tests memory, and sets breakpoints in the interpreter.

****
* CAUTION *
****

THE USER CAN DESTROY THE SYSTEM WITH THE BREAKPOINT COMMAND BY MODIFYING ABSOLUTE SYSTEM MEMORY. THIS COMMAND SHOULD BE USED ONLY BY THOSE THOROUGHLY FAMILIAR WITH THE DMC'S INTERNAL SOFTWARE ARCHITECTURE AND WITH **8086/8087** MACHINE LANGUAGE.

The following sequence initiates the BREAKPOINT command from the main command level:

```
*> B
B>
```

H – HELP INFORMATION Subcommand

The HELP INFORMATION **(H)** subcommand displays the following list of subcommands available for use via the BREAKPOINT command:

```
B> H
B -- BASE
C -- CHECKSUM
D -- DISPLAY
E -- EXECUTE
H -- HELP
M -- MODIFY
Q -- QUIT
S -- SINGLE STEP
T -- TEST MEMORY
B>
```

B – BASE SEGMENT ASSIGNMENT Subcommand

The BASE SEGMENT ASSIGNMENT **(B)** subcommand sets the segment address base for the DISPLAY and MODIFY memory subcommands. A segment is a physical memory address divided by 16. For example, the absolute address F8000 equals segment address F800. The MODIFY and DISPLAY subcommands calculate the address to display by adding the segment base-times-16 to the offset. The system sets the base to 0000 when it restarts.

The following example shows a display for the B subcommand:

```
B> B
BASE? FECO [RET]
BASE = FECO
```

C – CHECKSUM ERROR COUNT Subcommand

The CHECKSUM ERROR COUNT **(C)** subcommand displays the number of application or checksum errors that have occurred since the system reboot. This subcommand is used when memory board failures are suspected. The DMC continually computes checksums, and the checksum value should always be set to 0000. Any non-zero value indicates an illegal memory modification. Note that the MODIFY MEMORY **(M)** subcommand (described later) can cause a checksum error if it modifies memory that needs to be saved during power loss.

The following example shows a display for the C subcommand:

```
B> C
CSUM ERROR = 0000
B>
```

D – DISPLAY ABSOLUTE MEMORY Subcommand

The DISPLAY ABSOLUTE MEMORY **(D)** subcommand displays the absolute memory in either word, byte or ASCII form. The DMC calculates the displayed address from the current base address and the offset specified. The offset value is entered as a hexadecimal number; the length value is entered as a decimal number, but can be entered in hexadecimal if preceded by a **"#"** sign. The D subcommand uses the last offset entered if a [RET] is entered after the prompt. The subcommand defaults to a length of **"1"** and to byte format if no response is entered for these prompts.

The D subcommand uses a 60-character line if the terminal type is set to "scroll" mode (CRT mode), which allows 16 bytes of memory to be displayed per line. The subcommand displays 16 characters of output if the terminal type is set at "line" mode (handheld mode), which prints 2 bytes (1 word) per line.

The display line does not display multiple lines of a repeated word or byte. Instead, the D subcommand displays a "*" to indicate repeated data. When the data value changes, the output continues. This subcommand always provides the first and the last line of data.

When using the ASCII format, the DMC inserts a "." in the display line as a position holder for any ASCII character that is not printable.

The following example shows a display for the D subcommand:

```
B> D
OFFSET? 5400
OFFSET = 5400
LENGTH? 100 [RET]
FORMAT (W,A,B)? W [RET]
FECO: 5400 1E50 5488 D7FB 0000 CODE OOOA 0000 0000
FECO: 5410 2000 8000 0000 FFOO 0000 002F 2436 0000
*
FECO: 54C0 0000 0000 0000 0000
B> D
OFFSET? [RET]
OFFSET = 5400
LENGTH? 100
FORMAT (W, A, B)? [RET]
FECO: 5400 50 1E 88 54 FB D7 00 00 DE CO OA 00 00 00 00 00
FECO: 5410 00 20 00 80 00 00 00 FF 00 00 2F 00 36 24 00 00
*
FECO: 5460 00 00 00 00
B>D
OFFSET? 0 [RET]
OFFSET = 0000
LENGTH? 50
FORMAT (W,A,B)? A [RET]
FECO: 0000 . . . . . .
FECO: 0010 . . . . . E . 7 N O 808
FECO: 0020 7 A V A I L A B L E . . . . .
FECO: 0030 L 0
B>
```

E - EXECUTE BREAKPOINT Subconunand

The EXECUTE BREAKPOINT **(E)** subcommand sets a breakpoint in the middle of a segment and stops its execution. At that time, the user can examine the variables and can continue the segment at another location. If the breakpoint is set in a "periodic" segment, the subcommand provides the option to stop just that segment or all segments. If the breakpoint is set in an "interrupt" segment, however, all segments stop. The interpreter can only be stopped after the completion of a code instruction.

The user specifies the breakpoint by segment name and line number, using the same line number as the editor. The breakpoint follows the line number specified. To abort the breakpoint, enter a **[RET]** before the breakpoint is reached. The DMC displays the line number of the current line when the breakpoint is reached.
The segment remains halted until the operator enters another breakpoint or an E subcommand with no breakpoint address. The user can leave the BREAKPOINT command with the breakpoint outstanding, but must reenter the BREAKPOINT command at some later time to enable the segment to execute properly.

The only way to clear an outstanding breakpoint without specifying another breakpoint is to enter the E subcommand without specifying a line number for the breakpoint address. The only breakpoint in the system at any given time is the last breakpoint specified by the E or the SINGLE STEP **(S)** subcommand (described later).

The segment ID can be defaulted to the last segment given. This is same default segment name that the editor uses.

The following example shows a display for the E subcommand:

```
B> E
SEGMENT? USRSEG2 [RET]
LINE NUMBER? 1
STOP ALL? Y
BRKPNT LINE 2
B> E
SEGMENT? [RET]
LINE NUMBER? 1 [RET]
STOP ALL? Y
[RET]
BRKPNT ABORTED
B> E
SEGMENT? [RET]
LINE NO.? [RET]
B>
```

M – MODIFY ABSOLUTE MEMORY Subcommand

The MODIFY ABSOLUTE MEMORY **(M)** subcommand allows the user to modify bytes of absolute memory. The DMC calculates the memory address to modify from the segment base and offset. It then modifies the memory 1 byte at a time, and displays the next byte immediately after the first one changes. A **[RET]** entered as the first character of the new data terminates the subcommand. If a blank space precedes the **[RET]** entry, the current value remains unchanged and the next byte is displayed.

The M subcommand requires Level 2 access for 3 reasons: **(1)** using the M subcommand can destroy the application checksum, **(2)** attempted access to non-existent memory generates bus hog errors, and **(3)** modifying EEPROM with this subcommand can destroy the system. The user must take care to guard against these errors.

Note that the DMC generates a bus hog message if the modified memory does not exist on the system. Only valid memory should be examined.

The following example shows a display for the M subcommand:

```
B> M
OFFSET? 2000 [RET]
0000:2000 = 00-12 [RET]
0000:2001 = 00-__ [RET]
0000:2002 = 00-34 [RET]
0000:2002 = 00- [RET]
B>
```

S – SINGLE STEP SEGMENT Subcommand

The SINGLE STEP SEGMENT **(S)** subcommand is used with the EXECUTE **(E)** subcommand to breakpoint the interpreter. The S subcommand allows the next interpreter instruction to execute and then stop. All segments remain stopped after the S subcommand if this option was chosen on the initial EXECUTE **(E)** subcommand.

The system must be currently at a breakpoint when the S subcommand is entered, or the DMC displays an error message. Single stepping past the end of segment aborts the single step and continues the segment. The BREAKPOINT EXECUTE subcommand clears the final breakpoint. The S subcommand always leaves the segment halted, but can be aborted by entering a **[RET].**

NOTE: Be sure to use the E subcommand to clear the last breakpoint at the end of the debug session.

The following example shows a display for the S subcommand:

```
B> S
BRKPNT LINE 2
B> S
BRKPNT LINE 3
B> S
BRKPNT LINE 6
B> S
ALREADY AT EOS
B> S
BRKPNT NOT ACTIVE
B>
```

T – TEST MEMORY Subcommand

The TEST MEMORY (T) subcommand performs 2 functions: it determines what type of memory resides at a particular address, and it performs a nondestructive RAM test. This subcommand executes only with the controller off-line.

The T subcommand provides the physical address and memory type for the entire address space, and only prints a message when the memory type changes. With this subcommand, the DMC checks every 2K of memory for its type. When it detects RAM, the DMC checks every memory location for an error.
Note that some I/O boards with dual ported memory can fail the test when their control words are modified. This failure does not indicate that the I/O board is defective. All other RAM addresses (other than dual ported memory) should pass the memory test, If they fail, replace the board and conduct more extensive memory tests on that board.

The following example shows a display for the T subcommand:

```
B> T
00000 = RAM
02000 = NONE
04000 = EEPROM
0C000 = RAM
0C003 = RAM-BAD
0C004 = RAM
0D000 = NONE
F8000 = PROM
B>
```

Q - QUIT Subcommand

The QUIT (Q) subcommand exits the BREAKPOINT command and returns to the main command level. The following example shows a display for the Q subcommand:

```
B> Q
*>
```

NOTES

-- APPENDIX E --
LOCAL TERMINAL CHARACTERISTICS AND CONNECTIONS

LOCAL TERMINAL CHARACTERISTICS

Most general purpose CRTs and teleprinters suit the DMC for local terminal functions. The suitable terminals require the following characteristics:

- RS-232C connection
- Speed: 110, 300, 600, 1200, 2400, 4800, or 9600 baud
- ASCII characters and code: conventional alphabetic, numeric, and punctuation marks required. Special control characters are not required, but Control-X, Control-R, and Control-Z (ASCII CAN, DC2, and SUB respectively) are useful.
- Character: 7 bits with even parity, or 8 bits with no parity, 1 stop bit (2 for 110 baud)
- Display: One line of 16 characters, minimum. An 80 column by 24 line display is required for some functions.
- Duplex (or "echo-plex") operation, i.e. characters entered on the keyboard are sent to the DMC and not directly displayed; the displayed characters are received from the DMC.

TYPICAL LOCAL TERMINAL CONNECTIONS

The local terminal connector, located on the DMC's microprocessor board (HMPH), follows the RS-232 standard, Type D, in a Data Terminal Equipment (DTE) configuration. It incorporates a DB25P (male plug) connector.

The configuration for the general purpose connection suits many conventional terminals. The connection depends primarily on whether the device is wired as DTE or DCE (Data Communication Equipment) and on the type of connector used. Table E-l and the indicated figures illustrate typical local terminal connections.

Table E-l
TYPICAL LOCAL TERMINAL CONNECTIONS

DEVICE	CONFIGURATION	FIGURE	DEVICE* CONNECTOR	COMMENTS
DEC VT-100	DTE	E-l	DB25P	Cord mounted connector
HP 2621A	DTE	E-l	DB25P	
Beehive - DM1S	DTE	E-l	DB25S	
Lear Siegler - ADM 3A	DTE	E-l	DB25S	
STR-Link III	DCE	E-2	DB25S	
"103" type Modem	DCE	E-2	DB25S	
Texas Instruments 700	DTE	E-l	DB25P	Cord mounted connector
Termiflex HT/11	DTE	E-3	DA15P	Special connection

* In the device connector identification, the suffix "P" indicates a male plug, "S" indicates a female socket.

Figure E-l
DMC CONNECTION TO A "DTE" TYPE DEVICE**

SIGNAL DESCRIPTION RELATIVE TO HMPH	HMPH CONNECTOR: DB25P CIRCUIT	PIN		DEVICE CONNECTOR: DB25P - TYPICAL PIN	CIRCUIT
Transmitted Data	BA	2		2	BA
Received Data	BB	3		3	BB
Request to Send	CA	4		4	CA
Clear to Send	CB	5		5	CB
Carrier Detect	CF	8		8	CF
Data Set Ready	cc	6		6	cc
Data Terminal Ready	CD	20		20	CD
Signal Ground	AB	7		7	AB

**NOTE: This connection typifies commercially available "null modems". Most null modems contain two DB25S connectors, which are female sockets. When a terminal contains a DB25S connector, it may require a "gender changer". Such a device includes 2 identical connectors (DB25S or DB25P) with straight through wiring.

Figure E-2
DMC CONNECTION TO A "DCE" TYPE DEVICE

SIGNAL DESCRIPTION RELATIVE TO HMPH	HMPH CONNECTOR: DB25P CIRCUIT	PIN		DEVICE CONNECTOR: DB25S - TYPICAL PIN	CIRCUIT
Transmitted Data	BA	2	→	2	BA
Received Data	BB	3	←	3	BB
Request to Send	CA	4	-	4	CA
Clear to Send	CB	5	←	5	CB
Carrier Detect	CF	8	←	8	CF
Data Set Ready	CC	6	←	6	CC
Data Terminal Ready	CD	20	→	20	CD
Signal Ground	AB	7	—	7	AB

Figure E-3
DMC CONNECTION TO A TERMIFLEX HT/11

SIGNAL DESCRIPTION RELATIVE TO HMPH	HMPH CONNECTOR: DB25P CIRCUIT*	PIN		DEVICE CONNECTOR: DA15P - TYPICAL PIN	CIRCUIT	
Transmitted Data	BA	2	→	2	BA	
Received Data	BB	3	←	3	BB	
Request to Send	CA	4				
Clear to Send	CB	5	←			
Carrier Detect	CF	8		8		common
Data Set Ready	CC	6	←	6		common
Data Terminal Ready	CD	20				
Signal Ground	AB	7		7	AB	
				13		+5V power
	DCOM			14		+5V power
				15		+5V power
	+5V					

* NOTE: "DCOM" and "+5V" refer to power that is required for the TERMIFLEX unit.

NOTES

GENERAL ELECTRIC COMPANY • DRIVE SYSTEMS OPERATIONS
SALEM, VIRGINIA 24153

GENERAL@ ELECTRIC